

NVIDIA Skills Katalođu

321 Skill'in Tamamı — Türkçe

Açıklamalı Rehber

Kaynak: build.nvidia.com/skills (github.com/NVIDIA/skills)

Bu katalogdaki skill'ler, Claude Code gibi yapay-zekâ kod asistanlarına yüklenen resmi NVIDIA talimat paketleridir: skill'in kendisi iş yapmaz, asistana o konuda doğru kod ve komut yazmayı öğretir.

Hazırlayan: Claude (Fable) — Çağatay için

Tarih: 23 Temmuz 2026

DOCA Ailesi (Bölüm 1/2)

DOCA (Data Center-on-a-Chip Architecture), NVIDIA'nın BlueField DPU'ları için geliştirdiği bir yazılım geliştirme platformudur. BlueField DPU dediğimiz şey, sunucudaki normal ağ kartının içine gömülü, kendi başına çalışabilen küçük bir bilgisayardır (kendi işlemcisi, kendi belleği, kendi işletim sistemi vardır) — yani sunucunun ana "beyninin" yanına eklenmiş, ağ trafiğini ve bazı ağır işleri devralan ikinci bir "yardımcı beyin" gibi düşünülebilir. DOCA, bu yardımcı beyne şifreleme, veri sıkıştırma, ağ trafiği yönlendirme, güvenlik izleme gibi işleri "boşaltarak" (offload ederek) ana sunucu işlemcisini (CPU) rahatlatan programlar yazmayı sağlayan kütüphaneler, komut satırı araçları ve hazır servislerden oluşur. Bunu kullananlar tipik olarak büyük veri merkezleri, bulut sağlayıcıları ve yüksek performanslı ağ/güvenlik altyapısı yazan sistem programcılarıdır — MITAS'ın video/OCR pipeline dünyasından çok farklı, donanımına çok yakın, alt-seviye bir alan. Günlük hayattan bir benzetme: DOCA'yı, bir restoran mutfağına eklenen özel bir "yardımcı istasyon" gibi düşünebilirsin — normalde şef (ana sunucu CPU'su) doğrama, marinasyon, paketleme gibi her işi tek başına yapardı, ama artık bu tekrarlayan/ağır işlerin bir kısmını yardımcı istasyona (BlueField DPU) devrediyor; şef sadece asıl yemeğe, yani uygulamanın kendi mantığına odaklanabiliyor. Bu 30 skill, Claude Code gibi bir yapay-zekâ kod asistanına yüklendiğinde, asistanın bu "yardımcı istasyonu" doğru kurması, programlaması, test etmesi ve arızalandığında tamir etmesi için NVIDIA'nın resmî talimatlarını taşır. Skill'in kendisi hiçbir iş yapmaz — asistan bu konuyu önceden bilmeseyse bile, skill devreye girdiğinde doğru komutları, doğru kod kalıplarını ve doğru hata-ayıklama adımlarını üretebilir hale gelir.

doca-aes-gcm

Ne yapar: Bu skill, BlueField DPU veya ConnectX ağ kartı üzerinde veri şifreleme/şifre çözme işini donanımına "boşaltmak" (offload) için kullanılır. AES-GCM, hem veriyi şifreleyen hem de verinin araya girilip değiştirilmediğini garanti eden bir şifreleme yöntemidir (kimlik doğrulamalı şifreleme — yani sadece gizlilik değil, "bu veri bozulmadı" garantisi de verir). Skill; hangi anahtar boyutlarının donanım tarafından desteklendiğini (128 veya 256 bit — 192 bit desteklenmiyor) kontrol etmeyi, veri boyutunu donanımın sınırlarına göre ayarlamayı ve şifre çözme sırasında "bu veri değiştirilmiş" hatasını doğru yorumlamayı öğretir.

Örnek: Claude'a şöyle dersin: "BlueField kartımda büyük dosyaları AES-256-GCM ile şifrelemek istiyorum, CPU'yu yormadan" — skill devreye girer ve doğru DOCA fonksiyon çağrılarını, anahtar/boyut kontrollerini ve hata ayıklama adımlarını içeren kodu üretir.

doca-argp

Ne yapar: DOCA ile yazılmış programların komut satırı arayüzünü (CLI — programı terminalden --device gibi bayraklarla çalıştırma şekli) doğru kurmayı öğretir. Yeni bir komut satırı parametresi eklemek, mevcut bir örnek programı değiştirmek ya da parametre okuma sırasında oluşan hataları çözmek için kullanılır. NVIDIA'nın standart parametrelerini (--device, --representor, --json, --sdk-log-level gibi) bozmadan genişletmeyi sağlar.

Örnek: Claude'a şöyle dersin: "Bu örnek DOCA programına, kullanıcının bir sayı girebileceği yeni bir --tekrar-sayisi parametresi eklemek istiyorum" — skill devreye girer, DOCA'nın kendi CLI kütüphanesini doğru sırayla (başlat → parametreyi kaydet → başlat → kapat) kullanan kodu üretir.

doca-argus

Ne yapar: DOCA Argus, BlueField DPU üzerinde çalışan hazır bir "güvenlik gözcüsü" konteyner servsidir — sunucuda ve DPU'da şüpheli davranışları, bütünlük ihlallerini izler ve bulguları bir merkezi güvenlik izleme sistemine (SIEM — şirketlerin güvenlik olaylarını topladığı panel, örn. Splunk) iletir. Skill; bu servisi kurmayı, hangi davranışların "şüpheli" sayılacağını ayarlamayı, bulguların nereye gönderileceğini yapılandırmayı ve "konteyner çalışıyor ama hiçbir bulgu gelmiyor" gibi sorunları çözme kapsar.

Örnek: Claude'a şöyle dersin: "BlueField üzerinde Argus'u kurdum ama Splunk'a hiç veri düşmüyor" — skill devreye girer, dört ayarlı yapılandırmayı (tespit kuralları, iletim, örnekleme sıklığı, hangi sunucuların izleneceği) kontrol edip sorunu bulur.

doca-bare-metal-deployment

Ne yapar: Derlenmiş bir DOCA programını, konteyner veya Kubernetes kullanmadan doğrudan donanım üzerinde (host sunucu ya da BlueField'in kendi işlemcisi üzerinde) çalıştırmayı öğretir. BlueField'ı ilk kurulumdan (BFB paketi yükleme) sağlıklı çalışır hale getirmeyi, hangi işlemci çekirdeğinin/hangi kaynağın kullanılacağını ayarlamayı ve donanım seviyesindeki yedi katmanlı hata sınıflandırmasını kapsar.

Örnek: Claude'a şöyle dersin: "Derlediğim bu programı doğrudan BlueField kartı üzerinde, konteyner olmadan çalıştırmak istiyorum" — skill devreye girer, doğru PCI/CPU bağlama ayarlarıyla programı hangi modda (doğrudan, arka plan servisi olarak) başlatacağını gösterir.

doca-bench

Ne yapar: doca_bench adlı, DOCA'nın hızını ölçen genel amaçlı bir performans test aracını çalıştırmayı öğretir. Şifreleme, sıkıştırma, ağ, veri kopyalama gibi farklı DOCA kütüphanelerinin donanım üzerinde ne kadar hızlı çalıştığını (saniyede kaç işlem, gecikme süresi gibi) ölçmeye yarar. Doğru test ayarlarını seçmeyi, ölçüm sonuçlarını yanıltıcı olmadan yorumlamayı (örneğin "ısınma" turlarını sonuca katmamak gibi) da kapsar.

Örnek: Claude'a şöyle dersin: "Bu BlueField kartında şifreleme hızlandırması gerçekten ne kadar performans katıyor, ölçmek istiyorum" — skill devreye girer, doca_bench aracını doğru parametrelerle çalıştırıp güvenilir bir hız raporu üretir.

doca-bench-extension

Ne yapar: doca-bench aracının hazır ölçüm modları yetersiz kaldığında, onun için özel bir "eklenti" (plug-in — programa sonradan takılan ek modül) yazmayı öğretir. Örneğin GPU üzerinde çalışan özel bir iş yükünü ölçmek gibi, standart araçların kapsamadığı senaryolar için kullanılır.

Örnek: Claude'a şöyle dersin: "doca_bench'in hazır modları benim CUDA/GPU iş yükümü ölçmüyor, özel bir ölçüm eklentisi yazmam lazım" — skill devreye girer, NVIDIA'nın referans örneğini (doca_bench_cuda) temel alarak eklentinin nasıl yazılıp derleneceğini ve araca nasıl yükleneceğini gösterir.

doca-bf3-deployment

Ne yapar: BlueField-3 (NVIDIA DPU donanımının 3. nesli) kartını kutudan çıkmış haliyle çalışır hale getirmeyi öğretir — kart üzerine işletim sistemi paketini (BFB) yüklemeyi, host sunucu ile kart arasındaki yönetim bağlantısını kurmayı ve kartın çalışma modunu (bağımsız DPU modu mu, yoksa ayrılmış host/ağ kartı modu mu) seçmeyi kapsar.

Örnek: Claude'a şöyle dersin: "Yeni gelen BlueField-3 kartını host sunucuma taktım, üzerine işletim sistemini nasıl yüklerim" — skill devreye girer, kartı tanıma, BFB paketini yükleme ve kurulumu doğrulama adımlarını sırayla anlatır.

doca-bf4-deployment

Ne yapar: BlueField-4 kartını, yönetim kartı (BMC — sunucu kapalıyken bile uzaktan yönetim sağlayan ayrı bir küçük bilgisayar) üzerinden kurmayı öğretir. UYARI: burada anlatılan bazı işlemler (yazılım/donanım güncellemesi yakma, kartı fabrika ayarlarına döndürme, gücü kesip açma) GERİ ALINAMAZ ve yanlış yapılırsa kartı kalıcı olarak bozabilir — bu yüzden her adımdan önce ne yapılacağı ve etkisi açıkça gösterilir, kullanıcıdan onay istenir.

Örnek: Claude'a şöyle dersin: "BlueField-4 kartının firmware'ini BMC üzerinden güncellemek istiyorum" — skill devreye girer, ama önce riski ve geri dönüşü olmadığını açıkça belirtip onay ister, ardından adımları gösterir.

doca-caps

Ne yapar: doca_caps adlı, hiçbir şeyi değiştirmeyen, sadece "bu makinede DOCA ne görüyor" diye rapor veren salt-okunur bir komut satırı aracını çalıştırmayı öğretir. Hangi ağ kartlarının/DOCA cihazlarının tanındığını, hangi DOCA kütüphanelerinin bu sistemde kullanılabilir olduğunu listelemek için kullanılır.

Örnek: Claude'a şöyle dersin: "Bu sunucuda DOCA hangi kartları ve hangi özellikleri görüyor, kontrol eder misin" — skill devreye girer, doca_caps aracını çalıştırıp cihaz ve kütüphane listesini okunur şekilde sunar.

doca-collectx-deployment

Ne yapar: CollectX (kısaca "clx") adlı bir telemetri toplama sistemini kurmayı öğretir — bu sistem, DPU veya sunucudan çeşitli sayaç/ölçüm verilerini toplayıp Prometheus, Fluent Bit gibi izleme araçlarına aktarır. Skill; hangi verilerin toplanacağını ayarlamayı, toplama servisini çalıştırmayı ve verinin gerçekten dışarıya (izleme panosuna) ulaşmasını sağlamayı kapsar.

Örnek: Claude'a şöyle dersin: "BlueField'daki ağ sayaçlarını Prometheus panosunda görmek istiyorum ama toplama servisi hiç veri göndermiyor" — skill devreye girer, hangi ayarın eksik olduğunu (sağlayıcı bağlantısı, dışa aktarım ayarı) bulup düzeltir.

doca-comch

Ne yapar: Host sunucu ile BlueField DPU'nun kendi işlemcisi arasında, ağ üzerinden değil doğrudan kart bağlantısı (PCIe) üzerinden kısa kontrol mesajları göndermeyi/almayı öğretir ("Comch" = Communication Channel, iletişim kanalı). Kimin sunucu kimin istemci rolünde olacağını, mesajların nasıl gönderilip alınacağını ve bağlantı hatalarını çözmeyi kapsar.

Örnek: Claude'a şöyle dersin: "Host programım ile BlueField üzerindeki programım birbirine küçük kontrol komutları göndersin istiyorum, ağ kartını değil PCIe hattını kullanarak" — skill devreye girer, sunucu/istemci rollerini kurup mesajlaşma kodunu üretir.

doca-comm-channel-admin

Ne yapar: doca_comm_channel_admin adlı, sadece o an açık olan host↔DPU kontrol kanallarını (Comch bağlantılarını) tek seferlik tarayıp listeleyen bir araçtır — hangi sunucu bağlantısının açık olduğunu, kaç istemcinin bağlı olduğunu gösterir. Kanalı yeniden başlatma veya kapatma gibi bir işlevi YOKTUR, sadece "şu an ne var" diye bakar.

Örnek: Claude'a şöyle dersin: "Şu an BlueField üzerinde hangi Comch bağlantıları açık, kaç istemci bağlanmış, bir bakar mısın" — skill devreye girer, aracı çalıştırıp bağlantı tablosunu okunur şekilde sunar.

doca-common

Ne yapar: Bu, diğer tüm DOCA kütüphanelerinin üzerine oturduğu temel yapı taşlarını öğretir — her DOCA programının önce bir "cihaz" bulması, o cihazın hangi özellikleri desteklediğini kontrol etmesi, veri arabelleklerini (buffer — bellekte geçici veri tutulan alan) sıfır-kopyalama ile paylaşması ve iş tamamlanma bildirimlerini takip etmesi gerekir. Bu skill, her DOCA projesinin başında devreye giren "ortak dil"i kapsar.

Örnek: Claude'a şöyle dersin: "DOCA ile yeni bir program yazmaya başlıyorum ama önce cihazı nasıl bulup açacağımı bilmiyorum" — skill devreye girer, her DOCA programının ortak başlangıç iskeletini (cihaz bulma → özellik kontrolü → bellek ayarlama) kurar.

doca-compress

Ne yapar: Veri sıkıştırma/açma işlemini (gzip benzeri DEFLATE yöntemi her iki yönde, LZ4 yöntemi sadece açma yönünde desteklenir) BlueField veya ConnectX donanımına devretmeyi öğretir. Hangi veri boyutunda donanıma sıkıştırma yaptırmanın gerçekten faydalı olduğunu (küçük veriler için CPU ile yapmak daha mantıklı olabilir), bellek ayarlarını ve hataları da kapsar.

Örnek: Claude'a şöyle dersin: "Büyük log dosyalarını sıkıştırırken CPU'yu değil BlueField kartını kullanmak istiyorum" — skill devreye girer, donanım sıkıştırma görevini kuran kodu ve boyut eşiği (küçük dosyalarda offload'a değmeyeceği) uyarısını verir.

doca-container-deployment

Ne yapar: DOCA'nın hazır servis konteynerlerini (Argus, DMS, Firefly gibi paket halinde gelen hazır programlar) BlueField üzerinde konteyner olarak çalıştırmayı öğretir — bir YAML ayar dosyası hazırlayıp BlueField'ın kendi küçük Kubernetes benzeri sistemine (kubelet) bırakmayı, servisin sağlıklı çalışıp çalışmadığını kontrol etmeyi kapsar.

Örnek: Claude'a şöyle dersin: "Bu hazır DOCA servisini BlueField üzerinde konteyner olarak nasıl çalıştırırım" — skill devreye girer, doğru YAML ayar dosyasını nereye bırakacağını ve servisin loglarını nasıl okuyacağını gösterir.

doca-debug

Ne yapar: Herhangi bir DOCA sorununu (derleme hatası, bağlama/link hatası, çalışma zamanı hatası, sessizce çalışmayan bir servis) katman katman çözmeyi öğretir — sorunun kurulum mu, sürüm uyumsuzluğu mu, derleme mi, bağlama mı, çalışma zamanı mı, program mantığı mı, yoksa sürücü seviyesinde mi olduğunu adım adım ayırt etmeyi sağlar. Ayrıca hata ayıklama çıktısını (log seviyesini) nasıl artıracağını da öğretir.

Örnek: Claude'a şöyle dersin: "Programım derlenirken 'undefined reference to doca_flow_init' hatası veriyor, neden?" — skill devreye girer, bu hatanın hangi katmanda (bağlama/link katmanı) olduğunu tespit edip çözüm yolunu gösterir.

doca-devemu

Ne yapar: BlueField üzerinde, host sunucunun "gerçek bir donanım parçası" gibi göreceği sahte (emüle edilmiş) bir PCIe cihazı yaratmayı öğretir — örneğin host'un normal bir ağ kartı ya da disk sürücüsü sandığı, ama aslında arkasında BlueField'ın çalıştırdığı özel bir yazılım olan bir cihaz. Hangi cihaz türünün (genel PCIe, sanal ağ kartı, sanal dosya sistemi) uygun olduğunu seçmeyi kapsar.

Örnek: Claude'a şöyle dersin: "Host sunucusunun normal bir ağ kartı gördüğünü sansın ama aslında arkasında BlueField'ın kendi yazdığım kodu çalışsın istiyorum" — skill devreye girer, hangi emülasyon türünün (sanal ağ kartı) uygun olduğunu seçip kurulum kodunu üretir.

doca-dma

Ne yapar: İki bellek alanı arasında (örneğin host sunucu belleği ile BlueField belleği arasında, ya da iki farklı sunucu arasında) veri kopyalamayı, sunucunun ana işlemcisini (CPU) hiç yormadan yapmayı öğretir (DMA = Direct Memory Access, işlemciyi atlayarak doğrudan bellekten belleğe veri taşıma). Kopyalanacak bölgelerin izinlerini (okuma/yazma) doğru ayarlamayı da kapsar.

Örnek: Claude'a şöyle dersin: "Host sunucudaki büyük bir veri bloğunu, CPU'yu meşgul etmeden BlueField'a kopyalamak istiyorum" — skill devreye girer, doğru bellek izinleriyle donanım destekli kopyalama görevini kuran kodu üretir.

doca-dms

Ne yapar: DOCA Management Service (DMS) adlı, BlueField veya ağ kartlarını uzaktan yönetmeye yarayan bir servisi kurmayı ve kullanmayı öğretir — cihaz ayarlarını okuma/değiştirme, canlı ölçüm akışı almayı kapsar. UYARI: bu servis üzerinden kartı yeniden başlatma, işletim sistemini değiştirme, fabrika ayarlarına döndürme gibi GERİ ALINAMAZ ve üretim sistemini etkileyebilecek işlemler de yapılabilir — bu yüzden bu tür işlemlerden önce hedef cihazın doğrulanması ve kullanıcı onayı şart.

Örnek: Claude'a şöyle dersin: "DMS üzerinden BlueField kartının canlı sıcaklık/performans verilerini izlemek istiyorum" — skill devreye girer, güvenli (salt-okunur) izleme sorgularını kurar; kartı yeniden başlatma gibi riskli bir istek gelirse önce açıkça onay ister.

doca-dpa

Ne yapar: BlueField üzerindeki DPA denilen özel, programlanabilir küçük bir işlemciye (Data Path Accelerator — ağ trafiği yolunda çalışan, çok hızlı ama sınırlı amaçlı bir mini-işlemci) host sunucudan iş göndermeyi öğretir — önceden derlenmiş bir DPA programını yükleme, üzerinde iş parçacıkları (thread) başlatma, sonuçları toplama gibi adımları kapsar.

Örnek: Claude'a şöyle dersin: "Bazı basit hesaplamaları host CPU yerine BlueField'in DPA işlemcisinde çalıştırmak istiyorum" — skill devreye girer, DPA programını yükleyip iş başlatan ve sonuç toplayan host-tarafı kodu üretir.

doca-dpa-hl-tracer

Ne yapar: DPA işlemcisi üzerinde çalışan kodun içinde ne olduğunu (hangi işlem ne zaman başladı/bitti, senkronizasyon noktaları, tamamlanan işler) izlemeye yarayan bir izleme/loglama aracını kullanmayı öğretir — DPA tarafında normalde "görünmez" olan çalışmayı gözlemlenebilir hale getirir.

Örnek: Claude'a şöyle dersin: "DPA üzerindeki kodum yanlış sonuç veriyor ama host tarafında her şey normal görünüyor, DPA'nın içinde ne olduğunu görmem lazım" — skill devreye girer, izleme aracını doğru ayarlarla çalıştırıp DPA'nın iç adımlarını çözümleyen çıktı üretir.

doca-dpdk-bridge

Ne yapar: DPDK denilen (hızlı ağ paketi işleme için yaygın kullanılan, DOCA'dan bağımsız, daha eski bir kütüphane) bir program üzerinde zaten çalışan bir uygulamaya, o uygulamayı sıfırdan yeniden yazmadan DOCA özelliklerini (özellikle donanım destekli paket yönlendirme) eklemeyi öğretir — bir tür köprü görevi görür.

Örnek: Claude'a şöyle dersin: "Elimde DPDK ile yazılmış bir ağ programı var, onu değiştirmeden DOCA'nın hızlı paket yönlendirme özelliğini eklemek istiyorum" — skill devreye girer, DPDK'nın port'unu DOCA cihazına bağlayan köprü kodunu kurar.

doca-erasure-coding

Ne yapar: Veri kaybına karşı koruma sağlayan "hata düzeltme kodlaması" (erasure coding — bir disk/parça bozulsa bile veriyi kurtarmayı sağlayan matematiksel yedekleme yöntemi, RAID-6'ya benzer) işlemini donanıma devretmeyi öğretir. Yedekleme parçalarını oluşturmayı, bir parça bozulduğunda kurtarmayı ve sadece değişen kısmı güncellemeyi kapsar. Bu bir ağ özelliği değil, depolama dayanıklılığı özelliğidir — basit veri çoğaltmanın (replikasyon) yerine geçmez.

Örnek: Claude'a şöyle dersin: "Dağıtık depolama sistemimde bir disk bozulduğunda veriyi diğer parçalardan kurtarmak istiyorum, CPU'yu yormadan" — skill devreye girer, donanım destekli kurtarma görevini kuran kodu üretir.

doca-eth

Ne yapar: BlueField veya ConnectX kartı üzerinde ham ağ paketlerini gönderip almayı (bir ağ kartının en temel işi — paket alma/gönderme kuyrukları kurmak) düşük seviyede programlamayı öğretir. Hangi kuyruk türünün uygun olduğunu seçmeyi, paket boyutlarını ayarlamayı kapsar. Not: paketlerin hangi kuyruğa yönlendirileceğini belirleyen kısım (yönlendirme kuralları) ayrı bir skill olan doca-flow'a aittir.

Örnek: Claude'a şöyle dersin: "BlueField kartımda gelen ağ paketlerini doğrudan okuyan bir kuyruk kurmak istiyorum" — skill devreye girer, alım (RX) kuyruğunu kuran ve paketleri okuyan kodu üretir.

doca-firefly

Ne yapar: DOCA Firefly, BlueField üzerinde çok hassas zaman senkronizasyonu (PTP — Precision Time Protocol, sunucuların saatlerini mikrosaniye hassasiyetinde eşitleyen bir protokol; normal NTP'den çok daha hassas) sağlayan hazır bir servis konteynerini kurup çalıştırmayı öğretir. Servisin

altı alt-bileşenini (PTP daemonu, izleme, saat senkronizasyon köprüsü gibi) doğru ayarlamayı ve "senkron görünüyor ama saat ilerlemiyor" gibi sorunları çözmeyi kapsar.

Örnek: Claude'a şöyle dersin: "Sunucularım arasında mikrosaniye hassasiyetinde saat senkronizasyonu lazım, normal NTP yetmiyor" — skill devreye girer, Firefly servisini doğru PTP ayarlarıyla kurar ve senkronizasyon durumunu doğrulamayı gösterir.

doca-flow

Ne yapar: Ağ paketlerinin BlueField/ConnectX donanımı üzerinde hangi kurala göre nereye yönlendirileceğini (örneğin "bu tür paketleri şu porta gönder, şunları at") tanımlamayı öğretir — donanım seviyesinde çok hızlı çalışan bir trafik yönlendirme/filtreleme kural motoru kurmak gibi düşünülebilir. Kuralları tanımlama, test etme ve kaç paketin bu kurallara uyduğunu sayan sayaçları okumayı kapsar. Not: bu iş mutlaka DOCA Flow ile yapılmalı — işletim sisteminin kendi ağ araçlarıyla (tc, iptables gibi) yapılırsa DOCA'nın donanım hızlandırması devre dışı kalır.

Örnek: Claude'a şöyle dersin: "Belirli bir IP'den gelen paketleri BlueField donanımında doğrudan, CPU'ya hiç uğramadan filtrelemek/yönlendirmek istiyorum" — skill devreye girer, doğru eşleştirme/yönlendirme kuralını kuran ve doğrulayan kodu üretir.

doca-flow-dpa-perf

Ne yapar: doca-flow'daki yönlendirme kurallarının, DPA işlemcisi üzerinden ne hızla eklenip/kaldırılabilirdiğini ölçen özel bir performans test aracını çalıştırmayı öğretir — sadece belirli yeni nesil donanımlarda (ConnectX-7 ve üzeri, BlueField-3) kullanılabilir.

Örnek: Claude'a şöyle dersin: "BlueField-3 kartımda saniyede kaç yönlendirme kuralı ekleyip silebiliyorum, DPA üzerinden ölçmek istiyorum" — skill devreye girer, doğru ölçüm ayarlarıyla bu hızı test edip raporlar.

doca-flow-dpa-provider

Ne yapar: doca-flow ile kurulmuş bir yönlendirme kuralını veya kaynağını, BlueField'in DPA işlemcisinin doğrudan erişebileceği bir alana "aktarmayı" öğretir — böylece DPA üzerinde çalışan özel kod, bu kuralın sayaçlarını okuyabilir veya kuralı anlık olarak değiştirebilir, host'a hiç dönmeden.

Örnek: Claude'a şöyle dersin: "DPA üzerindeki kodumun, doca-flow ile kurduğum yönlendirme kuralının sayacını doğrudan okumasını istiyorum" — skill devreye girer, kuralı DPA'ya aktaran adımları sırayla kurar.

doca-flow-grpc-server

Ne yapar: doca-flow'un yönlendirme kurallarını, C++ dışındaki dillerden (Python, Go, Rust, Java gibi) uzaktan kontrol edebilmek için gRPC (ağ üzerinden fonksiyon çağırma protokolü) sunucusu kurmayı öğretir. **ÖNEMLİ GÜVENLİK NOTU:** bu sunucu şu an sadece şifresiz/düz metin (plaintext) bağlantı destekliyor — kimlik doğrulama veya şifreli bağlantı (TLS) yok; bu yüzden güvenilir/izole bir ağ segmentinde kullanılmalı, gerekirse önüne ayrı bir şifreleme katmanı (proxy) konulmalı.

Örnek: Claude'a şöyle dersin: "Python programımdan, BlueField üzerindeki DOCA Flow kurallarını uzaktan yönetmek istiyorum" — skill devreye girer, gRPC sunucusunu kurar ama bunun şifresiz olduğunu ve güvenli ağda kalması gerektiğini açıkça uyarır.

doca-flow-perf

Ne yapar: doca-flow yönlendirme kurallarının host sunucu veya BlueField'in kendi CPU'su üzerinden ne hızla işlendiğini ölçen genel performans aracını (doca_flow_perf) çalıştırmayı öğretir — bir JSON ayar dosyasından test senaryosu seçmeyi, tek seferlik hızlı testten sonra daha kapsamlı tekrarlı teste geçmeyi ve sonuçları güvenilir şekilde yorumlamayı kapsar.

Örnek: Claude'a şöyle dersin: "Bu DOCA Flow kural setinin saniyede kaç kural işleyebildiğini ölçmek istiyorum, CPU üzerinden" — skill devreye girer, uygun JSON test senaryosunu seçip aracı

alıřtırır ve sonucu yorumlar.

DOCA Ailesi (Bölüm 2/2)

DOCA, NVIDIA'nın BlueField DPU'ları (ağ kartı üzerinde çalışan, CPU işlerini üstlenen küçük bir bilgisayar) ve ConnectX ağ kartları için geliştirdiği yazılım platformu — ağ, güvenlik, depolama gibi işleri ana işlemciden alıp bu donanımlara devretmeyi sağlar. Aşağıdaki 30 skill, bu platformun farklı köşelerinde (kütüphaneler, araçlar, disiplinler) doğru kod/komut üretmek için asistana yol gösteren NVIDIA talimat paketleridir; ayrıntılı DOCA tanıtımı Bölüm 1'dedir.

doca-flow-tune

Ne yapar: `doca_flow_tune` aracıyla canlı ya da kaydedilmiş bir doca-flow işlem hattının (ağ paketlerini yönlendiren/filtreleyen kural zinciri) performansını ayarlamaya yardım eder. Beş alt-komutu vardır: `dump` (durumu kaydet), `monitor` (canlı izle), `web` (tarayıcı arayüzü), `analyze` (kaydedilmiş veriyi çevrimdışı incele), `visualize` (mermaid diyagramıyla görselleştir). Kural yerleşimi, kaynak boyutlandırma ya da donanım-hızlandırma modu gibi bir "ayar eksenini" seçilip, kural-kurulum hızı veya arama gecikmesi gibi uygun bir ölçütle değerlendirilir; sonuç CSV/JSON rapor olarak sunulur ya da öneri doğrudan Flow programına uygulanır.

Örnek: Claude'a şöyle dersin: "Flow tablomda kural kurulum hızı çok düşük, `doca_flow_tune` ile analiz et" — skill devreye girer, canlı pipe'in durumunu `dump` ile kaydeder, sonra `analyze` modunda inceleyip hangi ayarın (kural yerleşimi mi, tablo boyutu mu) hızı artıracığını raporlar.

doca-gpi

Ne yapar: GPU-Packet-Initiator (GPU'nun kendi başına ağ paketi başlatması) çalışmasını kurar: bir CUDA çekirdeğinin (GPU üzerinde çalışan paralel hesaplama fonksiyonu) host CPU'yu araya sokmadan doğrudan RDMA (ağ üzerinden CPU müdahalesiz bellek okuma/yazma) kuyruklarını GPU belleğinden sürmesini sağlar. `doca-gpi` ile `doca-gpunctio` arasında hangisinin seçileceğine karar verme, domain/channel nesne modelini kurma, GPU tarafı tanıtıcı (handle) devrini ve `DOCA_ERROR_*` hatalarının ayıklanmasını kapsar.

Örnek: "CUDA kernelim ağ üzerinden RDMA isteği göndersin, CPU'yu hiç karıştırmam" dersin — skill devreye girer, `doca_gpi` domain ve channel nesnelerini kurup GPU belleğini bu domain'e bağlayan adım adım kurulumu üretir.

doca-gpunctio

Ne yapar: CUDA çekirdeğini `doca-eth` ağ kuyruklarına (`doca_gpu_eth_rxq/txq`) bağlayan, GPU başına bir `doca_gpu` bağlamı kuran ve GPU-görünür kuyruğu sürekli boşaltan "kalıcı kernel" (persistent kernel) tasarımı kurar. GPU'nun doğrudan, host CPU'yu atlayarak paket alıp gönderdiği düşük gecikmeli ağ uygulamaları için kullanılır. Donanım yeteneği kontrolü (DOCA sorgusu + `cudaGetDeviceProperties`) ve `cudaMalloc` bellek havuzlarının kaydı da kapsam içindedir.

Örnek: "GPU'da çalışan paket-işleme kernelimi ağ kartına doğrudan bağlamak istiyorum" dersin — skill, `doca_gpu` bağlamını kurup `rxq/txq` kuyruklarını CUDA kerneline bağlayan iskelet kodu ve yetenek-kontrol adımlarını üretir.

doca-gpunctio-ib-write-bw

Ne yapar: `gpunctio_ib_write_bw` adlı bant genişliği (bandwidth) ölçüm aracını kapsar; client+server ikilisi olarak çalışır, sunucu tarafındaki bir CUDA çekirdeği RDMA WRITE isteklerini `doca-gpunctio` üzerinden göndererek GPU'nun sürdürdüğü sürekli yazma hızını ölçer. meson ile kurulu DOCA'ya karşı derlenir; sonucun GPU doyumluğundan mı, CPU sınırından mı, yoksa ağ bağlantısı doyumluğundan mı kaynaklandığını ayırt etmeye yardım eder.

Örnek: "GPU'dan RDMA WRITE ile ne kadar bant genişliği çekebiliyorum, ölç" dersin — skill, aracı meson ile derleyip client/server'ı doğru bayraklarla çalıştırır, sonucu GPU-mu/NIC-mi darboğaz analiziyle sunar.

doca-gpunctio-ib-write-lat

Ne yapar: Bant genişliği yerine gecikme (latency) ölçen kardeş araç: `gpunetio_ib_write_lat`. Ping-pong (karşılıklı gidiş-geliş) düzeninde bir CUDA kernelinin RDMA WRITE isteğini ne kadar hızlı gönderdiğini ölçer; `medyan/p99/jitter` (gecikme dalgalanması) gibi istatistiklerle gerçek-zamanlı kontrol döngüleri için karakterize eder.

Örnek: "Gerçek zamanlı kontrol döngüm için GPU'nun RDMA yazma gecikmesi kaç mikrosaniye, ölçmek istiyorum" dersin — skill aracı derleyip çalıştırır, `half-iter/full-iter` gecikme sütunlarını ve `p99/jitter` değerlerini yorumlar.

doca-hardware-safety

Ne yapar: Bu bir işlevsel araç değil, bir güvenlik disiplini/kontrol-listesi skill'idir: DPU/NIC donanım durumunu değiştiren HERHANGİ bir işlem önerileceği zaman (firmware parametresi yazma, BFB yeniden yükleme — DPU'nun ana yazılım imajı, SR-IOV açma, çekirdek önyükleme parametresi değiştirme, PCIe yeniden bağlama, soğuk yeniden başlatma) devreye girer. Ön-uçuş envanteri, bant-dışı (OOB — ana veri ağından bağımsız yönetim erişimi) erişilebilirlik kontrolü, bakım penceresi, `mlxconfig`'in soğuk güç-döngüsü kuralı, prova ortamında deneme ve geri-alma (rollback) planını zorunlu kılar.

Örnek: "BlueField'in firmware ayarını `mlxconfig` ile değiştirelim" dersin — skill devreye girer, komutu doğrudan vermeden önce envanter/OOB-erişim/bakım-penceresi/rollback-planı adımlarını sırayla sorgulayıp riskli işlemi güvenli bir prosedüre oturtur.

doca-mgmt

Ne yapar: BlueField/ConnectX cihazları üzerinde yönetim-düzlemi (management plane — cihazı yapılandıran, asıl veri trafiğinden ayrı kontrol katmanı) programlamayı kurar: `doca_mgmt_dev_ctx/doca_mgmt_dev_rep_ctx` bağlamları kurma, cihaz yeteneklerini sorgulama, tıkanıklık-kontrolü genel durumunu açıp kapama, dahili bellek (ICM) kotalarını ayarlama ya da `doca_mgmt_raw_cmd` ile doğru yetki kapsamıyla (`CONFIGURATION/DEBUG_READ_ONLY/DEBUG_WRITE/DEBUG_WRITE_FULL`) ham firmware komutu gönderme.

Örnek: "Filo yönetim aracım BlueField cihazlarının kapasite bilgisini toplu sorgulasın" dersin — skill, `doca_mgmt_dev_ctx` kurulumunu ve doğru yetki kapsamlı raw-command çağrısını üretir.

doca-pcc

Ne yapar: Host (ana bilgisayar) tarafından, BlueField DPU üzerine ÖZEL bir Programlanabilir Tıkanıklık Kontrolü (Programmable Congestion Control — ağdaki trafik sıkışıklığını yöneten algoritma) yükleme işini ele alır. Port başına `doca_pcc` bağlamı oluşturma, `dpacc` (DPA derleyicisi) ile derlenmiş bir `doca_pcc_app`'i RoCE (Ethernet üzerinden RDMA) taşıyan porta yükleme, üç eksenli yetenek kontrolü (DOCA sorgusu + DPA işlemcili BlueField + firmware'de özel-PCC yuvası açık) ve hata ayıklamayı kapsar.

Örnek: "Kendi yazdığım tıkanıklık kontrol algoritmasını BlueField'a yükleyip RoCE trafiğinde devreye sokmak istiyorum" dersin — skill, `doca_pcc` bağlamı kurup `dpacc` ile derlenmiş uygulamayı doğru porta yükleyen adımları verir.

doca-pcc-counters

Ne yapar: `pcc_counters.sh` adlı bash betiğini çalıştırmayı kapsar: ConnectX/BlueField cihazındaki sabit firmware/donanım PCC tanılama sayaçlarını (CNP, RTT, WRED-drop gibi tıkanıklık göstergeleri) `mst` aracı ve `mlx5 debugfs diag_cnt` arayüzü üzerinden kurup okur. Betiğin iki pozisyonel argümanı vardır — `set|query` ve bir `mst` cihaz yolu — `alt-komut` ya da `--help` yoktur.

Örnek: "CNP ve RTT sayaçlarım hep sıfır görünüyor, neden" dersin — skill, `pcc_counters.sh`'i doğru `mst` yoluyla önce `set` sonra `query` sırasıyla çalıştırıp sonucu yorumlar.

doca-pcc-ztr-rttcc-algo

Ne yapar: DOCA ile birlikte hazır gelen bir tıkanıklık-kontrol algoritması olan Zero-Touch RoCE RTT-based Congestion Control'ün (RTT = gidiş-dönüş gecikmesi) BlueField-3 DPA üzerinde devreye alınmasını, ayarlanmasını ve değerlendirilmesini kapsar. `doca_pcc_dev_ztr_rttcc_algo`'yu DOCA PCC örneğine bağlama, derleme zamanında varyant seçimi (vanilla/PM/RX-rate/multipath/window-probeless), host tarafından ayarlanan parametreleri ince ayarlama ve algoritma hatalarını teşhis etmeyi içerir.

Örnek: "RoCE-v2 trafiğim sıkışıklıkta doğru yavaşlamıyor, hazır bir CC algoritması dener misin" dersin — skill, DOCA'nın hazır ZTR RTTCC algoritmasını seçilen varyantla derleyip devreye alma adımlarını sunar; kendi algoritmanı yazmadan önce bu "no-config-required" temeli önerir.

doca-programming-guide

Ne yapar: Belirli bir kütüphaneden bağımsız, genel DOCA programlama rehberidir: ilk DOCA uygulamasını hazır bir örnekten (sample) türetme, kanonik `pkg-config doca-{library} + meson` derleme deseni (ya da Rust/Go/Python'dan FFI ile C ABI'ye bağlanma), `cfg-create` → `init` → `start` → `use` → `stop` → `destroy` yaşam döngüsünü izleme, ve `DOCA_ERROR_*` hatalarını `doca_error_get_descr()` ile çözme yollarını gösterir.

Örnek: "İlk DOCA Flow uygulamamı yazacağım, nereden başlarım" dersin — skill, hazır bir örnek DOCA uygulamasını nasıl kopyalayıp değiştireceğini, meson derleme satırını ve yaşam döngüsü adımlarını gösterir.

doca-public-knowledge-map

Ne yapar: Kaynak koduna erişim olmadan DOCA hakkında yetkili bilgiye ulaşmayı sağlar: doğru docs.nvidia.com sayfasını bulma, hangi DOCA kütüphanelerinin hangi sürümde kurulu olduğunu tespit etme, bir örneğin diskteki ya da GitHub'daki yerini bulma, `/opt/mellanox/doca` altındaki disk yolunu çözme, ya da 404 veren/yeniden adlandırılmış bir dokümantasyon URL'sini kurtarma. Bu bir öğretici değil, sorunun türüne göre doğru kaynağa yönlendiren bir "yönlendirme tablosu"dur.

Örnek: "DOCA RDMA hakkında resmi dokümantasyon nerede, bir de kurulu sürümüm ne" dersin — skill, doğru docs.nvidia.com sayfasına yönlendirir ve kurulu sürümü tespit etme komutunu verir.

doca-rdma

Ne yapar: Host, BlueField DPU veya ConnectX NIC üzerinde gerçek DOCA RDMA programlamayı kapsar: `doca_dev` üzerinde RDMA bağlamı kurma, bağlantı yöntemi seçme (RDMA CM, bridge/OOB, ya da gRPC üzerinden `doca_rdma_export()` takası), on bir görev tipinden birini etkinleştirme (Send/Receive, Read/Write, Atomic CmpSwap/FetchAdd gibi), `mmap+RDMA` izinlerini ayarlama. ÖNEMLİ kural: üretilen kod mutlaka `libdoca_rdma`'yı bağlamalı ve `doca_rdma_*` API'sini çağırmalı — ham `libibverbs/librdmacm` kullanıp "DOCA RDMA yaptım" demek kabul edilmez, çünkü o zaman DOCA'nın asıl faydası (ilerleme motoru, görev/olay yaşam döngüsü, donanımlar arası taşınabilirlik) devre dışı kalır.

Örnek: "İki BlueField arasında RDMA ile veri taşıyan bir C programı yaz" dersin — skill, `doca_rdma_*` API'sini kullanan, ham verbs'e düşmeyen bir program iskeleti üretir.

doca-rdmi

Ne yapar: DOCA RDMI (RDMA Initiator — hızlandırıcı taraflı tek-yönlü RDMA başlatıcısı) programlamasını kapsar: `doca-rdmi` ile `doca-rdma` arasında ne zaman hangisinin seçileceğine karar verme, bir `doca_rdmi_connection` ya da `doca_rdmi_poster` kurma, `doca_ctx_start()` öncesi bir `doca_dpa_completion` ya da `doca_verbs_cq` bağlama, bir DPA (Data Path Accelerator — BlueField'in veri yolu hızlandırıcı işlemcisi) çekirdeği için DPA tarafı tanıtıcı (handle) alma.

Örnek: "DPA çekirdeğim kendi başına RDMA yazma isteği göndersin, host'a ihtiyaç duymasın" dersin — skill, `doca_rdmi_poster` kurulumunu ve DPA tarafı handle devrini adım adım anlatır.

doca-rmax

Ne yapar: BlueField DPU veya ConnectX host üzerinde DOCA Rivermax çalışmasını kapsar: zamanlamaya duyarlı IP-üzerinden-medya akışları (SMPTE ST 2110 video/ses yayını, borsa verisi, bilimsel veri akışları gibi) için `doca_rmax_in_stream` (alım) oturumları kurma. **ÖNEMLİ** ön koşul: NVIDIA Rivermax SDK'sının ayrıca kurulu ve geçerli bir lisansının olması gerekir — DOCA, Rivermax'ı içermez, yalnızca onu sarmalar; bu kontrol edilmeden hiçbir DOCA-tarafı kod çalışmaz.

Örnek: "SMPTE ST 2110 video akışını BlueField üzerinden zamanlama hassasiyetiyle almak istiyorum" dersin — skill önce Rivermax SDK+lisans kurulu mu diye sorar, sonra `doca_rmax_in_stream` oturumu kurulumunu ve `doca-eth` kuyruklarıyla eşleşmesini gösterir.

doca-setup

Ne yapar: Kullanıcının iş yükü etrafındaki DOCA ortamını ele alır: kurulumun sağlıklı olduğunu doğrulama, derleme ortamını hazırlama (`pkg-config`, başlık dosyaları, `LD_LIBRARY_PATH`, `hugepages` — büyük bellek sayfaları, `devlink`, temsilör'ler), ortam kaynaklı hataları ayıklama, konteyner mi çıplak-donanım mı dağıtım şekli seçme, ya da DOCA kurulu olmayan bir hostta NGC DOCA konteyner yedek yolunu kullanma. Sistemin şeklini (host x86 / BlueField Arm çıplak-donanım / sadece-DPU / yeni laptop) tespit edip doğru yola yönlendiren bir "ön kapı" niteliğindedir.

Örnek: "Yeni bir BlueField aldım, DOCA kurulumu sağlıklı mı bilmiyorum, ne yapmalıyım" dersin — skill, sistem şeklini tespit edip container mı bare-metal mi gideceğine karar verir, sonra kurulum sağlık kontrolü adımlarını sırayla uygular.

doca-sha

Ne yapar: BlueField DPU veya ConnectX hızlandırıcısı üzerinde SHA-1/SHA-256/SHA-512 (kriptografik özet/hash fonksiyonları) hesaplamasını donanıma yükleme (offload) işini kapsar. Tek seferlik `doca_sha_task_hash` ile artımlı (parça parça) `doca_sha_task_partial_hash` arasında seçim yapma, `doca_sha_cap_*` ile algoritma desteğini ve minimum/maksimum tampon boyutlarını sorgulama, kaynak/hedef `doca_mmap` izinlerini ayarlama.

Örnek: "Birkaç GB'lık bir dosyanın SHA-256 özetini CPU yormadan hesaplamak istiyorum" dersin — skill, `doca_sha_task_hash` çağrısını ve gerekli `mmap` izin ayarlarını içeren kod iskeletini üretir.

doca-sha-offload-engine

Ne yapar: Var olan bir OpenSSL boru hattına, `doca-sha` kütüphanesine karşı yeniden yazmadan, DOCA SHA Offload Engine adlı bir OpenSSL ENGINE eklentisini bağlamayı kapsar. Tek seferlik SHA-1/256/512 (EVP_Digest arayüzü üzerinden) hesaplamalarını DOCA SHA donanımına yönlendirir. Motor yükleme mekaniği (`openssl engine dynamic`, `set_pci_addr` kontrolü, `-engine_impl`), offload'un gerçekten devreye girdiğini kanıtlayan SHA-224 negatif testi (SHA-224 desteklenmediği için hata vermesi beklenir — bu da motorun gerçekten çalıştığını gösterir) ve offload'un CPU'dan hangi mesaj-boyutu eşiğinden sonra daha hızlı olduğu bilgisini kapsar.

Örnek: "Mevcut openssl komutlarımı değiştirmeden SHA hesaplamalarını donanıma yükletmek istiyorum" dersin — skill, `engine dynamic` yükleme komutunu ve gerçekten offload olduğunu doğrulayan SHA-224 testini gösterir.

doca-socket-relay

Ne yapar: Soket-tabanlı (TCP/UDP soketleri kullanan) bir host uygulamasını, kodunu yeniden yazmadan bir BlueField DPU eşine köprülemeyi kapsar. Dağıtım şeklini seçme (`process-ici`, `yan-süreç/sidecar`, ya da BlueField servis konteyneri), host-tarafı soketi ve DPU-tarafı yönlendirme uç noktasını yapılandırma, `bind` → `connect` → `gidiş-dönüş testi` → filo geneli onaylama akışını izleme, ya da takılı/sessiz bir relay'i teşhis etme.

Örnek: "Eski soket tabanlı uygulamamı değiştirmeden BlueField'a taşımak istiyorum" dersin — skill, uygun dağıtım şeklini seçip host-DPU soket köprüsünü kuran yapılandırmayı ve bağlantı testini üretir.

doca-spcx-cc

Ne yapar: `doca_spcx_cc` adlı host-tarafı CLI aracıyla, DPA işlemcili bir BlueField üzerinde canlı bir RDMA/RoCE ağında Programlanabilir Tıkanıklık Kontrolü (SPCX sınıfı) algoritmasını yükleme, parametreleme, başlatma, izleme, durdurmayı kapsar; ya da SPCX'i yerleşik `doca-pcc` yüzeyiyle karşılaştırıp hangisinin seçileceğine karar verir. SPCX, `doca-pcc`'nin yeni-nesil programlanabilir-CC halefi olarak konumlanır.

Örnek: "Kendi RTT-tabanlı tıkanıklık kontrol algoritmamı canlı RoCE ağında denemek istiyorum, SPCX mi PCC mi kullanmalıyım" dersin — skill, SPCX-vs-PCC karar ağacını sunar, sonra `doca_spcx_cc` ile algoritmayı yükleyip canlı metrikleri izleme adımlarını verir.

doca-sta

Ne yapar: BlueField DPU veya ConnectX NIC üzerinde NVMe-over-Fabrics (ağ üzerinden NVMe disk erişimi) depolama-hedef tarafı hızlandırmasını kapsar: RDMA üzerinden hedef-tarafı NVMe-oF veri yolunu hızlandıran bir `doca_sta` bağlamı kurma, yerel NVMe-PCI disklerle desteklenen `doca_sta_subsystem` hedefleri (NQN + namespace'ler) tanımlama, `doca_sta_cap_is_supported` ile cihaz desteğini kontrol etme, bağlantı başına G/Ç kuyruklarını boyutlandırma. Not: bu, NVMe-oF'un initiator/host tarafı DEĞİL, hedef (target) tarafını hızlandırır.

Örnek: "BlueField'ı NVMe-oF depolama hedefi olarak sunmak istiyorum, veri yolunu RDMA ile hızlandır" dersin — skill, `doca_sta_subsystem` tanımlama ve yerel disklerle eşleştirme adımlarını gösterir.

doca-structured-tools-contract

Ne yapar: Başka bir DOCA skill'i "yapılandırılmış aracı tercih et" dediğinde ya da kullanıcı, birden çok manuel komutun toplu olarak vereceği bilgiyi (DOCA ortamı/sürümü/cihazları/yetenekleri/doğrulama/host-vs-DPU durumu) tek seferde istediğinde devreye giren bir sözleşme/kural skill'idir. Host'ta yapılandırılmış araç varsa onun çıktısı tercih edilir; yoksa aynı şema bölümündeki manuel komut zincirine düşülür — hangi yolun izlendiği her zaman raporlanır.

Örnek: "Kurulu DOCA hakkında bana her şeyi tek komutla anlatan bir yol var mı" dersin — skill, host'ta yapılandırılmış (--json veren) araç olup olmadığını kontrol eder, varsa onu çalıştırır, yoksa denk manuel komut zincirini uygular ve hangisini kullandığını söyler.

doca-telemetry

Ne yapar: `doca_dev` üzerinden DOCA donanım-sayaç OLAYLARINI, alan-başına (per-domain) DOCA Telemetry okuyucu kütüphaneleriyle (`doca_telemetry_pcc`, `_dpa`, `_diag`, `_adp_retx`, `_phy`, `_pci`) okumayı kapsar. Bu saf bir "sayaç-OKUYUCU" yüzeyidir — NetFlow/IPFIX gibi bir toplayıcı çerçevesi DEĞİL. Her alan kendi `doca_telemetry_{domain}_*` API'sini sunar: yetenek sorgusu, oluşturma, başlatma, sonra alana özgü okuma/örnekleme.

Örnek: "BlueField'imın PCC donanım sayaçlarını periyodik okumak istiyorum" dersin — skill, `doca_telemetry_pcc` bağlamını kurup periyodik okuma döngüsü kuran kod iskeletini üretir.

doca-telemetry-exporter

Ne yapar: DOCA kullanan bir programdan yapılandırılmış telemetri (sayaç/olay) yayınlamayı kapsar: bir `doca_telemetry_exporter_schema` tanımlama, kaynaklar oluşturma, sayaç/gösterge/olay tiplerinden seçme, limitleri varsaymadan önce yetenek sorgusu çalıştırma, ilk yayından önce şemaları kaydetme, `DOCA_ERROR_*` hatalarını ayıklama.

Örnek: "Kendi DOCA uygulamamdan özel sayaçları dışarıya yayınlamak istiyorum" dersin — skill, `telemetry_exporter_schema` tanımlama ve kayıt sırasını gösteren kod üretir.

doca-telemetry-utils

Ne yapar: `doca_telemetry_utils` adlı host-tarafı CLI aracını çalıştırmayı kapsar: tanımlama-sayaç şemasını keşfetme, sayaç adlarını ikili Data ID'lere çevirme, bir DOCA Telemetry exporter yapılandırmasını taahhüt etmeden önce cihaz-başına sayaç desteğini doğrulama, ya da yakalanmış

bir Data ID'yi ters çözme. Bu, exporter/toplayıcı boru hattını destekleyen bir operatör-tarafı yardımcı araçtır — ne geliştirici-tarafı toplayıcı kütüphane (doca-telemetry) ne de yayıncı kütüphanedir.

Örnek: "Exporter'ım veri gönderiyor ama toplayıcıda hiçbir şey görünmüyor" dersin — skill, doca_telemetry_utils ile hangi sayaçların bu cihazda gerçekten destekleniyor olduğunu ve Data ID eşlemesini kontrol eder.

doca-upgrade

Ne yapar: DOCA sürüm yükseltme ya da düşürme konusunda disiplini kapsar: hostu daha yeni bir DOCA sürümüne taşıma, BlueField BFB'yi (firmware imajı) tazeleme, NGC DOCA konteyner etiketini yükseltme, ya da geri alma. Kural: ÖNCE tespit et → raporla → SOR → ancak ONAY sonrası rehberli yükseltmeye geç. Asla otomatik yükseltme yapılmaz; kurulu sürüm ile mevcut daha yeni sürüm arasındaki farkı raporlayıp kullanıcıdan açık onay bekler.

Örnek: "Daha yeni bir DOCA sürümü var mı, geçsem mi" dersin — skill, kurulu sürümü tespit edip yeni sürümle farkı raporlar, sonra sana sorar "yükseltelim mi?" — otomatik yükseltme yapmaz, açık onayını bekler.

doca-urom

Ne yapar: Host tarafından, HPC/UCX/MPI (yüksek performanslı hesaplama iletişim katmanları) yığını altında doca-urom kütüphanesini kullanarak uzak bellek işlemlerini (put/get/atomic/toplu-işlemler) BlueField DPU'ya yüklemeyi (offload) kapsar. Bir UROM Service bağlamı (doca_urom_service_*) ve bunun üzerinde çalışan Worker bağlamı (doca_urom_worker_*) oluşturma, doca_urom_service_get_plugins_list ile eklentileri keşfetme, tamamlanmaları iletme.

Örnek: "MPI all-reduce işlemim host CPU'yu çok yoruyor, BlueField'a yükleyebilir miyim" dersin — skill, doca_urom_service ve worker bağlamlarını kurup UCX trafiğini BlueField'a taşıyan adımları gösterir.

doca-urom-svc

Ne yapar: BlueField Arm tarafında ÇALIŞAN DOCA UROM Service konteynerini işletmeyi kapsar (host-tarafı doca-urom kütüphanesinin DPU-tarafı karşılığı/yürütücüsü) — NGC konteynerini çekme, UCX bileşen yüzeyini seçme, kuyruk boyutlandırma, Comch (host-DPU kontrol kanalı) eşleştirmesini kurma. GÜVENLİK UYARISI: bu servisin kendi başına erişim kontrolü YOK — yetkilendirme tamamen DOCA Comch eşleştirmesi + RDMA izinlerine dayanıyor; yani Comch eşleştirmesi kurabilen herhangi bir host uzak bellek işlemlerini sürebilir, buna göre izole edilmesi gerekir.

Örnek: "BlueField'da UROM servisini ayağa kaldırmam lazım" dersin — skill, NGC konteynerini çekme ve Comch eşleştirmesini kurma adımlarını verir, ama aynı zamanda bu servisin erişim kontrolü olmadığı için izolasyon gerektiğini vurgular.

doca-verbs

Ne yapar: doca-rdma/doca-eth/doca-rmax gibi üst-seviye DOCA kütüphanelerinin altına inip ham-verbs (RDMA'nın en alt seviye, kütüphaneler tarafından sarmalanan çekirdek arayüzü) "kaçış kapısına" geçmeyi kapsar: DOCA Core içinde QP/CQ/PD/MR/SRQ/AH/CC-group/Ethernet-SQ-RQ (RDMA'nın temel yapı taşları) ilkelerini yönetme, libibverbs kodunu DOCA Core modeline taşıma. ÖNEMLİ UYARI: genel RDMA veri taşıma işleri (send/receive/read/write) için bu skill kapsam DIŞI — kullanıcı üst-seviye API'nin ifade edemediği spesifik bir ham QP/CQ özneliği belirtmediyse doca-rdma'ya yönlendirilmeli; "ham verbs daha az kod" gerekçesi kabul edilmez.

Örnek: "Üst-seviye DOCA RDMA API'sinin desteklemediği özel bir QP özneliğini ayarlamam lazım" dersin — skill, bunun gerçekten doca-verbs kapsamına girip girmediğini önce kontrol eder (genel veri taşıma ise doca-rdma'ya yönlendirir), giriyorsa doca_verbs_query_device ile özneliği sorgulayan kodu üretir.

doca-version

Ne yapar: DOCA sürüm yönetimini kapsar: kurulu sürümü tespit etme, dört-yönlü eşleşmeyi doğrulama (pkg-config doca-common, applications/VERSION, doca_caps --version, ve BlueField'da bfver/mlnx-release arasında), NGC konteyner etiketleri hakkında akıl yürütme, bir yeteneğin kurulu sürümde olup olmadığını arama, ya da derleme-zamanı ile çalışma-zamanı sürüm kayması (drift) teşhisi. Diğer tüm skill'ler sürüm konusunda bu skill'e yönelir, kuralları tekrar tanımlamaz.

Örnek: "Programım derlendi ama ağda hiçbir şey göndermiyor, dokümantasyon bir fonksiyonun var olduğunu söylüyor ama linker bulamıyor" dersin — skill, dört-yönlü sürüm eşleşmesini kontrol edip derleme-zamanı DOCA sürümüyle çalışma-zamanı sürümü arasında bir kayma olup olmadığını teşhis eder.

TAO Toolkit Ailesi (Bölüm 1/2)

TAO ("Train, Adapt, Optimize" — eğit, uyarla, optimize et), NVIDIA'nın görüntü ve video tanıma modellerini kendi verinle sıfırdan kod yazmadan eğitip ince ayar yapmanı sağlayan bir araç seti. Nesne tespiti (bir görüntüde "burada bir araba var, şurada bir insan var" diye kutu çizme), sınıflandırma (bir görüntüye tek bir etiket yapıştırma: "bu ürün sağlam" / "bu ürün hatalı"), segmentasyon (görüntüdeki her pikseli hangi nesneye ait olduğuna göre boyama) gibi işler için kullanılıyor. Günlük hayattan bir benzetme: TAO, hazır bir elbiseyi (önceden büyük veri kümeleriyle eğitilmiş "temel model") alıp kendi bedenine (senin kendi verine, kendi fabrika/ürünlerine) terzi eliyle uyarlamak gibi — sıfırdan kumaş dokuyup elbise dikmek zorunda kalmıyorsun, hazır olanı ölçüne göre kesip biçiyorsun. Aşağıdaki 28 skill, bu terziilik işinin farklı aşamalarını kapsıyor: hangi kumaşı (modeli) seçeceğinden, uyarlamanın nerede terslediğini bulmaya (hata analizi), uyarlanmış elbiseyi nerede deneyeceğine (hangi bilgisayar/bulut platformunda çalıştıracağına) kadar. Önemli bir noktayı baştan netleştirelim: bu skill'lerin hiçbirisi kendisi "iş yapmıyor" — her biri, Claude gibi bir yapay-zekâ kod asistanına "bu işi TAO ile nasıl doğru yaparsın" diye adım adım talimat veren bir rehber. Yani skill'i "çalıştırmıyorsun", skill'i Claude'a okutuyorsun; Claude da o rehbera göre doğru komutu/kodu üretiyor.

tao-analyze-changenet-rca

Ne yapar: ChangeNet adlı bir görsel karşılaştırma modelinin (test edilen bir görüntüyü, "sağlam/referans" olarak bilinen bir görüntüyle karşılaştırıp aradaki farkı bulan model) neden hata yaptığını görsel kanıtlarla araştırıyor. Yani sadece sayılara bakmıyor, gerçek görüntüleri tek tek inceleyip "burada model şunu kaçırmış çünkü ışık farklıydı" gibi somut, görüntüye dayalı sonuçlar çıkarıyor. Bu genelde PCB (baskılı devre kartı) gibi ürünlerin otomatik görsel muayenesinde (AOI — Automated Optical Inspection) kullanılıyor: kamera bir devre kartına bakıyor, model "sağlam mı bozuk mu" diye karar veriyor.

Örnek: Claude'a şöyle dersin: "AOI modelim çok fazla hatalı parçayı sağlam diye geçiriyor, RCA (kök neden analizi) yap." — skill devreye girer, deney klasöründeki eğitim/çıkartım sonuçlarını ve gerçek görüntüleri inceleyip hangi görüntü türlerinde (aydınlatma, açı, lehim türü) modelin sistematik olarak yanlış olduğunu görsel örneklerle raporlar.

tao-analyze-gaps-visual-changenet

Ne yapar: VCN (Visual Component Net) sınıflandırma modelinin sonuçlarını tarayıp, "karar eşiğine" (modelin sağlam/bozuk ayrımını yaptığı sınır çizgi) en yakın ve en riskli örnekleri buluyor. Bir NVIDIA konteynerini doğrudan çalıştırarak bu hesaplamayı yapıyor ve en zayıf örnekleri bir tabloya (parquet dosyası) döküyor — bu tablo sonraki adımda "bu örneklerle benzer daha fazla veri toplayalım" diye kullanılıyor.

Örnek: Claude'a şöyle dersin: "VCN modelimin PASS/NO_PASS sınırındaki zayıf örneklerini bul, augmentasyon (veri artırma) için hedef listesi çıkar." — skill konteyneri çalıştırıp en riskli örnekleri sıralayan bir çıktı dosyası üretir.

tao-analyze-gaps-vlm-bcq

Ne yapar: Bir VLM'nin (Vision-Language Model — görüntüyü görüp bunun hakkında yazıyla cevap verebilen model) evet/hayır tipi sorulara verdiği cevapları gerçek doğru cevaplarla karşılaştırıyor. Yanlış "evet" dediği (false positive) ve yanlış "hayır" dediği (false negative) örnekleri ayıklayıp bir liste hâline getiriyor.

Örnek: Claude'a şöyle dersin: "Bu VLM modelinin evet/hayır tahmin sonuçlarını gerçek etiketlerle karşılaştırmak, hataları çıkar." — skill tahmin dosyasını okuyup yanlış-pozitif ve yanlış-negatif örnekleri ayrı bir dosyaya yazar, böylece bir sonraki aşamada bu hatalara odaklanılabilir.

tao-convert-dataset-format

Ne yapar: TAO DAFT veri kümelerini (TAO'nun kendi veri kümesi biçimi) bir formattan başka bir desteklenen formata çeviriyor. Sadece DAFT formatındaki veriler için geçerli, genel dosya dönüştürme aracı değil.

Örnek: Claude'a şöyle dersin: "Bu DAFT veri kümesini KITTI formatına çevir." — skill tao-daft convert komutunu doğru kaynak/hedef format bayraklarıyla çalıştırır ve sonucu yeni klasöre yazar.

tao-finetune-clip

Ne yapar: CLIP, görüntü ile metni aynı "anlam uzayında" eşleştiren bir modeldir (örneğin "kırmızı araba" yazısıyla kırmızı araba fotoğrafını birbirine yakın konumlandırır). Bu skill CLIP'i kendi görüntü-yazı çiftleriyle ince ayar yapmayı, sıfır-örnekli sınıflandırma (zero-shot — modelin daha önce hiç görmediği bir kategoriyi, sadece o kategorinin adını okuyarak tanıması) yapmayı, görüntülerden sayısal "gömme" (embedding — bir görüntüyü temsil eden sayı dizisi) çıkarmayı ve modeli ONNX/TensorRT gibi hızlı çalışan formatlara dönüştürmeyi kapsıyor.

Örnek: Claude'a şöyle dersin: "Ürün fotoğraflarımla CLIP'i ince ayar yap, sonra TensorRT'ye aktar." — skill eğitim spesifikasyonunu hazırlar, ince ayarı başlatır ve sonunda modeli dağıtıma hazır formata çevirir.

tao-finetune-cosmos-embed

Ne yapar: Cosmos-Embed1, videoyu ve metni ortak bir sayısal uzayda temsil eden bir modeldir. Bu sayede "bu yazıyı anlatan video hangisi" (metinden videoya arama) ya da "bu videoya benzer başka video var mı" (videodan videoya arama) gibi sorular cevaplanabiliyor; ayrıca neredeyse birebir aynı videoları (kopyaları) ayıklamak için de kullanılıyor. Skill bu modelin eğitimi, değerlendirmesini, çıkarımını ve dışa aktarımını yönetiyor.

Örnek: Claude'a şöyle dersin: "Video arşivimde birbirinin kopyası olan videoları bul." — skill Cosmos-Embed1'i çalıştırıp videoları sayısal gömmelere çevirir ve birbirine çok yakın olanları (muhtemel kopyaları) listeler.

tao-finetune-cosmos-reason

Ne yapar: Cosmos3-Nano adlı modeli, video izleyip soru cevaplayabilen (video QA) bir modele dönüştürmek için ince ayar yapıyor. FSDP (Fully Sharded Data Parallel — büyük bir modeli birden fazla ekran kartına bölerek eğitme tekniği, tek kart belleğe sığmadığında kullanılır) ile çalışıyor. Model ağırlıkları HuggingFace'ten (bkz. aşağıda) geliyor, bazı durumlarda gated (izin gerektiren) olduğu için erişim anahtarı (HF_TOKEN) gerekiyor.

Örnek: Claude'a şöyle dersin: "Elimdeki video-soru-cevap verisiyle Cosmos3-Nano'yu ince ayar yap." — skill modeli HuggingFace'ten indirir, gerekiyorsa formatını dönüştürür ve çoklu-GPU eğitimi başlatır.

tao-finetune-huggingface-model

Ne yapar: HuggingFace (yapay-zekâ modellerinin GitHub'ı gibi düşünülebilecek, binlerce hazır modelin paylaşıldığı platform) üzerindeki neredeyse her görüntü/VLM/LLM modelini kendi NVIDIA ekran kartında, tam ince ayar ya da LoRA (Low-Rank Adaptation — modelin tamamını değil, sadece küçük bir eklenti parçasını güncelleyerek çok daha az bellek ve süreyle ince ayar yapma yöntemi) ile eğitiyor. Görüntü sınıflandırma, nesne tespiti, segmentasyon, derinlik tahmini ve görüntü-yazı VLM'lerini destekliyor; işlem bitince modeli HuggingFace Hub'a bir "model kartı" (modelin ne işe yaradığını açıklayan otomatik belge) ile birlikte geri yükleyebiliyor.

Örnek: Claude'a şöyle dersin: "google/vit-base modelini kendi ürün fotoğraflarımla LoRA ile ince ayar yap." — skill modeli indirir, veri kümesini doğru formata sokar, eğitimi çalıştırır ve istersen sonucu HuggingFace'e geri yükler.

tao-generate-image-grounding

Ne yapar: Elinde (görüntü, açıklama yazısı) çiftleri varsa, bu skill açıklama yazısındaki ifadeleri ("solda duran adam" gibi) görüntü üzerindeki gerçek piksel konumlarına (kutu koordinatlarına)

bağlıyor. İki adımlı: önce yazıdan "hangi ifadeler nesnelere işaret ediyor" çıkarılıyor, sonra bir VLM bu ifadeleri görüntü üzerinde kutulara (bounding box) eşliyor.

Örnek: Claude'a şöyle dersin: "Bu görüntü-açıklama çiftlerini işleyip her ifadeyi kutu koordinatlarıyla etiketle." — skill VLM'i (Gemini ya da benzeri) iki adımda çalıştırıp her ifade için piksel kutuları üretir.

tao-generate-referring-expressions

Ne yapar: KITTI formatında (otonom sürüş/nesne tespiti alanında yaygın, her nesnenin kutu koordinatıyla etiketlendiği bir format) hazır kutulu görüntülerden, dört adımda zengin açıklamalar üretiyor: her nesne için kısa açıklama, genel sahne açıklaması, bu açıklamaları kutularla gruplayan ifadeler, ve isteğe bağlı olarak bu eşleşmeleri tekrar kontrol eden bir doğrulama adımı.

Örnek: Claude'a şöyle dersin: "KITTI etiketli trafik görüntülerimden referans-ifade veri kümesi üret." — skill dört adımı sırayla çalıştırıp her görüntü için "bu nesne şu, şurada, şöyle görünüyor" tarzında zengin, kutulara bağlı açıklamalar içeren tek bir dosya üretir.

tao-generate-video-reasoning-annotations

Ne yapar: Ham videoları, adım-adım akıl yürütme (Chain-of-Thought / CoT — bir sonuca varırken ara adımları da yazılı gösterme) içeren eğitim verisine dönüştürüyor: çok seviyeli altyazılar, yapılandırılmış açıklamalar, ve çoktan seçmeli/evet-hayır/açık uçlu soru-cevap çiftleri üretiyor. Bunu yaparken VLM ve LLM'leri "öğretmen" gibi kullanıyor — video anlayan adımlarda VLM, sadece metinden metin üreten adımlarda daha ucuz bir LLM devreye giriyor.

Örnek: Claude'a şöyle dersin: "Bu güvenlik kamerası videolarından, modelin 'neden anormal' diye açıklayabileceği soru-cevap eğitim verisi üret." — skill videoları önce özetler, sonra bu özetlerden akıl yürütmeli soru-cevap çiftleri türetir.

tao-launch-workflow

Ne yapar: Herhangi bir TAO işini (eğitim, değerlendirme, çıkarım, dışa aktarma, TensorRT motoru üretme, ya da DEFT/otomatik iyileştirme döngüsü) başlatmadan önce çalışan ortak bir "ön kontrol" listesi. Hangi platformda (yerel bilgisayar, bulut, küme) çalışılacağını, gerekli kimlik bilgilerinin (API anahtarları vb.) hazır olup olmadığını, kullanılacak konteyner imajının doğru olduğunu kontrol ediyor — hiçbir şey başlatılmadan önce bu kapılardan geçilmesi zorunlu.

Örnek: Claude'a şöyle dersin: "Modelimi eğitmeye başla." — skill devreye girip önce "hangi platformda çalıştırmak istersin, kimlik bilgilerin hazır mı" diye sorar; her şey tamam olmadan gerçek eğitim komutunu çalıştırmaz.

tao-list-capabilities

Ne yapar: "Bu TAO skill koleksiyonu (Skill Bank) tam olarak neler yapabiliyor" sorusuna, paketlenmiş uygulama/veri/model/AutoML/platform listelerinden otomatik bir cevap üretiyor. Yani bir nevi kendi kendinin kataloğu.

Örnek: Claude'a şöyle dersin: "TAO Skill Bank ile neler yapabilirim, hangi modeller destekleniyor?" — skill paketlenmiş listeleri tarayıp desteklenen tüm uygulama, veri işleme ve model eğitim yeteneklerini özetler.

tao-mine-aoi-images

Ne yapar: Gap-analysis (yukarıdaki zayıf-örnek bulma) adımının çıktısını alıp, bu zayıf örneklerle görsel olarak en çok benzeyen görüntüleri geniş bir kaynak havuzundan buluyor. Bunu "embed sonra eşleştir" (embed-then-mine) mantığıyla yapıyor: önce hem hedef hem kaynak görüntüleri sayısal gömmelere (embedding — bir görüntünün "parmak izi" gibi düşünülebilecek sayı dizisi) çeviriyor, sonra en yakın komşuları (nearest neighbour) buluyor. Sonuç, bir sonraki eğitim turunda kullanılacak, tekrarsız hâle getirilmiş bir görüntü listesi.

Örnek: Claude'a şöyle dersin: "Gap-analysis'te bulduğumuz zayıf AOI örneklerine benzer gerçek görüntüleri kaynak havuzumdan bul." — skill üç konteyner çalıştırması zincirler (hedefleri göm, kaynak havuzu göm, en yakınları bul) ve sonunda benzer görüntülerin listesini verir.

tao-port-huggingface-model

Ne yapar: HuggingFace'teki bir görüntü-tanırma modelini (örn. ViT, DETR, SegFormer gibi mimariler) TAO Toolkit'in kendi ekosistemine (tao-core yapılandırması, tao-pytorch eğitici, tao-deploy TensorRT dağıtım hattı) tam olarak entegre ediyor. Bu, tek seferlik bir ince ayar değil — modeli TAO'nun kalıcı bir "vatandaşı" hâline getirme işi; 7 fazlı bir döngüyle (ön koşul kontrolü, HF modelini inceleme, TAO kod tabanını keşfetme, yapılandırma yazma, eğitici inşa etme, dağıtım hattı kurma, test) ilerliyor, her fazda "dene, test et, hatayı bul, düzelt" mantığıyla çalışıyor.

Örnek: Claude'a şöyle dersin: "Şu HuggingFace ViT modelini TAO Toolkit'e entegre et ki TAO'nun standart eğitim/dağıtım komutlarıyla çalışsın." — skill modeli inceler, TAO'nun beklediği yapılandırma dosyalarını ve eğitici kodunu yazar, TensorRT dağıtım hattını kurar ve uçtan uca test eder.

tao-route-visual-changenet-samples

Ne yapar: Gap-analysis'in bulunduğu en zayıf VCN örneklerini, hangi veri-artırma modülünün o etiketli işleyebildiğine göre ayrı gruplara bölüştürüyor. Örneğin k-NN mining (en yakın komşu arama) sadece kaynak havuzda zaten var olan etiketler için anlamlı; AnomalyGen (Cosmos ile sentetik kusur üretimi) ise sadece belirli sınıflı kusurları (PASS, EKSİK LEHİM, KÖPRÜ LEHİM gibi) üretebiliyor. Bu skill "hangi zayıf örnek hangi modüle gitsin" kararını otomatikleştiriyor.

Örnek: Claude'a şöyle dersin: "Gap-analysis çıktımı k-NN mining ve AnomalyGen arasında doğru şekilde böl." — skill her zayıf örneği etiketine göre uygun modülün girdi dosyasına yazar ve hangi kararın neden verildiğini özetleyen okunabilir bir rapor üretir.

tao-run-automl

Ne yapar: AutoML / HPO (Hyperparameter Optimization — modelin öğrenme hızı, katman sayısı gibi "ayar düğmelerini" elle denemek yerine otomatik olarak en iyi kombinasyonu arama) çalıştırıyor. Bayesian, Hyperband, ASHA, BOHB, hatta LLM-güdümlü arama gibi farklı arama algoritmalarını destekliyor; WandB (Weights & Biases — deney sonuçlarını görselleştirip karşılaştırmaya yarayan bir panel) ile deney takibi yapıyor ve herhangi bir TAO destekli platformda (bulut, küme, yerel) çalışabiliyor.

Örnek: Claude'a şöyle dersin: "Modelimin öğrenme oranı ve katman ayarlarını otomatik olarak en iyi hâle getir." — skill seçilen arama algoritmasıyla onlarca deneme çalıştırır, her denemenin sonucunu WandB'de takip eder ve en iyi ayar kombinasyonunu döndürür.

tao-run-automl-deft-pipeline

Ne yapar: Üç fazlı zincirleyen bir "köprü" skill: Faz 1'de AutoML ile başlangıç ayarlarını bulur (HPO); Faz 2'de DEFT döngüsünü çalıştırır (modelin hatalarını bulma → o hatalara benzer sentetik/gerçek veri toplama → yeniden eğitim); Faz 3'te bu DEFT ile zenginleştirilmiş veri üzerinde AutoML'i tekrar çalıştırıp ince ayarları günceller. Kendisi AutoML'i veya DEFT'i yeniden yazmıyor, sadece ikisini doğru sırayla birbirine bağlıyor.

Örnek: Claude'a şöyle dersin: "AOI modelimi uçtan uca iyileştir — önce ayarları optimize et, sonra hataları düzelt, sonunda tekrar optimize et." — skill önce tao-run-automl'u, ardından DEFT döngüsünü, en sonda tekrar AutoML'i sırayla tetikler.

tao-run-deft-aoi

Ne yapar: DEFT (kapalı-döngü veri/model iyileştirme süreci — muhtemelen "veri-verimli ince ayar" mantığıyla adlandırılmış) AOI iyileştirme döngüsünün tam hâli. Sırasıyla: temel değerlendirme yapar, RCA (kök neden analizi) ile hataları bulur, Cosmos AnomalyGen/AMP ile sentetik kusur görüntüleri üretir, k-NN ile benzer gerçek görüntüleri madenler, modeli yeniden eğitir — ve bunu

FAR (yanlış kabul oranı — hatalı bir parçayı yanlışlıkla "sağlam" diye onaylama oranı) belirli bir recall (duyarlılık — gerçek hataların yüzde kaçının yakalandığı) seviyesinde hedefin altına düşene kadar tekrarlar.

Örnek: Claude'a şöyle dersin: "AOI modelimin FAR'ını, recall %100'de %0.1'in altına indir." — skill döngüyü başlatır: değerlendirir, hataları bulur, sentetik/gerçek veri ekler, yeniden eğitir, hedefe ulaşana kadar tekrar dener.

tao-run-inference-service

Ne yapar: Eğitilmiş bir TAO model çıktısını (checkpoint) canlı bir çıkarım (inference) mikroservisi olarak ayağa kaldırıyor — yani modeli "sürekli soru sorulabilir" bir servis hâline getiriyor. Servisi başlatma, ona istek gönderme (örneğin sıcaklık/token gibi örnekleme ayarlarıyla) ve durdurma işlemlerini de yönetiyor.

Örnek: Claude'a şöyle dersin: "Eğittiğim modeli canlı bir servis olarak başlat ve ona test sorusu gönder." — skill uygun konteyneri başlatır, servisin hazır olmasını bekler, sonra senin belirttiğin isteği bu servise gönderip cevabı döndürür.

tao-run-on-brev

Ne yapar: Brev, NVIDIA'nın yönetilen bulut GPU kiralama hizmetidir (önceden sürücü/CUDA/Docker kurulu, kirala-kullan-sil mantığıyla çalışan sanal makineler). Bu skill TAO işlerini Brev üzerinde bir GPU makinesi kiralayarak çalıştırıyor.

Örnek: Claude'a şöyle dersin: "Kendi bilgisayarımda GPU yok, modelimi Brev üzerinde kiralık bir GPU'da eğit." — skill Brev'de bir makine oluşturur, TAO işini o makinede çalıştırır, iş bitince makineyi kapatabilir.

tao-run-on-kubernetes

Ne yapar: Kubernetes (birden fazla sunucudaki konteynerleri otomatik yöneten bir orkestrasyon sistemi — bir nevi "konteyner trafik polisi") üzerinde TAO işlerini birer iş (Job) olarak çalıştırıyor. EKS/GKE/AKS gibi bulut kümelerinde veya yerinde (on-prem) kümelerde, NVIDIA GPU Operator kurulu olduğu sürece çalışıyor; birden fazla makineye yayılan dağıtık eğitimi de destekliyor.

Örnek: Claude'a şöyle dersin: "Şirketin Kubernetes kümesinde bu eğitim işini çalıştır." — skill işi bir Kubernetes Job'ı olarak paketler, GPU kaynak talebini belirtir ve kümeye gönderir.

tao-run-on-local-docker

Ne yapar: TAO işlerini kendi bilgisayarında ya da doğrudan/uzaktan erişilen bir Docker sunucusunda (konteyner çalıştıran bir program) çalıştırıyor. Geliştirme, hata ayıklama ve küçük denemeler için düşünülmüş; büyük kümeler veya çok-makineli eğitim için uygun değil.

Örnek: Claude'a şöyle dersin: "Bu küçük test eğitimi kendi makinemdeki GPU'da çalıştır." — skill Docker konteynerini doğru GPU ayarlarıyla başlatıp işi yerel olarak yürütür.

tao-run-on-slurm

Ne yapar: SLURM (üniversite/kurum süper bilgisayarlarında yaygın olan, GPU kaynaklarını sıraya koyup dağıtan bir iş zamanlayıcı yazılımı) kümelerinde, SSH üzerinden bağlanıp sbatch/srun komutlarıyla iş gönderiyor. Sonuçlar genelde Lustre gibi paylaşımlı bir depolama sisteminde tutuluyor.

Örnek: Claude'a şöyle dersin: "Bu eğitimi kurumun SLURM kümesine gönder." — skill SSH ile giriş yapar, işi kümenin paylaşımlı diskine hazırlar ve sbatch ile kuyruğa sokar.

tao-run-platform

Ne yapar: TAO Execution SDK'sini kullanıyor — bu, iş takibi (bir "job handle" ile ilerlemeyi zaman içinde sorgulama), S3 (bulut depolama) ile otomatik veri giriş/çıkışı, ya da SLURM/Kubernetes gibi platforma özel gelişmiş özellikler gerektiğinde devreye giren opsiyonel bir Python katmanı. Çoğu

basit TAO işi buna ihtiyaç duymuyor; sadece `docker run` yeterli oluyor.

Örnek: Claude'a şöyle dersin: "Bu eğitim işini SDK üzerinden başlat ki ilerlemesini istediğim an sorgulayabileyim." — `skill tao_sdk` üzerinden bir Job nesnesi oluşturur, bu nesne üzerinden durum ve loglar takip edilebilir.

tao-setup-nvidia-gpu-host

Ne yapar: Bir makinenin TAO işlerini çalıştırmaya hazır olup olmadığını kontrol ediyor ve (kullanıcı onayıyla) eksik olanları kuruyor: NVIDIA ekran kartı sürücüsü, CUDA Toolkit (GPU'da hesaplama yapmayı sağlayan yazılım katmanı) ve NVIDIA Container Toolkit (Docker'ın GPU'yu görebilmesini sağlayan bağlantı parçası). Sadece kontrol etmek her Linux dağıtımında güvenli şekilde çalışıyor; gerçek kurulum ise kullanıcı açıkça onay verdiğinde ve belirli dağıtım ailelerinde (Ubuntu/Debian, Fedora/RHEL, openSUSE gibi) otomatikleşiyor.

Örnek: Claude'a şöyle dersin: "Bu yeni sunucuda GPU'lar TAO işlerine hazır mı kontrol et, eksik varsa bana sor." — `skill sürücü/CUDA/Container Toolkit` versiyonlarını kontrol eder, eksik olanı listeler ve kurulum için önce senden onay ister.

tao-train-action-recognition

Ne yapar: Video görüntülerinden "bu kişi ne yapıyor" (yürüme, düşme, koşma gibi) tanımlayan bir model eğitiyor. Hem renkli görüntü (RGB), optik akış (optical flow — görüntüdeki hareketin yönünü ve hızını piksel bazında ölçen bir teknik) veya ikisinin birleşimiyle çalışabiliyor.

Örnek: Claude'a şöyle dersin: "Güvenlik kamerası videolarımdan 'düşme' hareketini tanıyan bir model eğit." — `skill` uygun ön-eğitilmiş ağırlıkları yükler, RGB ve/veya optik-akış girdisiyle eğitimi yapılandırıp başlatır.

tao-train-bevfusion

Ne yapar: BEVFusion, LiDAR (lazerle mesafe ölçen bir sensör, otonom araçlarda çevredeki nesnelerin 3 boyutlu konumunu ölçmek için kullanılır) nokta bulutlarını ve kamera görüntülerini "kuşbakışı" (bird's-eye-view — sanki yukarıdan bakıyormuş gibi düzleştirilmiş bir görünüm) ortak bir uzayda birleştirerek 3 boyutlu nesne tespiti yapan bir model. Otonom sürüşte, tek bir sensöre güvenmek yerine birden fazla sensörü birleştirerek daha güvenilir algı sağlamak için kullanılıyor.

Örnek: Claude'a şöyle dersin: "LiDAR ve kamera verimi birleştirip aracın etrafındaki nesneleri 3 boyutlu tespit eden bir model eğit." — `skill` BEVFusion'a özel TAO konteynerini kullanarak veri dönüşümünü ve eğitimi yapılandırır (bu model için standart TAO imajı değil, BEVFusion'a özel bir konteyner gerektiğini not eder).

TAO Toolkit Ailesi (Bölüm 2/2)

TAO (Train, Adapt, Optimize), NVIDIA'nın hazır görüntü/video yapay-zekâ modellerini kendi verinizle eğitmenizi veya ince ayar (fine-tune) yapmanızı sağlayan bir araç takımıdır — TAO'nun ne olduğu ve genel mantığı Bölüm 1'de ayrıntılı anlatıldı. Bu bölümde, TAO ailesindeki 28 farklı model/işlev için Claude Code'a yüklenen talimat paketleri (skill) tek tek açıklanıyor; her skill kendisi model eğitmez, asistana o modele özgü doğru komutları ve ayar dosyalarını (config) nasıl hazırlayacağını öğretir.

tao-train-centerpose

Ne yapar: CenterPose, bir nesnenin merkezini bulup üzerine anahtar noktalar (keypoint — nesnenin köşe/kenar gibi belirgin noktaları) yerleştirerek nesnenin 3 boyutlu duruşunu tahmin eden bir modeldir. "6-DoF pose" (6 serbestlik dereceli duruş), nesnenin uzaydaki hem konumunu hem de yönünü (döndürülmüş mü, hangi açıda duruyor) birlikte ifade eder. Bu skill, böyle bir modeli eğitmek, test etmek (evaluate), dışa aktarmak (export — modeli başka bir çalıştırma formatına çevirmek) veya çalıştırmak (inference) için gereken adımları asistana öğretir.

Örnek: Claude'a şöyle dersin: "Şu kutu/nesne veri setiyle bir CenterPose modeli eğit, nesnenin 3D duruşunu tahmin etsin." Skill devreye girer, doğru komut ve ayar dosyasını hazırlar.

tao-train-deformable-detr

Ne yapar: Deformable DETR, resimlerdeki nesneleri kutu çizerek bulan (2D nesne tespiti) bir modeldir. "Deformable attention" adlı teknikle, resmin her yerine eşit dikkat harcamak yerine önemli bölgelere odaklanır; bu sayede benzer DINO modelinden daha hafif ama yakın başarımla çalışır.

Örnek: Claude'a şöyle dersin: "Deformable-DETR ile araç tespiti modeli eğit, DINO'dan daha hafif olsun." Skill, eğitim/test/dışa-aktarma adımlarını doğru parametrelerle kurar.

tao-train-depth-anything-v2

Ne yapar: Tek bir normal (RGB) fotoğraftan, resimdeki her pikselin kameraya olan uzaklığını (derinlik haritası) tahmin eden bir modeldir — buna "monoküler derinlik tahmini" denir (monoküler = tek kamerayla). İki çeşidi vardır: "metrik" (gerçek metre cinsinden mesafe) veya "göreceli" (hangi nokta hangisinden daha yakın/uzak olduğunu, kesin sayı vermeden gösteren) derinlik.

Örnek: Claude'a şöyle dersin: "Tek kameralı görüntülerden gerçek mesafe (metre) tahmini yapan bir derinlik modeli eğit." Skill, Depth Anything v2 tabanlı doğru eğitim akışını kurar.

tao-train-dino

Ne yapar: DINO, "DETR" ailesinden gelişmiş bir 2D nesne tespit modelidir; transformer mimarisi (dikkat mekanizmalı sinir ağı yapısı) kullanır ve "gürültü giderme" (denoising) tekniğiyle daha hızlı ve kararlı öğrenir. İsteğe bağlı olarak "distillation" (büyük/güçlü bir modelin bilgisini daha küçük bir modele aktarma) desteği de sunar.

Örnek: Claude'a şöyle dersin: "Sahnedeki tüm ürünleri kutuyla işaretleyen bir DINO modeli eğit, sonra küçük bir modele damıt." Skill hem eğitim hem damıtma (distillation) adımlarını kurar.

tao-train-fast-foundation-stereo

Ne yapar: İki kameradan (stereo çift) alınan görüntüleri karşılaştırarak derinlik haritası çıkaran FoundationStereo modelinin hafifletilmiş, hızlandırılmış versiyonudur (FFS). Orijinaline göre yaklaşık 10 kat daha az gecikmeyle (düşük "latency" — komut verilip cevap alınana kadar geçen süre) çalışacak şekilde, büyük modelden öğrenerek küçültülmüştür (distilled).

Örnek: Claude'a şöyle dersin: "Gerçek zamanlı çalışacak hızlı bir stereo derinlik modeli lazım, tam FoundationStereo çok yavaş kalıyor." Skill, FastFoundationStereo (FFS) varyantını doğru önceden-eğitilmiş ağırlıklarla kurar.

tao-train-foundation-stereo

Ne yapar: İki kameradan gelen (stereo) görüntü çiftlerini karşılaştırarak "diferans haritası" (disparity map — iki görüntü arasındaki kayma miktarı, uzaklık hesabında kullanılır) çıkarır ve bu sayede sahnenin 3 boyutlu olarak yeniden inşa edilmesini (3D reconstruction) sağlar.

Örnek: Claude'a şöyle dersin: "İki kameradan aldığım görüntülerle sahnenin 3D haritasını çıkarmak istiyorum." Skill, FoundationStereo modelini önceden eğitilmiş bileşenleriyle (Depth Anything v2, EdgeNeXt) kurup eğitim akışını hazırlar.

tao-train-grounding-dino

Ne yapar: Grounding DINO, DINO tespit modeliyle bir dil-anlama bileşenini (BERT metin kodlayıcı) birleştirir; böylece sabit bir sınıf listesi (örn. "kedi", "araba") olmadan, serbest metin tarifiyle ("kırmızı bir çanta" gibi) nesne arayabilir. Buna "açık-küme" veya "metin-güdümlü" tespit denir.

Örnek: Claude'a şöyle dersin: "Sabit kategori listesi yerine, 'sarı baret takan işçi' gibi yazılı tariflerle nesne bulan bir model istiyorum." Skill, Grounding DINO'yu bu açık-uçlu tespit için kurar.

tao-train-image-classification

Ne yapar: Bir resmin hangi kategoriye ait olduğunu (örn. "kedi" mi "köpek" mi) belirleyen klasik görüntü sınıflandırma modelidir. Farklı temel mimari yapı taşları (backbone — örn. ResNet, EfficientNet, FAN) arasından seçim yapılabilir; ayrıca modeli küçültme (distillation, quantization — sayıları daha az bit ile temsil ederek modeli hafifletme) desteği vardır.

Örnek: Claude'a şöyle dersin: "Ürün fotoğraflarını kategoriye göre ayıran bir sınıflandırıcı eğit, ResNet kullan." Skill, seçtiğin omurga ile eğitim/test/dışa-aktarma adımlarını kurar.

tao-train-mask-auto-encoder

Ne yapar: MAE (Masked Autoencoder), resmin rastgele parçalarını gizleyip modelin bu gizli kısımları tahmin ederek öğrenmesini sağlayan bir "kendi kendine öğrenme" (self-supervised — elle etiketlenmiş veri gerektirmeden öğrenme) tekniğidir. Böylece etiketsiz büyük veriyle güçlü bir görsel temel önceden eğitilip (pretrain), sonra asıl göreve uyarlanabilir (finetune).

Örnek: Claude'a şöyle dersin: "Elimde etiketlenmemiş binlerce sahne fotoğrafı var, önce bunlarla genel bir görsel model önceden-eğitmek istiyorum." Skill, MAE'nin pretrain ve ardından finetune aşamalarını kurar.

tao-train-mask-auto-label

Ne yapar: MAL (Mask Auto-Label), tam segmentasyon maskesi (nesnenin piksel piksel sınırını gösteren etiket) yerine sadece nokta veya kutu gibi çok az işaretlemeye çalışıp, tam maskeyi kendisi tahmin eden bir "zayıf-gözetimli" (weakly-supervised — eksik/kaba etiketle öğrenme) segmentasyon modelidir.

Örnek: Claude'a şöyle dersin: "Her nesnenin etrafına tam maske çizmek çok zaman alıyor, sadece kabaca kutu işaretlesem yeter mi?" Skill, MAL modelini kutu/nokta işaretlerinden tam maske üretecek şekilde eğitir.

tao-train-mask-grounding-dino

Ne yapar: Grounding DINO'nun üzerine bir "maske tahmin katmanı" eklenmiş halidir; yani hem metin tarifiyle nesne bulur hem de o nesnenin tam piksel sınırını (segmentasyon maskesi) çizer. Sabit kategori listesi olmadan, serbest metinle tarif edilen nesneleri maskeleyebilir.

Örnek: Claude'a şöyle dersin: "Yazıyla tarif ettiğim nesnenin sadece kutusunu değil tam piksel sınırını da istiyorum." Skill, Mask Grounding DINO modelini bu görev için kurar.

tao-train-mask2former

Ne yapar: Mask2Former, tek bir mimariyle üç farklı segmentasyon türünü birden yapabilen "evrensel" bir modeldir: panoptik (hem nesne hem arka plan birlikte), örnek (instance — her

nesneyi ayrı ayrı ayırma) ve semantik (her pikseli kategoriye göre boyama) segmentasyon. "Maskeli dikkat" (masked attention) tekniğiyle yüksek kaliteli sonuç üretir.

Örnek: Claude'a şöyle dersin: "Sahnedeki her nesneyi ayrı ayrı, hem de arka planla birlikte segmentlere ayıran bir model istiyorum." Skill, Mask2Former'ı ihtiyacına göre (panoptik/örnek/semantik) kurar.

tao-train-metric-learning-recognition

Ne yapar: "Metrik öğrenme" (metric learning), her görüntü için bir "parmak izi" niteliğinde sayısal temsil (embedding) öğrenip, aynı nesnenin farklı fotoğraflarının bu temsilde birbirine yakın çıkmasını sağlayan bir tekniktir. Bu skill, ürün tanıma gibi ince-taneli (fine-grained — çok benzer nesneler arasında ayırım yapma) eşleştirme/arama görevleri için böyle bir model eğitir.

Örnek: Claude'a şöyle dersin: "Mağaza rafındaki ürün fotoğraflarının, veritabanındaki hangi ürüne ait olduğunu bulan bir sistem istiyorum." Skill, triplet/contrastive kayıp (loss — modelin ne kadar yanlış olduğunu ölçen fonksiyon) fonksiyonlarıyla bir tanıma modeli eğitir.

tao-train-nvdinov2

Ne yapar: NVDINOv2, hiç etiket kullanmadan, bir "öğretmen-öğrenci" (teacher-student) kendi kendine damıtma yöntemiyle görsel transformer modelleri eğiten bir kendi-kendine-öğrenme tekniğidir. Sonuçta, birçok farklı göreve uyarlanabilecek genel amaçlı güçlü görsel özellikler (features) çıkarabilen bir model ortaya çıkar.

Örnek: Claude'a şöyle dersin: "Elimde etiketsiz çok fotoğraf var, bunlardan genel amaçlı, sonra her işe uyarlanabilecek bir görsel omurga (backbone) çıkarmak istiyorum." Skill, NVDINOv2 ile bu öğretmen-öğrenci eğitimini kurar.

tao-train-nvpanoptix3d

Ne yapar: NVPanoptix3D, konumu bilinen (posed) birden fazla RGB fotoğraftan, sahnenin 3 boyutlu panoptik segmentasyonunu (hem nesne hem arka planın 3D olarak ayrıştırılması) ve "occupancy completion" (kameranin hiç görmediği alanların da tahmin edilip doldurulması) çıkarır. VGGT adlı bir omurga ve Mask2Former tarzı bir karar katmanı (head) kullanır.

Örnek: Claude'a şöyle dersin: "Birkaç açıdan çekilmiş fotoğraflardan tüm odanın 3 boyutlu, nesnelere ayrılmış haritasını çıkarmak istiyorum." Skill, NVPanoptix3D'yi 2D ve 3D aşama kontrol noktalarıyla (checkpoint) kurar.

tao-train-ocdnet

Ne yapar: OCDNet, fotoğraf içindeki metin bölgelerini (herhangi bir açıda/dönük olsa bile) bulup çerçeveleyen bir "sahne metni tespiti" (scene text detection) modelidir — yani metnin NE OLDUĞUNU değil, NEREDE olduğunu bulur. "Ayrıştırılabilir ikilileştirme" (differentiable binarization) adlı bir teknik kullanır.

Örnek: Claude'a şöyle dersin: "Sokak tabelası fotoğraflarında yazı olan bölgeleri bulan bir model istiyorum, yazı eğik de olabilir." Skill, OCDNet'i eğitim/budama (pruning — modeli küçültmek için gereksiz kısımları çıkarma)/dışa-aktarma adımlarıyla kurar.

tao-train-ocrnet

Ne yapar: OCRNet, OCDNet'in bulunduğu metin bölgesi kırpmalarının İÇİNDEKİ yazıyı okuyup gerçek metne çeviren bir "sahne metni tanıma" (scene text recognition) modelidir. İki farklı okuma yöntemi (CTC ve dikkat-tabanlı çözücü/decoder) destekler.

Örnek: Claude'a şöyle dersin: "OCDNet ile bulduğum yazı kutularının içindeki metni gerçek karakterlere çevirmek istiyorum." Skill, OCRNet'i seçtiğin çözücü tipiyle eğitir.

tao-train-oneformer

Ne yapar: OneFormer de Mask2Former gibi panoptik, örnek ve semantik segmentasyonu tek bir modelde birleştirir; farkı, "görev-koşullu sorgular" (task-conditioned queries — modele hangi segmentasyon türünü istediğini bir girdi olarak söyleme) kullanmasıdır.

Örnek: Claude'a şöyle dersin: "Aynı model hem 'her pikseli kategoriye ayır' hem 'her nesneyi ayrı say' modunda çalışsın istiyorum." Skill, OneFormer'ı bu görev-koşullu yapı için kurar.

tao-train-optical-inspection

Ne yapar: "Optik muayene" (optical inspection), iki resmi (örneğin sağlam bir referans ürün ile üretim hattından gelen ürün) karşılaştıran "ikiz ağ" (Siamese network — iki girdiyi aynı ağırlıklarla işleyip karşılaştıran mimari) kullanarak üretim hatası, anomali veya kalite sorunu tespit eder.

Örnek: Claude'a şöyle dersin: "Devre kartı (PCB) üretiminde referans görüntüyle karşılaştırarak hatalı parçaları otomatik ayıklamak istiyorum." Skill, Optical Inspection modelini bu geçti/kaldı (PASS/NO_PASS) karşılaştırmaları için kurar.

tao-train-pointpillars

Ne yapar: PointPillars, LiDAR (lazer taramayla mesafe ölçen sensör) sensöründen gelen 3 boyutlu nokta bulutlarını, "sütun" (pillar) adı verilen bir yöntemle sahte bir 2D resme dönüştürüp, ardından üzerinde normal 2D nesne tespiti uygular. Otonom araçlarda ve robotikte 3D nesne tespiti için kullanılır.

Örnek: Claude'a şöyle dersin: "Otonom araç sensöründen gelen lazer nokta bulutunda araç/yaya tespiti yapmak istiyorum." Skill, PointPillars modelini bu 3D-den-2D dönüşüm yaklaşımıyla kurar.

tao-train-pose-classification

Ne yapar: ST-GCN (uzaysal-zamansal graf evrişimli ağ) mimarisiyle, bir kişinin vücut eklem noktalarının (pose keypoint — omuz, dirsek, diz gibi iskelet noktaları) zaman içindeki hareketine bakarak hangi eylemi yaptığını (yürüme, düşme, oturma gibi) sınıflandırır.

Örnek: Claude'a şöyle dersin: "Kamera görüntüsünden çıkardığım iskelet noktalarıyla kişinin düşüp düşmediğini anlayan bir model istiyorum." Skill, ST-GCN tabanlı pose classification modelini kurar.

tao-train-reid

Ne yapar: Kişi yeniden-tanıma (person re-identification / ReID), farklı kameralarda görünen aynı kişiyi — yüzü net görünmese bile kıyafet/duruş gibi özelliklerle — eşleştirebilen sayısal temsiller (embedding) öğrenen bir metrik-öğrenme modelidir.

Örnek: Claude'a şöyle dersin: "Bir kişi kamera A'da görünüp sonra kamera B'de tekrar çıkarsa, aynı kişi olduğunu otomatik anlamak istiyorum." Skill, ReID modelini bu kameralar-arası eşleştirme için eğitir.

tao-train-rtdetr

Ne yapar: RT-DETR (Gerçek-Zamanlı Tespit Transformeri), DETR ailesinin gerçek zamanlı çalışacak kadar hızlandırılmış versiyonudur; hız ile doğruluk arasında iyi bir denge sunar ve modeli küçültme (distillation, quantization) desteği vardır.

Örnek: Claude'a şöyle dersin: "Canlı kamera akışında gecikmesiz nesne tespiti yapan, ama DINO kadar ağır olmayan bir model istiyorum." Skill, RT-DETR'i gerçek-zamanlı kullanım için kurar.

tao-train-segformer

Ne yapar: SegFormer, her pikseli bir kategoriye atayan (semantik segmentasyon) hafif bir transformer modelidir; katmanlı (hiyerarşik) özellik çıkarımı sayesinde gerçek zamanlı çalışacak kadar hızlıdır.

Örnek: Claude'a şöyle dersin: "Yol görüntülerinde her pikseli 'yol', 'kaldırım', 'gökyüzü' gibi kategorilere ayıran hızlı bir model istiyorum." Skill, SegFormer'ı bu semantik segmentasyon görevi için kurar.

tao-train-single-step

Ne yapar: Bu skill belirli bir model mimarisine özgü değildir; TAO'daki HERHANGİ bir modeli en basit şekilde (önce eğit, sonra test et, istersen dışa aktar) çalıştırmak için kullanılan genel bir akış şablonudur. AutoML (otomatik hiperparametre arama) veya DEFT gibi tekrarlayan/döngüsel gelişmiş eğitim süreçleri OLMADAN, tek seferlik düz bir eğitim istendiğinde devreye girer.

Örnek: Claude'a şöyle dersin: "Karmaşık otomatik ayar aramasına gerek yok, şu modeli şu veriyle bir kere eğit, sonucu göster." Skill, doğrudan eğit → değerlendir → dışa-aktar sırasını izleyen en basit yolu kurar.

tao-train-sparse4d

Ne yapar: Sparse4D, birden fazla kameradan (multi-camera) gelen görüntüleri zaman içinde (temporal) birleştirerek 3 boyutlu nesne tespiti ve takibi yapar. "Seyrek sorgular" (sparse queries — sahnenin her noktasını değil, sadece olası nesne adaylarını inceleyerek hız kazanma) ve zamana yayılan bir "örnek bankası" (instance bank — takip edilen nesnelerin hafızası) kullanır.

Örnek: Claude'a şöyle dersin: "Aracın etrafındaki altı kameradan gelen görüntüyle, zaman içinde nesneleri takip eden bir 3D algı sistemi istiyorum." Skill, Sparse4D modelini bu çoklu-kamera zamansal takip görevi için kurar.

tao-train-visual-changenet

Ne yapar: Visual ChangeNet, görsel muayene ve hata tespiti için iki modu destekler: "Classify" modu iki resmi (örn. sağlam/şüpheli devre kartı) karşılaştırıp geçti/kaldı (PASS/NO_PASS) diye sınıflandırır; "Segment" modu ise öncesi/sonrası iki resim arasındaki değişen bölgeleri piksel piksel bir maske olarak işaretler.

Örnek: Claude'a şöyle dersin: "PCB üretiminde önce/sonra fotoğrafını karşılaştırıp sadece değişen/hatalı bölgeyi işaretleyen bir model istiyorum." Skill, Visual ChangeNet'in Segment modunu bu görev için kurar.

tao-validate-dataset-format

Ne yapar: Bu skill model eğitmez; NVIDIA'nın "TAO DAFT" adlı özel veri seti formatındaki (yapısı önceden tanımlanmış klasör/dosya düzeni) veri kümelerinin doğru yapılandırılıp yapılandırılmadığını kontrol eder — dosya yapısını, şemayı (schema — beklenen alan/format kuralları) ve veri içi çapraz-referans hatalarını (örn. bir etiketin işaret ettiği dosyanın gerçekten var olup olmadığı) tarar. Sadece DAFT formatı içindir, başka veri seti formatları için kullanılmaz.

Örnek: Claude'a şöyle dersin: "Eğitime başlamadan önce bu veri setinin TAO DAFT formatına uygun olup olmadığını kontrol et." Skill, tao-daft validate komutunu doğru format adı ve yol parametreleriyle çalıştırır ve hataları raporlar.

NeMo / Nemotron Ailesi

NeMo, NVIDIA'nın büyük dil modelleri (LLM — Large Language Model, insan diliyle metin üreten/anlayan yapay-zekâ modelleri) için geliştirdiği eksiksiz bir yazılım çatısıdır: model eğitimi, veri hazırlama (Curator), belge/arama sistemleri (Retriever, RAG), güvenlik bariyerleri (Guardrails) ve konuşma tanıma/sentezleme (Riva / Nemotron Speech) gibi birçok alt bileşeni bir arada barındırır. Nemotron ise NVIDIA'nın kendi eğittiği, açık ve NVIDIA donanımına özel optimize edilmiş büyük dil modeli ailesinin adıdır — tıpkı OpenAI'nin GPT'si ya da Meta'nın Llama'sı gibi, ama NVIDIA imzalı bir model serisi. Bu dosyadaki 47 kayıt, NVIDIA'nın build.nvidia.com/skills kataloğunda yayınladığı, Claude Code gibi yapay-zekâ kod asistanlarına yüklenen resmi "talimat paketleridir" — her biri kendisi iş yapmaz, sadece asistanın o konuda doğru komut/kod/ayar dosyası yazmasını sağlayan bir el kitabı gibi çalışır. Bir benzetmeyle anlatmak gerekirse: asistan aşçıdır, bu skill'ler ise "bu yemek şöyle yapılır" diyen tarif kartlarıdır — tarifi okuyan aşçı doğru malzemeyi doğru sırayla kullanır, ama tarifi kendisi mutfığa girip yemek pişirmez. 47 skill kabaca altı aileye ayrılır: (1) NeMo AutoModel — modeli birden fazla GPU'ya dağıtarak eğitme, (2) Megatron-Bridge performans ailesi — dev GPU kümelerinde eğitim hızı/bellek ayarları, (3) NeMo Relay — çalışan bir yapay-zekâ uygulamasının içini izleme/loglama altyapısı, (4) NeMo-RL — pekiştirmeli öğrenme deneyleri ve oturum yönetimi, (5) Nemotron müşteri-özeleştirme, konuşma, güvenlik-politikası ve arama araçları, (6) tek başına duran yardımcı skill'ler (sentetik veri üretimi, değerlendirme, belge arama, NemoClaw dokümantasyonu). Bu skill'lerin büyük çoğunluğu, devasa GPU kümeleri üzerinde milyarlarca parametrelili modelleri eğiten şirketler için yazılmıştır; MITAS gibi tek-geliştiricili bir video/OCR projesinde muhtemelen doğrudan kullanılmayacaktır, ama NVIDIA'nın kendi yapay-zekâ altyapısını nasıl kurguladığını ve bir "skill kataloğunun" ne işe yaradığını somut olarak görmek için öğreticidir.

AutoModel ailesi — model eğitimini GPU'lara dağıtma ve tarif geliştirme (4 skill)

nemo-automodel-distributed-training

Ne yapar: Bir büyük dil modelini tek bir GPU'ya (grafik işlemci — modelin hesaplamalarını yapan özel donanım) değil, birden fazla GPU'ya paylaştırarak eğitmenin nasıl yapılandırılacağını anlatır. FSDP2, Megatron FSDP, DDP gibi "paralellik stratejilerinin" (modelin parçalarının GPU'lar arasında hangi mantıkla bölüneceğinin yöntemleri) hangi durumda seçileceğini, gereken ayar dosyası (YAML) alanlarını ve boyutlandırma kurallarını öğretir.

Örnek: Claude'a "40 milyar parametrelili bir modeli 32 GPU'lu bir kümede nasıl eğitirim?" dersin — skill devreye girer, tensor-paralellik (TP) ve pipeline-paralellik (PP) kombinasyonunu, gerekli YAML alanlarını ve veri-paralellikinin nasıl otomatik hesaplandığını gösterir.

nemo-automodel-launcher-config

Ne yapar: Bir eğitim işini nasıl başlatacağını anlatır: tek makinede interaktif olarak (torchrn ile), büyük bir sunucu kümesinde (Slurm — üniversite/şirketlerin kullandığı iş kuyruğu sistemi) veya bulutta (SkyPilot). Her yöntem için gereken ayar alanlarını (kaç sunucu, ne kadar süre, hangi container imajı) ve ucuz ama kesintiye açık "spot" bulut sunucularında nelere dikkat edileceğini açıklar.

Örnek: "Bu eğitimi Slurm kümesinde 4 sunucuda çalıştırmam gerekiyor" dediğinde skill devreye girer ve gerekli slurm: ayar bloğunu (sunucu sayısı, süre, container imajı) hazırlar.

nemo-automodel-model-onboarding

Ne yapar: Yeni çıkan bir model mimarisini NeMo AutoModel sistemine nasıl "tanıtacağını" adım adım anlatır: modelin yapısını config.json dosyasından tanıma, gerekli kod dosyalarını yazma, modeli sisteme kayıt etme ve doğrulama testleri ekleme — beş aşamalı sıralı bir süreç.

Örnek: "Yeni çıkan X modelini AutoModel'e eklemek istiyorum" dediğinde skill, mimariyi sınıflandırma → dosyaları oluşturma → kayıt etme → test ekleme sırasını takip eder.

nemo-automodel-recipe-development

Ne yapar: Bir eğitim veya değerlendirme "tarifi" (recipe — hazır ayar dosyası ve çalıştırma mantığının birleşimi) yazmayı veya var olan bir tarifi değiştirmeyi öğretir: YAML yapısı, parçaları bir araya getiren "builder" fonksiyonları ve komut satırından nasıl çalıştırılacağı.

Örnek: "Var olan fine-tuning tarifi yeni bir veri kümesi eklemek istiyorum" dediğinde skill, en yakın YAML dosyasını temel alıp veri/optimizer/öğrenme-hızı gibi hangi bölümün değiştirileceğini gösterir.

Tek başına duran veri/değerlendirme araçları (2 skill)

nemo-data-designer-plugin

Ne yapar: Sentetik (gerçek olmayan ama gerçekçi görünen, yapay olarak üretilmiş) veri kümesi oluşturmak isteyen kullanıcıya rehberlik eder — "Data Designer" adlı kütüphaneyi kullanarak istenen açıklamaya uygun bir veri üretim hattı kurar. İki modu vardır: kullanıcıya sorular sorarak ilerleyen "İnteraktif" mod, ya da kullanıcı "sen karar ver" dediğinde otomatik ilerleyen "Autopilot" modu.

Örnek: "Müşteri şikayeti örnekleriyle sentetik bir veri kümesi oluştur, sen karar ver" dersin — Autopilot modu devreye girer ve makul varsayımlarla veri kümesini üretir.

nemo-evaluator-plugin

Ne yapar: Çalışan bir NeMo Platform sunucusuna karşı model değerlendirme (evaluation — modelin ne kadar iyi performans gösterdiğini ölçme) işlerini yönetmeyi anlatır — nemo evaluator komut satırı arayüzü üzerinden hangi ölçüm türlerinin (metriklerin) mevcut olduğunu keşfetme ve bir değerlendirme işi başlatma gibi işlemleri kapsar.

Örnek: "Bu modelin doğruluğunu hangi metriklerle ölçebilirim?" dediğinde skill nemo evaluator metric-types komutunu önerir ve sonuçları yorumlar.

Megatron-Bridge performans ailesi — büyük ölçekli GPU-kümesi eğitiminde hız ve bellek ayarları (20 skill)

Bu grup, dev modelleri yüzlerce/binlerce GPU'da eğitirken ortaya çıkan "yetmiyor/yavaş/çöküyor" sorunlarına çözüm sunan, birbirini tamamlayan uzman skill'lerden oluşur.

nemo-mbridge-mlm-bridge-training

Ne yapar: Megatron-LM (MLM — eski/temel eğitim motoru) ile Megatron Bridge (üzerine inşa edilen yeni köprü katmanı) arasında eğitim sonuçlarının tutarlı olup olmadığını test etmeyi anlatır — iki sistemi aynı küçük modelle çalıştırıp kayıp (loss — modelin ne kadar hatalı olduğunu gösteren sayı) değerlerinin örtüşüp örtüşmediğini doğrular.

Örnek: "Bridge'e geçtikten sonra eğitim sonuçları eskisiyle aynı mı?" dediğinde skill, iki sistemde de aynı küçük modeli (2 katman, 256 boyut) çalıştırıp kayıp değerlerini karşılaştırma komutlarını verir.

nemo-mbridge-multi-node-slurm

Ne yapar: Tek sunucuda çalışan bir eğitim komutunu, birden fazla sunucuyu (node) birden kullanan bir Slurm toplu-iş betiğine çevirmeyi ve çok-sunuculu hatalarda (örn. ağ zaman aşımı) ilk bakılacak yerleri anlatır.

Örnek: "Bu tek-GPU eğitim komutunu 8 sunuculu kümede çalıştıracak hale getir" dediğinde skill sbatch betiğini ve container/ağ ayarlarını hazırlar; "eğitim NCCL zaman aşımı hatası veriyor" dediğinde ise ilk kontrol edilecek log satırlarını gösterir.

nemo-mbridge-perf-activation-recompute

Ne yapar: GPU belleğini (VRAM) tasarruf etmek için "activation recompute" (ileri geçişte hesaplanan ara sonuçları saklamak yerine geri geçişte yeniden hesaplama) tekniğini nasıl kullanacağını anlatır — bellekten tasarruf ederken ekstra hesaplama maliyetine katlanma değiş tokuşunu açıklar.

Örnek: "Modelim GPU belleğine sığmıyor, bellek tasarrufu lazım" dediğinde skill önce bellek parçalanmasını kontrol etmeni, sonra "selective" (seçici) ya da "full" (tüm katman) recompute seçeneklerinden hangisinin uygun olduğunu önerir.

nemo-mbridge-perf-cpu-offloading

Ne yapar: GPU belleği dolduğunda bazı verileri (ara sonuçlar veya optimizier/eğitim-durumu bilgileri) geçici olarak bilgisayarın normal belleğine (CPU/RAM) taşıma (offloading) tekniğini anlatır. İki farklı yöntemi karşılaştırır: katman bazlı aktivasyon taşıma ve optimizier durumu taşıma.

Örnek: "30 milyar parametrelili MoE modelim GPU belleğine sığmıyor ama pipeline-paralellik de kullanmam lazım" dediğinde skill, aktivasyon taşımanın bu durumda kısıtlı olduğunu, onun yerine optimizier durumu taşımayı önerir.

nemo-mbridge-perf-cuda-graphs

Ne yapar: "CUDA graph" (GPU işlemlerini bir kere kaydedip sonra hızlıca tekrar tekrar oynatan, komut gönderme yükünü azaltan bir teknik) kullanımını anlatır — hangi modda (yerel tam-iterasyon veya katman-bazlı) hangi model tipi için uygun olduğunu belirtir.

Örnek: "Eğitimim GPU'ları verimli kullanmıyor, hızlandırmak istiyorum" dediğinde skill, dikkat (attention) ve MLP katmanları için katman-bazlı CUDA graph kullanmayı önerir.

nemo-mbridge-perf-expert-parallel-overlap

Ne yapar: MoE (Mixture-of-Experts — modelin her girdi için ağın tamamı yerine sadece bir kısım "uzman" alt-ağını kullandığı mimari) modellerinde, uzmanlar arası veri gönderme/toplama iletişimini hesaplamayla aynı anda (örtüşerek) yürütmeyi anlatır, böylece iletişim gecikmesi gizlenmiş olur.

Örnek: "MoE modelim uzmanlar arası veri aktarımında çok zaman kaybediyor" dediğinde skill, overlap_moe_expert_parallel_comm ayarını ve DeepEP/HybridEP gibi alt-sistemleri önerir.

nemo-mbridge-perf-hierarchical-context-parallel

Ne yapar: Çok uzun metin dizileri (context) üzerinde eğitim yaparken, diziyi GPU'lar arasında iki seviyeli/hiyerarşik biçimde bölmeyi (context parallelism) anlatır — hangi ayarların birbiriyle matematiksel olarak uyumlu olması gerektiğini belirtir.

Örnek: "Çok uzun metinlerle eğitim yapacağım, dizi paralellliğini 4'e bölmek istiyorum ama tek seviye yetmiyor" dediğinde skill, hierarchical_context_parallel_sizes = [2, 2] gibi iki-seviyeli bir bölme önerir.

nemo-mbridge-perf-megatron-fsdp

Ne yapar: "Megatron FSDP" (modelin parametrelerini, gradyanlarını ve optimizier durumlarını GPU'lar arasında paylaştıran, belleği tasarruflu kullanan bir eğitim stratejisi) etkinleştirmeyi anlatır — hangi ayar satırlarının değişmesi gerektiğini gösterir.

Örnek: "FSDP kullanarak Llama3-8B modelini eğitmek istiyorum" dediğinde skill, gerekli cfg.dist.use_megatron_fsdp = True gibi ayarları verir.

nemo-mbridge-perf-memory-tuning

Ne yapar: GPU bellek yetersizliği (OOM — out of memory, "bellek doldu" hatası) sorunlarını çözmek için genel bir kılavuzdur: önce bellek parçalanmasını (fragmentation — bellekte kullanılamayan küçük boşlukların birikmesi) gidermeyi, sonra parametre/aktivasyon/geçici bellek kaynaklarını sırayla ele almayı önerir. Ayrıca eğitim öncesi bellek tahmini yapan bir araç sunar.

Örnek: "Eğitimim rastgele OOM hatası veriyor ama model küçük" dediğinde skill önce `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` ayarını dener, işe yaramazsa paralellik/recompute/offloading sırasına geçer.

nemo-mbridge-perf-moe-comm-overlap

Ne yapar: MoE modellerinde uzman-paralel iletişimini (token — kelime/parça birimi dağıtma/toplama) hesaplamayla örtüştürme ayarlarını anlatır — "expert-parallel-overlap" skill'ine yakın ama daha çok "ne zaman açılmalı" kararına odaklanır ve gecikmeli ağırlık-gradyan hesaplama gibi ek seçenekleri de kapsar.

Örnek: "MoE modelimde dispatch/combine iletişimi profil çıktısında öne çıkıyor, throughput'u (birim zamanda işlenen veri miktarını) artırmak istiyorum" dediğinde skill iletişim örtüşmesini açmayı ve shared-expert örtüşmesini kapatmayı önerir.

nemo-mbridge-perf-moe-dispatcher-selection

Ne yapar: MoE modellerinde token'ları uzmanlara dağıtan sistemi (dispatcher) donanıma göre seçmeyi anlatır: `alltoall` (basit/genel-amaçlı), `DeepEP` veya `HybridEP` (daha gelişmiş, belirli GPU nesillerinde daha hızlı çalışan yöntemler).

Örnek: "H100 GPU'larda MoE eğitimi yapıyorum, hangi dispatcher'ı kullanmalıyım?" dediğinde skill `DeepEP`'i (kurulu ise) önerir, kurulu değilse `alltoall` ile başlamayı söyler.

nemo-mbridge-perf-moe-hardware-configs

Ne yapar: Farklı NVIDIA GPU platformları (H100, B200, GB200, GB300) için MoE eğitiminde kullanılan tipik paralellik ayarlarını ve kabaca beklenen performans bantlarını özetleyen bir referans tablosu sunar.

Örnek: "GB200 kümesinde DSV3 benzeri bir MoE modeli eğiteceğim, hangi ayarlarla başlamalıyım?" dediğinde skill `HybridEP` + belirli TP/EP/PP değerlerini içeren hazır bir yapılandırma verir.

nemo-mbridge-perf-moe-long-context

Ne yapar: MoE modellerini çok uzun metin dizileriyle (uzun context) eğitirken nelerin değiştiğini anlatır — context-paralellik boyutlandırma, seçici yeniden-hesaplama ve dispatcher seçiminin uzun dizilerde nasıl davrandığını açıklar.

Örnek: "Modelimi 128K token uzunluğunda eğitmek istiyorum, hangi ayarlar değişmeli?" dediğinde skill context-paralel boyutunu artırma ve seçici recompute kullanma tavsiyesini verir.

nemo-mbridge-perf-moe-optimization-workflow

Ne yapar: MoE eğitimini optimize etmenin sistematik bir yol haritasını sunar — "Üç Duvar" (bellek duvarı, iletişim duvarı, hesaplama/host-yükü duvarı) çerçevesini kullanarak önce modeli belleğe sığdır, sonra ölçekle, sonra profille (ölçüm yaparak darboğazı bul), sonra yeniden ayarla sırasını izler.

Örnek: "MoE eğitimimi nereden optimize etmeye başlamalıyım, çok fazla ayar var" dediğinde skill "önce sığdır, sonra ölçekle, sonra profille" sıralı yol haritasını uygular.

nemo-mbridge-perf-moe-vlm-training

Ne yapar: Görsel+dil modellerini (VLM — Vision-Language Model, hem resim hem metin işleyen model) MoE mimarisiyle eğitirken FSDP mi yoksa 3B-paralellik mi (TP+PP+DP birleşimi) kullanılacağına dair rehberlik eder.

Örnek: "Qwen3-VL benzeri bir MoE görsel-dil modelini ilk defa eğitiyorum" dediğinde skill, önce FSDP ile basit bir ilk çalıştırma yapmayı, sonra stabil olunca 3B-paralellığe geçmeyi önerir.

nemo-mbridge-perf-parallelism-strategies

Ne yapar: Model boyutuna ve GPU sayısına göre hangi paralellik stratejilerinin (TP, PP, DP, CP, EP — modelin farklı boyutlarda GPU'lara bölünme yöntemleri) hangi kombinasyonda kullanılacağına dair boyut bazlı bir karar tablosu sunar; hem yoğun (dense) hem MoE modeller için ayrı tablolar içerir.

Örnek: "70 milyar parametrelili bir modeli 128 GPU'da eğiteceğim, nasıl bölmeliyim?" dediğinde skill `TP=8 + PP=4-8 + DP` kombinasyonunu önerir.

nemo-mbridge-perf-sequence-packing

Ne yapar: Farklı uzunluktaki metin örneklerini boşluk bırakmadan tek bir "pakette" birleştirerek eğitim verimliliğini artıran "sequence packing" tekniğini anlatır — LLM'ler için çevrimdışı paketleme, VLM'ler için parti-içi (in-batch) paketleme ayarlarını ve context-paralellik ile uyumluluk kısıtlarını açıklar.

Örnek: "Fine-tuning verimde çok fazla boşluk (padding) var, verimi artırmak istiyorum" dediğinde skill çevrimdışı paketleme ayarlarını (`enable_offline_packing = True` gibi) önerir.

nemo-mbridge-perf-tp-dp-comm-overlap

Ne yapar: Tensor-paralellik (TP), veri-paralellik (DP) ve pipeline-paralellik (PP) arasındaki iletişimi hesaplamayla örtüştürerek eğitim hızını artırma ayarlarını anlatır.

Örnek: "TP=4 ile eğitim yapıyorum ama iletişim eğitimi yavaşlatıyor" dediğinde skill `tp_comm_overlap=True` ve ilgili gradyan/parametre toplama örtüşme ayarlarını önerir.

nemo-mbridge-recipe-recommender

Ne yapar: Kullanıcının model adı/boyutu, GPU sayısı ve eğitim hedefine (ön-eğitim/SFT/PEFT) göre hazır bir Megatron Bridge tarifi (recipe) önerir ve bu tarifi nasıl özelleştirileceğini (paralellik yeniden boyutlandırma, batch ayarı, yaygın tuzaklar) anlatır.

Örnek: "8 GPU'm var, Llama3-8B'yi SFT ile eğitmek istiyorum, hangi tarifi kullanmalıyım?" dediğinde skill `llama3_8b_sft_config` tarifini ve çalıştırma komutunu önerir.

nemo-mbridge-resiliency

Ne yapar: Uzun süren eğitim işlerinin arızalara (donanım hatası, sunucu kesintisi) dayanıklı olmasını sağlayan özellikleri anlatır: hata toleransı (fault tolerance), "straggler" (yavaş kalan işlemci) tespiti, işlem-içi yeniden başlatma, kesinti öncesi durum kaydetme.

Örnek: "Slurm'da günlerce sürecektir bir eğitim başlatacağım, bir sunucu çökerse ne olur?" dediğinde skill `FaultTolerancePlugin` ayarlarını ve otomatik yeniden başlatma sayısı gibi parametreleri önerir.

NeMo Relay ailesi — çalışan yapay-zekâ uygulamalarını izleme/loglama altyapısı (10 skill)

NeMo Relay, bir yapay-zekâ uygulamasının içinde neler olup bittiğini (hangi araç çağrıldı, hangi model isteği yapıldı, ne kadar sürdü) kaydedip izlemeyi sağlayan bir "gözlemleme" (observability) katmanıdır — bir uçağın kara kutusuna benzetilebilir.

nemo-relay-debug-runtime-integration

Ne yapar: NeMo Relay kurulu olduğu halde beklenen davranışı göstermediğinde nasıl teşhis edileceğini anlatır — hangi katmanın (yükleme, aktif "kapsam"/scope, olay kaydı) bozuk olduğunu sırayla kontrol etmeyi öğretir.

Örnek: "Relay kurdum ama hiç olay (event) görünmüyor" dediğinde skill, önce native eklentinin yüklenip yüklenmediğini, sonra aktif bir kapsam (scope) olup olmadığını kontrol etmeni söyler.

nemo-relay-get-started

Ne yapar: NeMo Relay'i ilk kez deneyecek kullanıcıya en az karmaşık başlangıç yolunu (komut satırı aracı, hazır entegrasyon, veya doğrudan kod ile) seçtirir — amaç, üretim kurulumuna girmeden önce tek bir gözle görülür başarı elde etmektir.

Örnek: "Relay'i denemek istiyorum ama uygulama kodumu değiştirmek istemiyorum" dediğinde skill, komut satırı (CLI) üzerinden "hemen dene" yolunu önerir.

nemo-relay-install

Ne yapar: NeMo Relay'i kurmak için doğru paketi/yolu seçmeyi anlatır: komut satırı aracı, Python/Node.js/Rust paketi, veya LangChain/LangGraph gibi hazır çatı entegrasyonları. Sadece kurulumla sınırlıdır, çalışma-zamanı ayarlarına girmez.

Örnek: "Python uygulamama Relay eklemek istiyorum" dediğinde skill doğru pip paketini ve temel kurulum kontrolünü gösterir.

nemo-relay-instrument-calls

Ne yapar: Uygulamanın zaten var olan araç (tool) veya model çağrılarını NeMo Relay ile "sarmalayarak" (wrap) izlenebilir hale getirmeyi anlatır — orijinal davranışı bozmadan, kapsam (scope) ve yönetilen çalıştırma API'lerinin nasıl kullanılacağını gösterir.

Örnek: "Uygulamamdaki LLM çağrılarını Relay ile izlemek istiyorum" dediğinde skill, `llm.execute(...)` gibi yönetilen çağrı fonksiyonlarıyla sarmalamayı önerir.

nemo-relay-instrument-context-isolation

Ne yapar: Aynı anda birden fazla isteğin/görevin (eşzamanlı istekler, async görevler, thread'ler) çalıştığı uygulamalarda her birinin kendi bağımsız izleme "kapsam yığınına" (scope stack) sahip olması gerektiğini anlatır — paylaşılan durumun karışmasını önler.

Örnek: "Uygulamam aynı anda birden fazla kullanıcı isteğini işliyor, loglar birbirine karışıyor" dediğinde skill her isteğe kendi scope'unu vermeni söyler.

nemo-relay-instrument-typed-wrappers

Ne yapar: Ham JSON yerine daha güçlü tipli (typed) veri yapıları kullanmak isteyenler için NeMo Relay'de tip-dönüştürücü (codec) katmanları eklemeyi anlatır — ara katman yazılımının (middleware) yine de JSON görmesini sağlayarak uyumluluğu korur.

Örnek: "JSON yerine Pydantic modelleri kullanmak istiyorum ama Relay uyumluluğunu bozmak istemiyorum" dediğinde skill PydanticCodec kullanmayı önerir.

nemo-relay-migrate-from-flow

Ne yapar: Eski "NeMo Flow" sisteminden yeni "NeMo Relay" sistemine geçiş yapmayı anlatır — bunu davranış değişikliği değil, çoğunlukla mekanik bir isim değiştirme ve dil-bazlı doğrulama işlemi olarak ele alır. Yardımcı bir Python betiği önce "deneme" (dry-run — gerçek değişiklik yapmadan ne olacağını gösterme) modunda çalıştırılır, sonra onay alınıp gerçek değişiklik yapılır.

Örnek: "Eski Flow kodumu Relay'e taşımak istiyorum" dediğinde skill önce dry-run ile nelerin değişeceğini gösterir, onay alınca gerçek değişikliği uygular.

nemo-relay-plugin-adaptive-tuning

Ne yapar: Zaten Relay ile izlenen bir uygulamada, çalışma zamanı sinyallerine göre kendini ayarlayan ("adaptive" — uyarlanabilir) davranışları (gecikme, paralellik, önbellek kullanımı) yapılandırmayı anlatır. Önce mevcut durumu gözlemleyip bir temel (baseline) oluşturmadan bu skill kullanılmamalıdır.

Örnek: "Relay zaten kurulu, şimdi otomatik performans ayarlaması yaptırmak istiyorum" dediğinde skill önce mevcut davranış ölçmeni, sonra tek seferde bir değişiklik açmanı önerir.

nemo-relay-plugin-build

Ne yapar: Tekrar kullanılabilir Relay çalışma-zamanı davranışını (örn. bir güvenlik kontrolü veya loglama kümesi) yapılandırma dosyasıyla etkinleştirilen bir "eklenti" (plugin) olarak paketlemeyi anlatır.

Örnek: "Bu güvenlik kontrolünü her uygulamada tekrar yazmak yerine paylaşılabılır bir eklenti yapmak istiyorum" dediğinde skill, eklenti yapılandırma doğrulaması ve kayıt akışını gösterir.

nemo-relay-plugin-observability

Ne yapar: NeMo Relay'in yakaladığı olayları görünür kılmak için hangi "gözlemleme" (observability) çıktısının seçileceğini anlatır: konsol logu, ham olay akışı (ATOF), taşınabilir yürütme izleri (ATIF), OpenTelemetry veya OpenInference tabanlı izleme.

Örnek: "Relay'in yakaladığı olayları görmek istiyorum ama hangi format uygun bilmiyorum" dediğinde skill, kısa süreli inceleme için konsol logunu, kalıcı analiz için OpenTelemetry'yi önerir.

Belge arama aracı (1 skill)

nemo-retriever

Ne yapar: PDF, taranmış görüntü, Office belgesi, ses ve video dosyalarından oluşan bir klasörü arama yapılabilir hale getiren bir komut satırı aracıdır (retriever). Belgeleri "ingest" (içer alma/indeksleme) işlemiyle bir vektör veritabanına (LanceDB — anlamsal arama için özel bir veri deposu) yükler, sonra retriever query ile anlamsal (kelime eşleşmesi değil, anlam benzerliği tabanlı) arama yapar. Asistanın kendi RAG (arama-destekli üretim) sistemini sıfırdan yazması yerine bu hazır aracı kullanmasını sağlar.

Örnek: "Bu 200 PDF'lik klasörde 'sözleşme feshi' geçen tüm maddeleri bul" dediğinde skill devreye girer, önce klasörü indeksler sonra anlamsal arama yapar.

NeMo-RL ailesi — pekiştirmeli öğrenme deneyleri ve oturum yönetimi (4 skill)

nemo-rl-auto-research

Ne yapar: Pekiştirmeli öğrenme (RL — Reinforcement Learning, modelin deneme-yanılma ile bir ödül sinyaliye göre öğrendiği eğitim yöntemi) deneylerini otonom biçimde yürüten bir araştırma ajanı iş akışıdır — git'i (versiyon kontrol sistemi) deney günlüğü olarak kullanarak hipotez test etme, temel/karşılaştırma çalıştırma ve sonuç analiz etme adımlarını kapsar. Kullanıcı onayı olmadan branch açmaz veya GPU harcayan iş başlatmaz.

Örnek: "Bu ödül fonksiyonunu değiştirirsem model daha iyi öğrenir mi, dene ve karşılaştır" dediğinde skill devreye girer, önce plan sunar, onay alınca deneyleri çalıştırıp git üzerinden kaydeder.

nemo-rl-brev-etiquette

Ne yapar: "Brev" adlı bulut geliştirme ortamında çalışırken disk alanı yönetimini anlatır — kod ve küçük ayarları ana dizinde tutup, büyük dosyaları (checkpoint — eğitim sırasında kaydedilen model durumu, log, ön bellek) ayrı ve daha büyük bir depolama alanına (/ephemeral) taşımayı öğretir.

Örnek: "Brev'de eğitim çalıştırıyorum ama disk doluyor" dediğinde skill, checkpoint ve ön bellekleri /ephemeral altına taşımayı önerir.

nemo-rl-docs

Ne yapar: NeMo-RL projesinde dokümantasyon yazma kurallarını anlatır — yeni bir markdown dosyası eklendiğinde docs/index.md'nin güncellenmesi gerektiğini ve fonksiyon açıklamalarının (docstring) hangi formatta (Google stili) yazılacağını belirtir.

Örnek: "Yeni bir özellik ekledim, dokümantasyonunu nasıl yazmalıyım?" dediğinde skill, Google-stili docstring formatını ve docs/index.md güncelleme adımını hatırlatır.

nemo-rl-session-memory

Ne yapar: Uzun süren veya deneysel bir çalışma oturumunun durumunu (hedef, mevcut alt görev, planlar, engeller) bir dosyada kalıcı olarak kaydetmeyi anlatır — böylece bağlantı kesintisi veya yeniden başlatma sonrası başka bir ajan/oturum kaldığı yerden devam edebilir.

Örnek: "Bu uzun deney bilgisayar kapansa da devam edebilsin istiyorum" dediğinde skill, session/session_state.md gibi bir durum dosyası oluşturup düzenli aralıklarla güncellemeyi önerir.

NemoClaw ve Nemotron ailesi — dokümantasyon, özelleştirme, konuşma ve güvenlik araçları (6 skill)

nemoclave-user-guide

Ne yapar: "NemoClaw" adlı ürünün resmi dokümantasyonuna yapay-zekâ asistanları üzerinden nasıl erişileceğini anlatır — bir MCP sunucusu (asistanın canlı belgeye bağlanmasını sağlayan bir bağlantı protokolü) üzerinden veya doğrudan markdown belge indeksi üzerinden arama yaparak güncel/doğru bilgiye ulaşmayı sağlar.

Örnek: "NemoClaw'ı nasıl kurarım?" dediğinde skill, önce canlı MCP dokümantasyon sunucusuna bağlanmayı, yoksa llms.txt indeksinden ilgili sayfayı çekmeyi önerir.

nemotron-asr-finetune

Ne yapar: Konuşma tanıma (ASR — Automatic Speech Recognition, ses kaydını yazıya döken teknoloji) modelini belirli bir alana veya dile uyarlamak isteyen kullanıcı için en ucuz/yeterli yöntemi seçen bir "yönlendirme" skill'idir: önce kelime-vurgulama (word boosting — belirli kelimelere öncelik verme), sonra n-gram dil modeli, yetmezse tam fine-tuning'e (yeniden eğitim) geçer. Her aşamayı doğru alt-skill'e yönlendirir ve maliyet/süre sorularını yanıtlar.

Örnek: "Türkçe tıp terimlerini daha iyi tanıyan bir ASR istiyorum ama az verim var" dediğinde skill önce kelime-vurgulama gibi ucuz bir yöntemi dener, yetersiz kalırsa fine-tuning aşamasına yönlendirir.

nemotron-customize

Ne yapar: Nemotron modelleri üzerinde uygulanabilecek özelleştirme adımlarını (veri temizleme/Curator, çeviri, SFT/PEFT fine-tuning, ön-eğitim, RL hizalama — DPO/RLVR/GRPO/RLHF, checkpoint dönüştürme, ModelOpt ile optimize etme, mevcut/barındırılan bir modeli değerlendirme) tek bir istekten çok-adımlı bir "boru hattına" (pipeline) planlayan geniş kapsamlı bir yönlendirme skill'idir.

Örnek: "Elimdeki ham veriyi temizleyip bir modeli fine-tune edip sonra değerlendirmek istiyorum" dediğinde skill, veri-temizleme → fine-tuning → değerlendirme adımlarını birbirine bağlayan bir akış (DAG) planı çıkarır.

nemotron-policy-generator

Ne yapar: NVIDIA'nın içerik güvenliği (content-safety) modelleri için özel güvenlik politikaları (hangi içeriğin engelleneceğini/izin verileceğini tanımlayan kurallar) üretir — kaba bir istekten ("silah yasak, tıbbi içeriğe izin ver") yola çıkarak resmi bir markdown politika belgesi, JSON taksonomi (kategori ağacı) ve modele verilecek hazır komut (prompt) şablonları oluşturur.

Örnek: "Bir çocuk eğitim uygulaması için içerik güvenlik politikası hazırla, şiddet ve yetişkin içerik tamamen yasak olsun" dediğinde skill, bu isteği resmi kategori yapısına dönüştürüp eksiksiz bir politika belgesi üretir.

nemotron-retrieval-recipes

Ne yapar: Nemotron'un genel-amaçlı belge gömme (embedding — metni sayısal bir vektöre çevirme) ve yeniden-sıralama (reranking — arama sonuçlarını alaka düzeyine göre yeniden sıraya dizme) modellerini kullanan hazır "tarifleri" (recipe) planlamayı, hata ayıklamayı, ayarlamayı,

değerlendirmeyi ve devreye almayı anlatır.

Örnek: "Nemotron embed modelini kullanan arama sistemim yavaş, nasıl hızlandırırım/değerlendiririm?" dediğinde skill, ilgili tarifi önizleme/çalıştırma/rapor komutlarını verir.

nemotron-speech

Ne yapar: NVIDIA Nemotron Speech (kamuya açık adıyla "Riva") konuşma-işleme NIM'lerini (NVIDIA'nın hazır container olarak dağıttığı, çalıştırılabilir yapay-zekâ hizmetleri) devreye almayı, çalıştırmayı ve test etmeyi anlatır — konuşma tanıma (ASR), metinden sese (TTS) ve makine çevirisi (NMT) için build.nvidia.com üzerinden bulut kullanımı veya kendi sunucunda barındırma seçeneklerini kapsar.

Örnek: "Kendi sunucumda Türkçe konuşmayı metne çeviren bir servis kurmak istiyorum" dediğinde skill, Docker ile self-hosted Riva ASR kurulum adımlarını ve gRPC/HTTP protokol seçimini anlatır.

Jetson Ailesi

NVIDIA Jetson, avuç içi büyüklüğünde, düşük güçle çalışan minik bilgisayarlardır — kredi kartından biraz büyük bir devre kartı üzerinde işlemci (CPU), ekran kartı (GPU) ve yapay-zekâ hızlandırıcı devreleri bir arada bulunur. Bu kartlar robotlara, dronlara, güvenlik kameralarına, endüstriyel makinelere ve otonom araçlara gömülerek çalışır; ayrı bir kasa veya sunucu değil, doğrudan cihazın kendisinin "beyni" olurlar. Jetson'ın asıl farkı, yapay-zekâ modelini uzak bir bulut sunucusuna göndermeden, doğrudan cihazın üzerinde çalıştırabilmesidir — buna sektörde "uç yapay zekâsı" (edge AI) denir. Günlük hayattan bir benzetmeyle: akıllı telefonun kamerası yüzünü tanıırken internete bağlanıp bir sunucuya sormaz, telefonun kendi çipi anında karar verir — Jetson da robotik ve endüstriyel dünyada aynı işi yapan, çok daha güçlü bir çip ailesidir. Ailenin küçük ve ucuz "Orin Nano"dan, güçlü "AGX Orin"e, en yeni ve en güçlü "Thor"a kadar farklı büyüklükte üyeleri vardır; hangisinin seçileceği cihazın bütçesine ve ihtiyaç duyulan yapay-zekâ gücüne göre değişir. Aşağıdaki 33 skill, bu Jetson cihazlarını sıfırdan kurmak, donanım bağlantılarını (kamera, ağ, USB gibi) özelleştirmek, üzerinde yapay-zekâ modeli çalıştırmak ve performansını ölçmek için NVIDIA'nın resmi olarak hazırlayıp yayınladığı talimat paketleridir — her biri bir yapay-zekâ kod asistanına "bu işi nasıl doğru yapacağını" öğretir, işi kendisi yapmaz.

jetson-build-source

Ne yapar: Kaynak koddaki (bsp_sources altındaki) değişikliklerden yola çıkarak çekirdek tarafı çıktıları yeniden derler: DTB (Device Tree Blob — donanımın hangi parçalarının nasıl bağlı olduğunu anlatan bir tür harita dosyası), OOT modülleri (ağaç-dışı, yani çekirdeğin ana kaynağı dışında tutulan sürücü modülleri) ve çekirdek imajı. Hangi parçaların yeniden derlendiğini bir manifest (döküm listesi) dosyasına yazar. Derlenen dosyaları BSP imajının içine kendisi kopyalamaz — bu iş jetson-promote-image skill'ine aittir.

Örnek: Claude'a "BSP kaynak kodunda değişiklik yaptım, çekirdeği yeniden derle" dersin — skill devreye girer, değişen dosyalara bakarak DTB ve modülleri yeniden derler, hangi parçaların değiştiğini bir manifest dosyasına kaydeder.

jetson-customize-camera

Ne yapar: Jetson Thor veya Orin tabanlı özel bir taşıyıcı kartta MIPI/GMSL (kamera sensörlerinin bağlandığı iki farklı fiziksel bağlantı standardı) kamera sensörlerini etkinleştirir. NVIDIA'nın o sensör için hazır sunduğu DTSI (Device Tree Source Include — donanım bağlantısını tanımlayan kaynak parça) dosyasını temel alarak, kartına özel bir donanım-tanım yaması (overlay) üretir. UPHY hız yolu (lane) paylaşımı veya ODMDATA (üretim/donanım yapılandırma verisi) düzenlemeleri bu skill'in kapsamında değildir.

Örnek: Claude'a "şu kamera sensörünü Thor kartıma bağlamak istiyorum" dersin — skill sensörün resmi NVIDIA DTSI dosyasını bulur, kablolama bilgisini oradan alır ve senin özel kartın için donanım-tanım yaması üretir.

jetson-customize-clocks

Ne yapar: Flaşlama (cihaza yazma) öncesinde CPU, GPU ve EMC (bellek denetleyicisi) saat hızlarını sabitler veya sınırlar; DVFS'i (Dynamic Voltage and Frequency Scaling — yüke göre otomatik hız/voltaj ayarlama) açar/kapatır; cpufreq governor'ını (işletim sisteminin saat hızını nasıl yöneteceğini belirleyen kural seti) değiştirir. Bunu BPMP DTB (güç/saat yönetici çipinin donanım tanımı) ve nvpower.sh betiği üzerinde yapar. Cihaz zaten açıkken canlı ayar değiştirme veya nvpmode düzenlemeleri bu skill'in işi değildir.

Örnek: Claude'a "GPU hızını sabit maksimumda tutmak istiyorum, performans testlerinde dalgalanma olmasın" dersin — skill BPMP DTB dosyasını ve nvpower.sh betiğini düzenleyerek GPU saatini sabit bir tavana kilitlet.

jetson-customize-fan

Ne yapar: Jetson üzerindeki nvfancontrol (fan kontrol yazılımı) profillerini ekler, siler, düzenler, listeler veya varsayılan profili değiştirir. Sıcaklık-hız (PWM/RPM) eğrilerini ve kontrol modunu düzenler. Değişiklikler doğrudan üretim dosyasına değil, ara bir "overlay tracker" (değişiklik izleyici) katmanına yazılır.

Örnek: Claude'a "fan çok geç devreye giriyor, daha erken ve daha hızlı dönsün" dersin — skill nvfancontrol yapılandırma dosyasındaki sıcaklık-hız eğrisini senin istediğin şekilde günceller.

jetson-customize-mgbe

Ne yapar: Jetson Thor kartında 25G/10G/1G MGBE (Multi-Gigabit Ethernet — yüksek hızlı ağ denetleyicisi) portlarını, QSFP (yüksek hızlı fiber/bakır ağ konektörü) çıkışına bağlayan donanım-tanım yaması üretir. UPHY hız yolu ataması veya ODMDATA düzenlemeleri bu skill'e ait değildir, kardeş skill'lerce yapılır.

Örnek: Claude'a "Thor kartımdaki 25G ağ portunu aktif etmek istiyorum" dersin — skill MGBE denetleyicisini QSFP konektörüne bağlayan çekirdek donanım-tanım yamasını oluşturur.

jetson-customize-nvpmode

Ne yapar: Jetson'ın güç modlarını (nvpmode — cihazın CPU/GPU/EMC hızını ve çekirdek sayısını bir arada belirleyen önceden tanımlı "profil") ekler, siler, düzenler, listeler veya açılışta hangisinin varsayılan olacağını değiştirir. Değişiklikler karta özel bir yapılandırma dosyasında yapılır; üretim (upstream) kopyası salt okunur kalır.

Örnek: Claude'a "yeni bir güç modu eklemek istiyorum, CPU'yu 4 çekirdekle sınırlasın" dersin — skill nvpmode yapılandırma dosyasına senin istediğin sınırlamalarla yeni bir güç profili ekler.

jetson-customize-pcie

Ne yapar: Jetson Thor veya Orin'de her bir PCIe (bilgisayar içi yüksek hızlı genişleme yuvası standardı) denetleyicisinin açık/kapalı durumunu, hız-yolu (lane) sayısını ve bağlantı hızını ayarlar. Bunu hem ODMDATA hem de çekirdek donanım-tanım yaması üzerinde birlikte yapar — biri unutulursa cihaz açılışta hata verebilir veya port yarı-çalışır durumda kalabilir. UPHY hız-yolu ataması veya "endpoint modu" (cihazın PCIe'de yönetici değil köle taraf olarak çalışması) bu skill'in kapsamı dışındadır.

Örnek: Claude'a "2 numaralı PCIe portunu kapatmak istiyorum, kullanmıyorum" dersin — skill hem ODMDATA'yı hem çekirdek tanımını güncelleyerek o portu güvenli şekilde devre dışı bırakır.

jetson-customize-pinmux

Ne yapar: Jetson'ın her bir fiziksel pininin hangi işlevde (SFIO — bir pinin üstlenebileceği alternatif elektriksel fonksiyon), hangi yönde (giriş/çıkış) ve hangi başlangıç durumunda çalışacağını, NVIDIA'nın resmi pinmux Excel tablosunu (XLSM) temel alarak yapılandırır. Sonucu üç ayrı donanım-tanım dosyasına (pinmux, gpio, padvoltage) tek seferde yazar. Kamera/PCIe/UPHY gibi kardeş skill'lerin aksine, bu skill'in çekirdek yaması veya ODMDATA çıktısı yoktur — kaynak, doğrudan Excel tablosudur.

Örnek: Claude'a "şu pini GPIO çıkışı olarak ayarlamak istiyorum" dersin — skill NVIDIA'nın resmi pin haritası tablosunu okur, o pinin hangi fonksiyonları destekleyebildiğini sorar-cevaplar şeklinde sana gösterir ve seçimini ilgili donanım-tanım dosyalarına yazar.

jetson-customize-uphy

Ne yapar: Özel taşıyıcı kartlarda UPHY (birden fazla yüksek hızlı arabirimin — PCIe, USB3, MGBE gibi — paylaştığı ortak fiziksel hız-yolu donanımı) lane (hız yolu) dağılımını seçer ve bunu ODMDATA'ya yazar. Bu seçim yapıldıktan sonra, ilgili kardeş skill'leri (PCIe, MGBE, USB) otomatik olarak tetikleyip çekirdek tarafının da bu seçimle uyumlu olmasını sağlar. Pinmux veya sadece-PCIe düzenlemeleri için kullanılmaz.

Örnek: Claude'a "kartımda PCIe ve USB3 aynı hız yollarını paylaşıyor, hangi dağılımı seçmeliyim" dersin — skill mevcut UPHY dağılım seçeneklerini tablo halinde gösterir, seçimini ODMDATA'ya işler ve ardından PCIe/USB/MGBE skill'lerini devreye sokarak çekirdek tarafını da senkronlar.

jetson-customize-usb

Ne yapar: Jetson Thor veya Orin'de USB2/USB3 SS (SuperSpeed — USB3'ün yüksek hız modu) portlarını açar, kapatır veya rolünü (host/cihaz) değiştirir. Port kablolamasını (VBUS-EN, aşırı akım koruma pinleri, Type-C CC pinleri gibi) kaydeder ve bunu tek bir çekirdek donanım-tanım yamasında birleştirir. UPHY hız-yolu ataması veya ODMDATA düzenlemesi bu skill'e ait değildir.

Örnek: Claude'a "USB3 portlarından birini host yerine cihaz modunda kullanmak istiyorum" dersin — skill o portun rolünü ve ilgili pin bağlantılarını güncelleyen bir donanım-tanım yaması üretir.

jetson-derive-carrier

Ne yapar: Kendi tasarladığın özel taşıyıcı kart için NVIDIA'nın referans geliştirme kartı (devkit) dosyalarını temel alarak başlangıç noktası oluşturur — referans kart dosyalarını senin kartının adıyla kopyalar/yeniden adlandırır ve boş bir donanım-tanım yaması iskeleti hazırlar. Diğer tüm "customize-*" (özelleştirme) skill'lerinin ilk çalıştırılması gereken ön koşuldur; çekirdek temel donanım tanımını değil, sadece taşıyıcı-kart seviyesindeki farkları ele alır.

Örnek: Claude'a "kendi tasarladığım taşıyıcı kart için NVIDIA'nın referans kartını temel almak istiyorum" dersin — skill referans kartın dosyalarını senin özel kartına göre kopyalar/yeniden adlandırır ve üzerine ekleme yapabileceğin boş bir donanım-tanım yaması iskeleti oluşturur.

jetson-diagnostic

Ne yapar: Çalışan bir Jetson cihazının kimliği, belleği, GPU'su, sıcaklığı, gücü, depolaması, servisleri ve en çok kaynak tüketen süreçleri hakkında salt-okunur (hiçbir şeyi değiştirmeyen) tek bir özet rapor çıkarır. Normalde tegrastats, jtop, nvpmode, free, df gibi birçok farklı komutun ayrı ayrı verdiği bilgiyi tek yerde toplar.

Örnek: Claude'a "Jetson'im şu an ne durumda, neden yavaş çalışıyor" dersin — skill bellek, GPU, sıcaklık, güç modu ve çalışan servisler dahil tüm bu bilgileri tek bir okunaklı raporda birleştirir.

jetson-download-bsp

Ne yapar: NVIDIA'nın Jetson Linux BSP (Board Support Package — bir donanımı işletim sistemine tanıtan resmi yazılım/dosya paketi) arşivini, örnek dosya sistemini (sample rootfs), kaynak kodları (public_sources) ve çapraz-derleme araçlarını (x-tools) indirir. Sadece indirme işini yapar; arşivleri açmaz, kaynak ağacını hazırlamaz, profil dosyasını düzenlemez — bunlar sonraki skill'lerin işidir.

Örnek: Claude'a "Jetson kurulumuna sıfırdan başlıyorum, gerekli dosyaları indir" dersin — skill doğru sürüm BSP paketini, örnek dosya sistemini ve kaynak kodlarını NVIDIA sunucularından indirip beklenen klasörlere yerleştirir.

jetson-flash-image

Ne yapar: Hazırlanmış (promote edilmiş) bir BSP imajını, cihaz RCM (Recovery Mode — flaşlama için kullanılan özel kurtarma modu) durumundayken flash.sh veya l4t_initrd_flash.sh aracılığıyla fiziksel Jetson cihazına yazar. Kart tipi, önyükleme aygıtı gibi ayarları profil veya cihazın kendi EEPROM'undan (üzerindeki kalıcı kimlik hafızası) okuyarak doğrular.

Örnek: Claude'a "hazırladığım imajı cihaza yükle" dersin — cihazı RCM moduna aldıktan sonra skill flash.sh aracını doğru parametrelerle çalıştırıp imajı Jetson'a yazar.

jetson-generate-kb

Ne yapar: Aktif Jetson hedefinin (profilinin) BSP klasör yapısını ve kaynak kod ağacını tarayıp, hangi alt klasörlerin var olduğunu, hangi belgelerin (kılavuz, şematik vb.) bağlı olduğunu tek bir markdown özet dosyasında toplar. Bu dosya bir "anlık görüntü"dür (snapshot) — her çalıştırıldığında yeniden üretilir, böylece sonraki oturumlarda her şeyi baştan keşfetmek gerekmez.

Örnek: Claude'a "bu Jetson kurulumunun genel görünümünü bir dosyada özetle" dersin — skill BSP klasörünü ve kaynak ağacını tarayıp neyin nerede olduğunu tek bir markdown dosyasında özetler.

jetson-headless-mode

Ne yapar: Cihazın grafik arayüzünü (masaüstü) ve gereksiz arka-plan servislerini güvenli ve geri alınabilir şekilde kapatarak bellek kazandırır — önce bir plan sunar, onay verirsen uygular. Ekran çıkışı gerçekten gerekiyorsa bu değişiklikleri önermez.

Örnek: Claude'a "bu Jetson'da ekran kullanmayacağım, boşa giden belleği geri kazanmak istiyorum" dersin — skill önce hangi masaüstü bileşenlerinin ve servislerin kapatılabileceğine dair bir plan sunar, onayladığında bunları güvenli şekilde devre dışı bırakır.

jetson-inference-mem-tune

Ne yapar: Bir yapay-zekâ dil/görsel modelini (LLM/VLM) Jetson üzerinde çalıştırmak için hangi sunucu yazılımının (vLLM, SGLang, llama.cpp, TensorRT gibi) ve hangi bellek ayarlarının kullanılacağını, cihazın gerçek bellek durumuna bakarak önerir. Modelin hangi hassasiyette (quantization — modelin sayılarını daha az bit ile temsil ederek küçültme) çalıştırılacağını seçmez, sadece hangi hassasiyeti hangi motorun verimli işleyebileceğine işaret eder.

Örnek: Claude'a "Orin Nano 8GB üzerinde 7 milyar parametrelili bir modeli çalıştırmak istiyorum, bellek yetiştir mi" dersin — skill cihazın bellek durumuna bakıp hangi sunucu yazılımını ve hangi bellek bayraklarını kullanman gerektiğini önerir.

jetson-init-image

Ne yapar: İndirilmiş Jetson Linux ve örnek dosya sistemi arşivlerini açar, apply_binaries.sh betiğini çalıştırarak BSP imajını kurulum için hazır hale getirir ve kurulum yolunu aktif profile kaydeder. jetson-init-source'un (kaynak kod tarafı) öncesinde, imaj tarafını hazırlayan adımdır.

Örnek: Claude'a "indirdiğim BSP dosyalarını kur" dersin — skill arşivleri doğru klasöre açar, apply_binaries.sh betiğini çalıştırır ve kurulum yolunu profil dosyasına kaydeder.

jetson-init-source

Ne yapar: BSP özelleştirmesi için gereken kaynak-kod çalışma alanını kurar: git ile izlenen bir Linux_for_Tegra ağacı, çekirdek/sürücü/donanım-tanım kaynak dosyalarını içeren bsp_sources klasörü ve çapraz-derleme (cross-compile — bir mimaride çalışırken başka bir mimari için kod üretme) araç zinciri. Bundan sonraki tüm "customize-*" ve derleme skill'lerinin ön koşuludur.

Örnek: Claude'a "çekirdek/donanım tanım dosyalarında değişiklik yapacağım, kaynak çalışma alanını hazırla" dersin — skill git ile izlenen bir kaynak ağacı kurar ve çapraz-derleyici araç zincirini hazır hale getirir.

jetson-init-target

Ne yapar: Yeni bir Jetson hedef-platform profili (referans geliştirme kartı ve isteğe bağlı özel taşıyıcı kart bilgisiyle) oluşturur ve bunu "aktif hedef" olarak işaretler. Bu profil, sonraki tüm skill'lerin (kurulum, özelleştirme, derleme, flaşlama) hangi cihaz için çalıştığını bilmesini sağlayan tek referans kaynağıdır.

Örnek: Claude'a "yeni bir Jetson Thor geliştirme kartı için proje profili oluştur" dersin — skill hangi ürünü ve hangi flaş yapılandırmasını kullandığını sorar, cevaplarını bir profil dosyasına yazar.

jetson-link-docs

Ne yapar: Diskte önceden indirilmiş NVIDIA referans belgelerini (geliştirici kılavuzu, tasarım kılavuzu, pinmux tablosu, şematikler) aktif profiledeki ilgili alanlara bağlar — kendisi hiçbir şey indirmez, sadece var olan dosyaları isimle eşleştirip kaydeder. Bu sayede diğer skill'ler (örneğin generate-kb, customize-pinmux) bu belgeleri isimle bulabilir.

Örnek: Claude'a "pinmux Excel dosyasını ve şematik PDF'lerini profile bağla" dersin — skill diskteki dosyaları profildeki ilgili alanlarla eşleştirir ve kaydeder.

jetson-llm-benchmark

Ne yapar: Jetson üzerinde çalışan bir dil/görsel modelinin (LLM/VLM) sunum performansını vLLM, llama.cpp veya Ollama gibi farklı motorlarda ölçer ve sonucu karşılaştırılabilir, yapılandırılmış bir JSON çıktısı olarak verir. Böylece farklı modelleri, ayarları veya güç modlarını objektif olarak karşılaştırmak mümkün olur.

Örnek: Claude'a "bu modeli farklı ayarlarla test edip hangisi daha hızlı karşılaştır" dersin — skill doğru ölçüm betiğini (motoruna özel) çalıştırır ve sonucu karşılaştırılabilir JSON formatında sunar.

jetson-llm-serve

Ne yapar: Jetson üzerinde bir dil/görsel modelini gerçek zamanlı istek karşılayan bir sunucu (API) olarak ayağa kaldırır — cihaz tipine ve JetPack (NVIDIA'nın Jetson işletim-sistemi/araç paketi) sürümüne göre vLLM veya SGLang'ı seçer, güç modunu MAXN'e (en yüksek performans moduna) ayarlar ve OpenAI-uyumlu bir uç nokta (endpoint) başlatır.

Örnek: Claude'a "bu modeli Jetson üzerinde bir API gibi sun, uygulamamdan istek atabileyim" dersin — skill cihazına ve JetPack sürümüne uygun vLLM ya da SGLang'ı seçip başlatır, sana bağlanabileceğin bir uç nokta verir.

jetson-memory-audit

Ne yapar: Jetson'ın DRAM (ana bellek) ve NvMap (NVIDIA'nın GPU/medya bellek ayırıcısı) kullanımını ölçer; bir değişiklik öncesi/sonrası anlık görüntü (snapshot) alarak, o değişikliğin gerçekten bellek kazandırıp kazandırmadığını doğrular. Özellikle bir sunucu süreci durdurulduktan sonra bellek görünüşte "boşalmıyor" gibi görüldüğünde, bunun ön belleğe takılı kalmış bellek mi yoksa gerçek bir sızıntı mı olduğunu ayırt eder.

Örnek: Claude'a "vLLM'yi durdurdum ama bellek hâlâ dolu görünüyor, gerçekten boşaldı mı" dersin — skill önce/sonra bellek anlık görüntüsü alır ve belleğin gerçekten boşalıp boşalmadığını doğrular.

jetson-optimize-memory

Ne yapar: Kullanılmayan alt sistemleri (örneğin kamera) devre dışı bırakarak DRAM'i (ana bellek) geri kazandırır. Bunu önyükleme sürecinin dört farklı katmanında yapar: MB1/MB2 BCT (ilk açılış donanım yapılandırma tabloları), çekirdeğin ayırdığı rezerve bellek bölgeleri ve SWIOTLB (DMA — donanımın belleğe doğrudan erişimi için ayrılan tampon havuzu). CPU/GPU hız ayarlarıyla ilgilenmez, sadece bellek ayırma ile ilgilenir.

Örnek: Claude'a "kamera kullanmıyorum, o alt sistemin ayırdığı belleği geri kazanmak istiyorum" dersin — skill önyükleme aşamalarında ve çekirdekte kameraya ayrılmış bellek bölgelerini kapatan değişiklikleri uygular.

jetson-package

Ne yapar: Jetson'ın GPU'suna gerçekten uygun Docker konteynerlerini, vLLM imajlarını ve Python paket kaynaklarını (PyPI index) önerir. Bunun nedeni, "aarch64 destekler" yazar birçok genel ARM paketinin, Jetson'ın özel GPU mimarisi (SM — streaming multiprocessor, GPU'nun hesaplama birimi hedefi) için derlenmemiş olabilmesidir; bu skill varsayılan olarak NVIDIA'nın Jetson için özel derlediği paketleri önerir.

Örnek: Claude'a "PyTorch'u pip ile kurdum ama GPU'yu kullanmıyor" dersin — skill senin Jetson modeline (Orin mi Thor mu) ve JetPack sürümüne uygun, NVIDIA'nın özel derlediği konteyner veya paket kaynağını önerir.

jetson-print-bsp-info

Ne yapar: Host bilgisayardaki (Jetson'ın kendisi değil, kurulumu yaptığın PC'deki) bir Linux_for_Tegra (BSP) klasörünün L4T sürümünü, desteklenen kart yapılandırmalarını ve dosya

sistemi durumunu özetleyerek salt-okunur bir kontrol yapar. NVIDIA'nın bu skill'i, kendi skill-yazım standardı için bir referans/örnek olarak sunduğunu belirtir — yani başka skill yazmak isteyenlere "böyle yazılır" göstermek amacı da taşır.

Örnek: Claude'a "bu BSP klasörü doğru açılmış mı, hangi L4T sürümü" dersin — skill Linux_for_Tegra klasörünü tarayıp sürüm, desteklenen kartlar ve dosya sistemi durumunu özetler.

jetson-print-device-info

Ne yapar: Üzerinde çalıştığı, fiziksel olarak açık bir Jetson cihazının modül modelini, L4T/çekirdek/işletim-sistemi sürümünü ve güncel güç modunu okuyup özetler — salt-okunur bir kontroldür. Bu da NVIDIA'nın referans/örnek olarak sunduğu basit bir skill'dir; genellikle bir performans testinden önce "başlangıç durumu" kaydı almak için kullanılır.

Örnek: Claude'a "şu an bağlı olduğum Jetson hangi modül, hangi çekirdek sürümünde çalışıyor" dersin — skill cihazın modül modelini, sürüm bilgilerini ve güç modunu okuyup özetler.

jetson-promote-image

Ne yapar: Özelleştirme (customize-*) ve derleme (build-source) adımlarının ürettiği tüm çıktıları, flaşlamaya hazır olacak şekilde BSP imaj klasörüne kopyalar. Kendisi ne dosya derler ne de cihaza yazar — sadece "hazırlanan her şeyi doğru yere taşıma" işini yapar; flaşlama işini jetson-flash-image üstlenir.

Örnek: Claude'a "yaptığım tüm özelleştirmeleri flaşlamaya hazır hale getir" dersin — skill kaynak ağacındaki değişiklikleri ve derlenmiş çıktıları BSP imaj klasörüne kopyalar.

jetson-quick-start

Ne yapar: Jetson/IGX BSP özelleştirme sürecinin giriş noktasıdır — tek bir soru formuyla "Otomatik Kurulum", "Rehberli Kurulum" veya "Var Olan Çalışma Alanını Kullan" seçeneklerinden birini seçtirir, sonra cevaplarını doğru sırayla ilgili kurulum skill'lerine (indirme, imaj/kaynak kurulumu, belge bağlama vb.) aktarır. Kendisi hiçbir dosya indirmez veya oluşturmaz — sadece yönlendirir.

Örnek: Claude'a "Jetson kurulumuna baştan başlamak istiyorum ama hangi skill'i çağıracağımı bilmiyorum" dersin — skill sana kısa bir kurulum-modu sorusu sorar, cevabına göre doğru sırayla diğer kurulum skill'lerini devreye sokar.

jetson-set-target

Ne yapar: Zaten oluşturulmuş birden fazla hedef-platform profili arasında "aktif" olanı değiştirir — yeni bir profil yazmaz, sadece hangisinin şu an kullanılacağını işaretleyen göstergeyi (pointer) günceller. Yeni profil oluşturmak için kardeşi olan jetson-init-target kullanılır.

Örnek: Claude'a "iki farklı Jetson projem var, şimdi diğerine geçmek istiyorum" dersin — skill zaten var olan profil dosyalarından seçtiğini aktif hale getirir.

jetson-speculative-decoding

Ne yapar: Bir vLLM sunucusuna, token (kelime/kelime-parçası) üretim hızını artırmak için "spekülatif kod çözme" (speculative decoding — küçük bir taslak modelin önerdiği kelimeleri büyük modelin tek seferde toplu doğrulaması yöntemi) ekler; EAGLE-3 tekniği veya ayrı bir taslak model kullanılabilir. Bunun Jetson'da ne zaman işe yarayıp yaramayacağını da bilir: kazanç GPU belleğine bağlıdır, bu yüzden bunu sadece Thor veya AGX Orin gibi bol bellekli cihazlarda önerir, Orin Nano/NX gibi düşük bellekli cihazlarda önermez.

Örnek: Claude'a "modelim token üretirken çok yavaş, hızlandırmak istiyorum" dersin — skill, eğer cihazın Thor veya AGX Orin olduğunu görürse, vLLM sunucusuna spekülatif kod çözme ayarını ekler; Orin Nano gibi düşük bellekli bir cihazsa bunun bellek yetmezliğine yol açacağını söyleyip önermez.

jetson-validate-image

Ne yapar: Flařlama sonrasında, BSP'nin doęru kurulduęunu iki řekilde doęrular: diskteki dosyaları statik olarak kontrol ederek, ve/veya (baęlantı bilgisi varsa) doęrudan cihaza baęlanıp temel/duman testleri (smoke test — sistemin en azından ayakta kalıp kalmadıęını gösteren hızlı kontrol) ve regresyon testleri alıřtırarak. Yeniden imaj hazırlama veya flařlama yapmaz, sadece sonucu doęrular.

Örnek: Claude'a "imajı cihaza yükledim, doęru alıřıyor mu kontrol et" dersin — skill hem disk üzerindeki BSP dosyalarını statik olarak kontrol eder hem de cihaza baęlanıp temel testler alıřtırır.

Sağlık / Tıbbi Cihaz Ailesi (Isaac for Healthcare, Holoscan, DICOM, HSB)

Bu bölümdeki 33 skill, NVIDIA'nın sağlık ve tıbbi cihaz alanına özel dört alt ailesini kapsıyor: Holoscan, DICOM, HSB (Holoscan Sensor Bridge) ve i4h (Isaac for Healthcare). Holoscan, ameliyathanede kullanılan bir ultrason cihazı ya da endoskopi kamerası gibi tıbbi cihazların ürettiği görüntü/sensör verisini gerçek zamanlı (çok düşük gecikmeyle, saniyenin onda biri değil binde biri mertebesinde) işleyip ekrana veya bir yapay-zekâ modeline aktaran bir yazılım platformu — bir nevi "tıbbi cihazların işletim sistemi". DICOM ise hastanelerde çekilen MR, BT (bilgisayarlı tomografi), röntgen gibi görüntülerin dünya çapında ortak dosya ve etiket standardı; hastane cihazları farklı marka olsa da DICOM sayesinde aynı dosya her yerde açılabilir — günlük hayattaki JPEG/PDF'in tıbbi görüntüdeki karşılığı gibi düşünülebilir, ama üstüne hasta adı, doğum tarihi gibi kişisel bilgi de taşır. HSB (Holoscan Sensor Bridge), kameraların/sensörlerin ham verisini bir FPGA (yeniden programlanabilir özel bir donanım çipi) üzerinden düşük gecikmeyle bir NVIDIA cihazına (devkit) taşıyan fiziksel bir köprü kartı — kamera ile bilgisayar arasına giren bir "çevirmen" gibi. i4h (Isaac for Healthcare) ise bunların üstüne oturan simülasyon katmanı: ameliyat robotu, kateter (damar içine giren ince tıbbi tel) gibi cihazları gerçek hastaya dokunmadan önce sanal ortamda ("dijital ikiz" üzerinde) eğitip test etmeyi sağlıyor — uçuş simülöründe pilot yetiştirmek gibi, ama burada eğitilen bir cerrahi robot ya da yapay-zekâ modeli. Bu skill'lerin hiçbirisi kendi başına iş yapmaz; her biri Claude Code gibi bir yapay-zekâ asistanına "bu konuda doğru komutu/kodu şöyle yaz, şu sırayı izle" diye öğreten bir talimat metnidir — tarifi uygulayan aşçı değil, tarifi ta kendisidir.

DICOM Ailesi — Tıbbi Görüntü Dosyalarıyla Çalışma

dicom-metadata-extract

Ne yapar: Tek bir DICOM dosyasından (bir MR/BT görüntüsünün dosyası) seçili metadata alanlarını (hasta bilgisi, cihaz ayarları gibi etiketleri) çıkarır ve standart etiketlerde PHI (Protected Health Information — hasta kimliğini ifşa edebilecek bilgi, örn. isim/doğum tarihi) bulunup bulunmadığını işaretler. Anonimleştirme yapmaz, sadece tespit edip raporlar; klinik kullanım için değildir.

Örnek: Claude'a "bu MR dosyasında hasta bilgisi sızdırıyor mu kontrol et" dersin — skill devreye girer, dosyayı okur ve hangi standart etiketlerde kişisel bilgi bulunduğunu bir JSON raporu olarak listeler.

dicom-series-preflight

Ne yapar: Bir DICOM serisinin (aynı taramaya ait çok sayıda kesit dosyasının bulunduğu klasör) başlık bilgilerini (header) okuyarak, dosyaları asıl işleme veya dönüştürmeye sokmadan önce ön kontrol yapar — eksik kesit veya tutarsız ayar gibi sorunları erken yakalar. De-identifikasyon (kimlik bilgisi temizleme) veya klinik onay için değildir.

Örnek: "Bu BT klasörünü işlemeden önce sağlıklı mı bak" dersin — skill klasördeki dosyaları hızlıca tarar, sorunlu ya da eksik bir şey varsa raporlar, böylece saatler sonra hatalı veri işlediğini fark etmezsin.

dicom-series-to-volume

Ne yapar: Bir BT (CT) DICOM serisini tek bir 3 boyutlu hacme (NIfTI formatında, HU — Hounsfield Unit, yani doku yoğunluğunu ölçen tıbbi birim — değerleriyle) dönüştürür ve konumlama bilgisini (affine matris) kanıt olarak ekler. Çok-kareli (multi-frame) DICOM dosyaları için veya klinik kullanım için tasarlanmamıştır.

Örnek: "Bu BT serisini simülasyonda kullanabileceğim 3D dosyaya çevir" dersin — skill klasördeki onlarca 2D kesiti tek bir .nii.gz hacim dosyasında birleştirir.

Holoscan Ailesi — Kurulum Yolları

holoscan-install-conda

Ne yapar: Holoscan SDK'yı (Holoscan yazılım geliştirme kiti) bir Conda (Python paket/ortam yöneticisi) ortamına, CUDA 13 (NVIDIA'nın GPU hızlandırma altyapısının 13. sürümü) uyumlu bir Linux x86_64 makinede kurar. CUDA 12 kullanan makineler için uygun değildir; o durumda container veya wheel (pip paketi) kurulumuna yönlendirir.

Örnek: "Holoscan'i Conda ortamıma kur" dersin — skill önce `nvidia-smi` ile GPU/CUDA sürümünü kontrol eder, sonra doğru `conda-forge/rapidsai` paketlerini kurar.

holoscan-install-container

Ne yapar: Holoscan SDK'nın NGC (NVIDIA'nın Docker imaj deposu) üzerindeki resmi Docker container'ını indirir, makinenin GPU/CUDA yapısına uygun etiketi (tag) seçer ve içindeki hazır Python/C++ örnekleriyle kurulumu doğrular. Apt, pip veya Conda gibi doğrudan (native) kurulumlar için değildir.

Örnek: "Holoscan'i Docker ile kurmak istiyorum" dersin — skill doğru container imajını çeker, GPU erişimini test eder ve örnek bir uygulamayı çalıştırarak kurulumun sağlıklı olduğunu doğrular.

holoscan-install-debian

Ne yapar: Holoscan SDK'nın C++ çalışma zamanı ve başlık dosyalarını Ubuntu üzerine apt (Ubuntu'nun paket yöneticisi) ile doğrudan kurar; makinenin CUDA sürücüsüne uygun paket varyantını (`holoscan-cuda-12` / `holoscan-cuda-13`) seçer. Python bağlamaları apt ile gelmez — Python gerekiyorsa `holoscan-install-wheel` ile birlikte kullanılması gerekir.

Örnek: "Bu Ubuntu makinede Holoscan'i C++ tarafı için kurmam lazım" dersin — skill doğru paketi apt üzerinden kurar ve örnek C++ uygulamalarıyla doğrular.

holoscan-install-source

Ne yapar: Holoscan SDK'yı kaynak koddan (GitHub'daki `holoscan-sdk` deposundan), içindeki `./run` betiği aracılığıyla (bu betik derlemeyi bir Docker container içinde yapar) inşa eder. Hazır paketler (Conda/container/apt/wheel) ihtiyacı karşılamadığında — örneğin debug sembolleri veya özel CMake ayarları gerektiğinde — kullanılır.

Örnek: "Holoscan'i özel derleme ayarlarıyla kaynağından derlemem gerekiyor" dersin — skill depoyu klonlar, `./run` betiğini doğru parametrelerle çalıştırır ve yerel bir kurulum ağacı üretir.

holoscan-install-wheel

Ne yapar: Holoscan SDK'nın Python bağlamalarını (bindings) pip (Python paket kurulum aracı) ile bir sanal ortama (venv) kurar; CUDA 12 veya 13'e uygun wheel paketini (`holoscan-cu12/holoscan-cu13`) seçer. Sadece Python tarafını kurar — C++ başlık dosyaları için `holoscan-install-debian` ile birlikte kullanılmalı.

Örnek: "Python ile Holoscan uygulaması yazacağım, pip ile kurar mısın" dersin — skill venv oluşturur, doğru wheel'i kurar ve `hello_world/video_replayer` örnekleriyle test eder.

holoscan-setup

Ne yapar: Kurulum yapmadan önce makinenin donanımını, işletim sistemini, CUDA sürücüsünü ve mevcut araçları inceleyip Holoscan SDK için en uygun kurulum yöntemini (container, apt, wheel, conda, kaynak) önerir ve o yönetime özel skill'e yönlendirir — hangi kapıdan gireceğini belirleyen bir yönlendirici.

Örnek: "Holoscan kurmak istiyorum ama nasıl kuracağımı bilmiyorum" dersin — skill sisteminizi tarar (Ubuntu mu, Jetson mu, hangi CUDA sürümü), sonra "senin için en uygunu şu" diyerek doğru install skill'ini devreye sokar.

HSB Ailesi — Holoscan Sensor Bridge Donanımı

hsb-app

Ne yapar: Bağlı bir HSB (Holoscan Sensor Bridge) kartına sahip bir devkit (geliştirme kiti) üzerinde çalıştırılacak örnek uygulamaları listeler; kullanıcının platformuna, HSB yazılım sürümüne, kart tipine ve sensörlere göre filtreler. Süreli çalıştırma, hata analizi ve kod düzeltme önerileri de sunar; devkit'in zaten kurulu (SSH, container hazır) olduğunu varsayar.

Örnek: "Bağlı HSB kartımda hangi örnek uygulamaları çalıştırabilirim" dersin — skill donanımını tespit eder, uyumlu uygulamaları listeler ve seçtiğini demo container'ı içinde çalıştırır.

hsb-flash

Ne yapar: HSB kartının üzerindeki FPGA'nın (Field-Programmable Gate Array — yeniden programlanabilir özel bir çip) donanım yazılımını (firmware) günceller veya düşürür. İki farklı kart tipini (Lattice kartı ve Leopard Imaging VB1940 kamerası) destekler ve bunların komutlarını kesinlikle karıştırmaz — çünkü yanlış tip için verilen bir komut cihazı kalıcı olarak bozabilir (brick).

Örnek: "HSB kartımın FPGA'sını yeni sürüme yükseltmek istiyorum" dersin — skill önce kart tipini doğrular (Lattice mi, VB1940 mü), sonra sadece o tipe özel flash komutunu çalıştırır; belirsizlik varsa işlemi durdurup sana sorar.

hsb-setup

Ne yapar: NVIDIA'nın Holoscan Sensor Bridge deposunu klonlar, hangi devkit'in kullanıldığını sorar, platforma göre host makineyi ayarlar, doğru demo container'ı derleyip çalıştırır ve HSB bağlantısını 192.168.0.2 adresine ping atarak doğrular — sıfırdan ilk bağlantıyı kurma sürecinin tamamı.

Örnek: "Yeni aldığım HSB kartını devkit'ime bağladım, kurulumu yapar mısın" dersin — skill depoyu indirir, container'ı derler ve kartla ilk iletişimi (ping) test eder.

hsb-test

Ne yapar: HSB donanımı üzerinde QA (kalite güvence) test planlarını çalıştırır — kullanıcının verdiği bir test dokümanını (yerel dosya veya web bağlantısı) okur, mevcut kurulumu uygun testleri filtreler, otomatik çalıştırılabilenleri geçti/kaldı (pass/fail) şeklinde değerlendirir ve yapılandırılmış bir sonuç raporu üretir.

Örnek: "Elimdeki bu test planını HSB kartımda çalıştır" dersin — skill dokümandaki testleri donanımınla eşleştirir, uygun olanları sırayla çalıştırır ve hangisinin geçtiğini/kaldığını raporlar.

i4h — Kateter Navigasyonu (Endovasküler Simülasyon)

i4h-catheter-navigation

Ne yapar: Kateter navigasyonu (damar içine giren ince tıbbi telin/kateterin simülasyonu) iş akışının genel tanıtımını yapar — hangi çalışma modları (fluorosim/DRR görüntüleme, XPBD fizik motoru, damar dijital ikizi) var, birbirleriyle nasıl ilişkili, hangi alt-skill'e ne zaman geçilmeli. Kendisi hiçbir işlem adımını çalıştırmaz, sadece yönlendirir.

Örnek: "Kateter navigasyon iş akışı ne işe yarıyor, nereden başlamalıyım" dersin — skill mevcut modları ve akışı özetler, sonra ihtiyacına göre uygun alt-skill'e (dijital ikiz, DRR render, viewport gibi) yönlendirir.

i4h-catheter-navigation-digital-twin

Ne yapar: Bir hastanın BT (CT) görüntüsünden damar ağının "dijital ikizini" (digital twin — gerçek anatominin bilgisayar ortamındaki kopyası) oluşturur: BT'yi ön işler (attenuation/zayıflatma ön belleğine çevirir), damarları segmente eder (ayırır) ve merkez hattını (centerline) çıkarır. Bu çıktı sonraki DRR ve viewport adımlarının girdisidir.

Örnek: "Bu hasta BT'sinden damar haritası çıkar" dersin — skill BT'yi işler, arter ağını segmente eder ve simülasyonda kullanılacak damar maskesi ile merkez hattını üretir.

i4h-catheter-navigation-e2e

Ne yapar: Kateter navigasyon iş akışının kurulumdan dijital ikize, DRR render'dan birim testlerine kadar uçtan uca bir "duman testi" (smoke test — her şeyin en azından çalıştığını doğrulayan hızlı kontrol) çalıştırır. Ekran gerektiren interaktif viewport adımını atlar.

Örnek: "Bütün kateter iş akışının çalıştığından emin olmak istiyorum, hızlı bir kontrol yap" dersin — skill sırayla kurulum, dijital ikiz, DRR render ve testleri çalıştırıp özet bir sonuç verir.

i4h-catheter-navigation-render-drr

Ne yapar: Tek bir DRR (Digitally Reconstructed Radiograph — 3D BT verisinden sentetik olarak üretilen 2 boyutlu röntgen/floroskopi görüntüsü) karesi üretir. Ya önceden hazırlanmış bir BT önbelleğiyle ya da veri gerektirmeyen sentetik bir "fantom" (phantom — test amaçlı yapay model) ile çalışabilir; GPU ve slangpy (render motoru) gerektirir.

Örnek: "Bu damar modelinden floroskopi görüntüsü üret" dersin — skill hazırladığın BT önbelleğini kullanarak (ya da veri yoksa sentetik fantomla) bir DRR karesi render eder.

i4h-catheter-navigation-setup

Ne yapar: Kateter navigasyon iş akışı için makine ve GPU gereksinimlerini kontrol eder, ./i4h komut satırı aracının iş akışını doğru gördüğünü teyit eder ve CPU üzerinde çalışan hızlı testleri koşar. Kurulum sırasında veya içe aktarma (import)/GPU/slangpy hataları alındığında kullanılır.

Örnek: "Kateter navigasyonu ilk kez kuruyorum, her şey doğru mu bak" dersin — skill donanımını ve bağımlılıklarını kontrol eder, eksik bir şey varsa söyler.

i4h-catheter-navigation-smoke

Ne yapar: fluorosim bileşenlerinin (floroskopi görüntülerini oluşturan sistemin) ve komut satırı ayrıştırıcılarının (parser) sadece CPU üzerinde, GPU gerektirmeden çalışan birim testlerini yürütür — CI (sürekli entegrasyon, otomatik test ortamı) gibi GPU'suz ortamlarda güvenle çalıştırılabilir.

Örnek: "GPU'suz bir sunucuda kateter kodunun bozulmadığını doğrulamam lazım" dersin — skill sadece CPU testlerini koşar ve "Ran N tests... OK" çıktısını doğrular.

i4h-catheter-navigation-viewport

Ne yapar: Gerçek zamanlı XPBD (Extended Position Based Dynamics — yumuşak cisim fiziğini simüle eden bir yöntem; burada kateterin bükülüp esnemesini gerçekçi gösterir) kateter fiziğine sahip interaktif floroskopi görüntüleme penceresini (viewport) açar; dijital ikiz önbelleği gerektirir ve ekran (X11) erişimi ister.

Örnek: "Kateteri elle yönlendirip damar içinde nasıl ilerlediğini görmek istiyorum" dersin — skill hazırlanmış damar dijital ikizini kullanarak interaktif bir floroskopi penceresi açar, kateteri gerçek zamanlı yönlendirebilirsin.

i4h — Agentic İş Akışı (Robotik Öğrenme Pipeline'ı)

i4h-lerobot-viz

Ne yapar: Dönüştürülmüş bir LeRobot (robotik öğrenme verisi için yaygın bir açık format) veri setini tarayıcıda görüntülemek için HTML tabanlı bir görselleştirici sunucusu başlatır. HDF5'ten (ham kayıt formatından) dönüştürme yapmaz, sadece zaten dönüştürülmüş veriyi gösterir.

Örnek: "Az önce dönüştürdüğüm robot veri setini görsel olarak kontrol etmek istiyorum" dersin — skill yerel bir web sunucusu açar, tarayıcıdan veri setindeki kayıtları adım adım izleyebilirsin.

i4h-workflow

Ne yapar: i4h'nin "agentic" (robot/yapay-zekâ ajanı) iş akışının genel tanıtımını yapar — hangi ortamlar (env), robotlar, politikalar (GR00T/openpi gibi yapay-zekâ modelleri) destekleniyor, alt-projeler nasıl bir araya geliyor. Kendisi bir adım çalıştırmaz, doğru alt-skill'e yönlendirir.

Örnek: "Bu agentic iş akışı ne işe yarıyor, hangi robotları destekliyor" dersin — skill genel haritayı çizer ve ihtiyacına göre kayıt/dönüştürme/eğitim gibi alt-skill'lere yönlendirir.

i4h-workflow-create

Ne yapar: Var olan bir ortama (environment — simülasyondaki görev/sahne tanımı) en yakın olanı çoğaltarak (fork) yepyeni bir "agentic" ortam oluşturur: YAML yapılandırması, varlıklar (assets), görev tanımı, ortam sınıfı ve doğrulamayı kapsar. Var olan bir sahneyi düzenlemek için değildir — o iş i4h-workflow-scene-edit'e aittir.

Örnek: "Var olan görevlere benzer ama farklı bir yeni robot görevi tanımlamak istiyorum" dersin — skill en yakın örnek ortamı bulur, onu temel alarak yeni ortamın iskeletini oluşturur.

i4h-workflow-dataset-annotate

Ne yapar: Bir VLM'in (Vision-Language Model — görüntüyü anlayıp yorumlayabilen yapay-zekâ modeli) her kaydedilmiş bölümün (episode) ortamın görev tanımını gerçekten yerine getirip getirmediğini doğrulamasını sağlar. Bölümleri etiketlemek, başarısız denemeleri elemek veya ince-ayarı (fine-tuning) bir başarı sınıflandırıcısına bağlamak için kullanılır. İsteğe bağlıdır, doğrulama sırasında kullanıcı istemeden çalıştırılmaz.

Örnek: "Kaydettiğim robot denemelerinden hangileri görevi gerçekten tamamlamış, otomatik ayıklar mısın" dersin — skill bir görüntü-dil modeliyle her kaydı değerlendirip başarılı/başarısız etiketler.

i4h-workflow-dataset-convert

Ne yapar: Ham bir "agentic" HDF5 kaydını (parquet, meta veri ve videolar içeren) bir LeRobot veri setine dönüştürür — eğitim için kullanılabilir hale getirir. Sadece görselleştirmek istiyorsan bu değil, i4h-lerobot-viz kullanılır.

Örnek: "Kaydettiğim HDF5 dosyasını modeli eğitmek için uygun formata çevir" dersin — skill kaydı LeRobot'un beklediği parquet+video+meta yapısına dönüştürür.

i4h-workflow-dataset-mimic

Ne yapar: Var olan bir HDF5 kaydını, hareket (action) ve durum (state) verilerine küçük gürültü (noise) ekleyerek çoğaltır — yeni bölümler kaydetmeden veri setini genişletir/çeşitlendirir. Yeni demo kaydetmek için değildir (o iş i4h-workflow-dataset-teleop'ta).

Örnek: "Elimdeki az sayıdaki robot kaydını çoğaltıp veri setini büyütmek istiyorum" dersin — skill mevcut kayıtları küçük varyasyonlarla çoğaltarak daha büyük bir eğitim seti üretir.

i4h-workflow-dataset-replay

Ne yapar: Kaydedilmiş bir HDF5 bölümünü Isaac Sim (NVIDIA'nın robotik simülasyon ortamı) içinde tekrar oynatarak görsel olarak doğrulamayı sağlar — "kayıt gerçekten doğru mu oldu" sorusuna görsel bir cevap verir.

Örnek: "Az önce kaydettiğim robot hareketinin simülasyonda doğru görüldüğünden emin olmak istiyorum" dersin — skill kaydı Isaac Sim içinde adım adım tekrar oynatır.

i4h-workflow-dataset-teleop

Ne yapar: Bir "agentic" ortam için insan tarafından uzaktan kumanda edilen (teleoperation — klavye, SO-ARM kumanda kolu veya VR ile) bölümleri kaydederek HDF5 dosyasına yazar — insan gösterimlerinden (demonstration) eğitim verisi üretmenin ilk adımıdır.

Örnek: "Robotu klavyeyle yönlendirip görevi kendim göstererek kayıt almak istiyorum" dersin — skill teleoperasyon oturumunu başlatır ve hareketlerini HDF5 dosyasına kaydeder.

i4h-workflow-e2e

Ne yapar: Tüm "agentic" iş akışını uçtan uca çalıştırır: kayıt → çoğaltma (mimic) → etiketleme (annotate/filtreleme) → tekrar oynatma (replay) → dönüştürme (convert) → görselleştirme → ince-ayar (finetune) → doğrulama (validate). Tüm pipeline'ı tek seferde denemek veya demo etmek içindir.

Örnek: "Bütün robot eğitim hattının uçtan uca çalıştığını görmek istiyorum" dersin — skill tüm aşamaları sırayla zincirleyerek çalıştırır ve nerede takıldığını gösterir.

i4h-workflow-finetune

Ne yapar: Var olan bir LeRobot veri seti üzerinde GR00T veya openpi PI0 (ikisi de robotik hareket üretmeyi öğrenen yapay-zekâ politika modelleri) modelini ince ayar (fine-tuning — önceden eğitilmiş modeli belirli bir göreve özelleştirme) ile eğitir. Bir checkpoint'i (kaydedilmiş model durumunu) değerlendirmek için değildir — o iş i4h-workflow-validate'e aittir.

Örnek: "Kaydettiğim demolarla robot modelimi bu göreve özel eğitmek istiyorum" dersin — skill veri setini kullanarak GR00T/openpi modelini ince ayardan geçirir.

i4h-workflow-scene-edit

Ne yapar: Var olan bir ortamın sahnesini yerinde düzenler — nesneleri taşımak/ölçeklendirmek/değiştirmek, kameraları ayarlamak, görev tanımını veya başarı sınırlarını (success bounds) ve rastgeleleştirmeyi (randomization) değiştirmek için bir "köprü" (bridge) düzenleme oturumu açar. Yepyeni bir ortam oluşturmak için değildir — o i4h-workflow-create'e aittir.

Örnek: "Az önce oluşturduğum ortamdaki masayı biraz kaydırıp kamerayı düzeltmek istiyorum" dersin — skill sahneyi düzenleme moduna açar, değişiklikleri canlı olarak uygulayabilirsin.

i4h-workflow-setup

Ne yapar: Makine gereksinimlerini kontrol eder ve "agentic" iş akışını başlatmak için tekrar-çalıştırılabilir (idempotent — aynı komutu tekrar çalıştırmak zarar vermez) setup.sh betiğini çalıştırır. Eksik sanal ortam (.venv), üçüncü parti kod ya da motor (engine) hatalarında kullanılır.

Örnek: "Bu agentic iş akışını ilk kez kuruyorum" dersin — skill gerekli sanal ortamı ve bağımlılıkları kurar, eksik bir şey varsa raporlar.

i4h-workflow-validate

Ne yapar: Bir politika (öğrenilmiş model) veya elle yazılmış durum-makinesi (state machine) kontrolcüsünü bir ortama karşı çalıştırıp doğrulama bölümlerini HDF5 olarak kaydeder — "bu model/checkpoint gerçekten görevi başarıyor mu" sorusuna cevap verir.

Örnek: "Eğittiğim modelin gerçekten işe yarayıp yaramadığını test etmek istiyorum" dersin — skill modeli ortamda birkaç kez çalıştırır, başarı oranını ve kayıtları üretir.

Video / Görüntü İşleme Ailesi (VSS, DeepStream, DALI, AMC, Digital Human)

Bu bölüm, NVIDIA'nın build.nvidia.com/skills kataloğundan gelen ve video/görüntü işleme etrafında toplanan 28 skill kaydını kapsıyor. Önce temel bir hatırlatma: bu skill'lerin hiçbiri kendi başına bir iş YAPMIYOR — her biri, Claude Code gibi bir yapay-zekâ kod asistanına yüklenen bir "talimat paketi"; asistanın belirli bir NVIDIA aracını (DeepStream, VSS, DALI, AMC, klinik ASR hattı) doğru şekilde kurması, çağırması veya kod üretmesi için resmi rehber görevi görüyor — yani mutfaktaki tarif kitabı, aşçının kendisi değil. **DeepStream**, videoyu bir kamera/dosyadan alıp GPU içinde (CPU'ya hiç çıkmadan) çözen, modelden geçiren, nesne tespit/takip eden ve sonucu ekrana/dosyaya yazan çok-kanallı bir video analitiği motoru — bir fabrika bandı gibi düşünün: her video karesi bu bantta sırayla istasyonlardan geçiyor (çöz → toplu-işle → modelle tahmin et → takip et → çizim ekle → çıktı ver), hiçbir adımda banttın inmediği için onlarca kamerayı aynı anda gerçek zamanlı işleyebiliyor. **VSS (Video Search & Summarization)**, bu motorun üstüne inşa edilen bir "akıllı video arşivi" — kaydedilmiş binlerce saatlik görüntüde doğal dille arama yapmayı ("kırmızı kamyon ne zaman geldi"), otomatik özet çıkarmayı ve olay/uyarı yönetimini sağlayan hazır bir blueprint (referans mimari). **DALI** ise model EĞİTİMİ tarafında çalışan bir hızlandırıcı: normalde CPU'da yavaşça açılıp modele beslenen görüntü/video verisini GPU üzerinde çözüp hazırlayarak eğitimin CPU'yu beklemesini önüyor — aşçının (GPU/model) beklemesine gerek kalmadan malzemeyi (veri) hızlıca hazırlayan bir yardımcı şef gibi. **AMC (AutoMagicCalib)**, çoklu kamera kurulumlarında (ör. depo/warehouse senaryosu) kameraların birbirine göre 3 boyutlu konumunu otomatik hesaplayan bir kalibrasyon aracı — VSS'in çoklu-kamera 3B takip özelliğinin önkoşulu. Son olarak bir netleştirme: bu veri diliminde "Digital Human" (dijital-insan/avator) ailesi değil, aslında **digital-health-clinical-asr** ailesi bulunuyor — bu, avator teknolojisiyle değil, KLİNİK KONUŞMA TANIMA (ASR) modellerini tıbbi terimlere göre ölçüp iyileştiren 4 aşamalı ayrı bir hat; başlıkta geçen isimle karışmasın diye burada ayrıca belirtiyorum. Günlük hayattan bir benzetmeyle özetlersek: DeepStream/VSS ikilisi, yüz binlerce videoyu izleyip senin yerine not tutan, soru sorulduğunda arşivde arayan ve özetleyen bir ekip; DALI o ekibin eğitim/hazırlık aşamasını hızlandıran bir bant; AMC ise "kameraları doğru açıyla kurdun mu" diye kontrol eden bir teknisyen; klinik ASR hattı ise tamamen ayrı bir masada, doktorun ağzından çıkan tıbbi terimleri doğru yazıya döken bir sistemi eğitip test eden bir ekip.

AMC (AutoMagicCalib) — Kamera Kalibrasyon Ailesi

amc-run-sample-calibration

Ne yapar: AMC, depo gibi çoklu-kamera kurulumlarında kameraların birbirine göre 3 boyutlu konumunu otomatik hesaplayan bir kalibrasyon aracı. Bu skill, halihazırda çalışan bir AMC mikroservisini (arka planda REST API sunan küçük bir servis), NVIDIA'nın hazır paket olarak sağladığı örnek veri setiyle ([sdg_08_2_sample_data_010926.zip](#)) uçtan uca test eder — yani "kurulum gerçekten çalışıyor mu" sorusuna cevap arayan bir sağlık kontrolüdür (sanity check). Kullanıcı kendi video dosyalarını ya da canlı RTSP (gerçek zamanlı kamera yayını) akışını kalibre etmek isterse bu skill değil, ilgili başka skill'ler devreye girer. AMC mikroservisi henüz çalışmıyorsa önce onu kurmayı öneriyor.

Örnek: Claude'a "AMC kurulumunu örnek veri setiyle doğrula" dersin — skill, paketle gelen örnek zip'i çalışan AMC servisine gönderip kalibrasyonun baştan sona hatasız tamamlandığını raporlar.

amc-run-video-calibration

Ne yapar: Kullanıcının kendi elindeki, önceden kaydedilmiş (canlı olmayan) MP4 video dosyalarını AMC'nin REST API'si üzerinden kalibre eder — komut satırı script'i ya da Docker bind-mount gerekmez, sadece çalışan bir mikroservis ve senkronize video dosyaları yeterlidir. Videoların [cam_00.mp4](#), [cam_01.mp4](#) gibi adlandırılmış, zaman-senkronize ve yaklaşık 1920x1080 çözünürlükte olması bekleniyor. Skill ayrıca bir veri gizliliği uyarısı taşıyor: yüklenen videolar AMC servisine

aktarılıyor, bu yüzden hassas görüntü içeriği varsa dikkat edilmesi gerekiyor. Canlı RTSP akışları için bu skill değil, ayrı bir RTSP-kalibrasyon skill'i kullanılıyor.

Örnek: Claude'a "şu klasördeki cam_00.mp4, cam_01.mp4 videolarını kalibre et" dersin — skill dosyaları AMC REST API'sine yükleyip kalibrasyon sonucunu (kameraların birbirine göre konumu) döndürür.

amc-setup-calibration-stack

Ne yapar: AMC mikroservisini ve web arayüzünü (UI) sıfırdan ayağa kaldıran kurulum skill'i — NGC (NVIDIA'nın konteyner/model deposu, "NVIDIA GPU Cloud") üzerinden resmi release imajlarını Docker Compose ile indirip başlatır. Sırasıyla: Docker'ın sudo'suz çalıştığını doğruluyor, NGC hesabına giriş yapıyor, isteğe bağlı olarak VGGT (muhtemelen bir kalibrasyon/3B-yeniden-yapılandırma modeli) indiriyor, Docker Compose dosyasını yapılandırıp servisleri başlatıyor ve hazır olduklarını kontrol ediyor. Diğer iki AMC skill'inin (örnek veya gerçek video kalibrasyonu) önkoşulu bu skill.

Örnek: Claude'a "AMC'yi kur ve başlat" ya da "auto calibration deploy et" dersin — skill NGC'den imajları çekip Docker Compose ile mikroservis+UI ikilisini ayağa kaldırır, sonunda servislerin sağlıklı çalıştığını doğrular.

DALI — Veri Yükleme Hızlandırıcı

dali-dynamic-mode

Ne yapar: DALI (NVIDIA Data Loading Library), model eğitimi sırasında görüntü/video verisini CPU yerine GPU'da hızlıca çözüp hazırlayan bir kütüphane. Bu spesifik skill, DALI'nin daha yeni "dynamic mode" (emirsel/imperative — yani önceden bir işlem grafiği tanımlamadan, sıradan Python kodu gibi adım adım yazılan) API'si olan `nvidia.dali.experimental.dynamic` (kısaca "ndd") ile kod yazmayı, incelemeyi veya eski "pipeline mode" kodundan geçişi (migration) kapsıyor. Eski `Pipeline/@pipeline_def/pipe.build()` tarzı grafik-tabanlı API yerine doğrudan ndd çağrıları kullanmayı, okuyucuların (reader) durumlu (stateful) tutulup epoch'lar arasında yeniden kullanılmasını, `batch_size`'in her çağrıya açıkça verilmesini öğretiyor. Sadece pipeline-mode ile ilgili görevlerde bu skill atlanıyor.

Örnek: Claude'a "bu DALI pipeline'ını dynamic mode'a taşı" dersin — skill, `Pipeline/pipe.run()` tarzı eski kodu ndd çağrılarına çevirirken doğru API kurallarını (`device="gpu"`, `Batch.tensors`, `.torch()` dönüşümü) uygular.

DeepStream — Video Analitiği Ailesi

deepstream-dev

Ne yapar: DeepStream SDK ile video analitiği pipeline'ları (GStreamer tabanlı — GStreamer, ses/video akışını küçük işleme bloklarına bölüp birbirine bağlayan açık kaynak bir çatı) geliştirmenin genel/temel skill'i; Python `pyservicemaker` API'sini kullanır. TensorRT (NVIDIA'nın eğitilmiş modelleri hızlandıran çalışma-zamanı kütüphanesi) ile model entegrasyonu, nesne tespiti/takibi, Kafka gibi mesaj kuyruklarıyla entegrasyon konularını kapsıyor. Skill aktifken ezberle kod yazmak yerine önce referans dokümanları okumayı zorunlu tutuyor — çünkü tam property adları ve API kullanımı kolay yanlış hatırlanabiliyor. Tipik pipeline akışı: Kaynak → Stream Muxer (birden çok akışı toplu hale getiren bileşen) → Çıkarım (Inference) → [Takip] → OSD (ekran üstü çizim) → Render.

Örnek: Claude'a "DeepStream ile RTSP kameradan nesne tespiti yapan bir Python scripti yaz" dersin — skill, `pyservicemaker` API'sinin doğru element adlarını ve pipeline sırasını kullanarak kodu üretir.

deepstream-generate-pipeline

Ne yapar: Kullanıcıya birkaç soru sorarak (interaktif anket — kaynak türü, model türü, cihaz mimarisi vb.) hazır çalıştırılabilir `gst-launch-1.0` komut satırı pipeline'ları üretir. Arka planda,

270'ten fazla doğrulanmış pipeline örneği üzerinde BM25 (bir metin benzerlik/arama algoritması, dış bağımlılık gerektirmeden) araması yaparak en uygun şablonu bulur ve yapısal detaylarla (hangi eleman hangi elemanla uyumlu) harmanlar. "Görüntüde çıkarım yapan pipeline", "video akışında nesne tespiti" gibi doğal dil istekleriyle tetikleniyor.

Örnek: Claude'a "4 RTSP akışında tespit ve takip yap, Jetson'da göster" dersin — skill soruların çoğunu isteğin içinden zaten çıkarır, eksik kalanları sorar ve doğrudan çalıştırılabilir `gst-launch-1.0` komutunu üretir.

deepstream-import-vision-model

Ne yapar: HuggingFace ya da NVIDIA NGC'den herhangi bir GÖRSEL NESNE TESPİTİ modelini alıp DeepStream pipeline'ına uçtan uca entegre eden otomasyon skill'i: ONNX (modelleri çerçeveler arası taşınabilir kılan açık format) indirme/dışa aktarma, TensorRT motoru (engine) derleme, modelin çıktısını DeepStream'in anlayacağı kutulara (bbox) çeviren özel bir ayrıştırıcı (parser) yazma, çoklu-akış benchmark'ı çalıştırma ve sonunda bir PDF rapor üretme. Şu an kapsam sadece nesne tespiti modelleriyle sınırlı — sınıflandırma (classification) veya bölütleme (segmentation) modelleri `config.json`'da tespit edilirse iş erken durduruluyor (fail fast).

Örnek: Claude'a "HuggingFace'teki şu YOLO modelini DeepStream'e bağla ve performansını ölç" dersin — skill modeli indirir, TensorRT motoruna çevirir, özel bbox ayrıştırıcısı yazar, tek-akış ve çoklu-akış benchmark'ı çalıştırıp grafikli bir PDF rapor teslim eder.

deepstream-profile-pipeline

Ne yapar: Elinde zaten çalışan bir DeepStream pipeline'ı olan kullanıcı için, tahmine dayalı ayar yapmak yerine Nsight Systems (NVIDIA'nın profil çıkarma/performans ölçüm aracı) ile pipeline'ı gerçekten ölçer — "çıkarmın düzleştiği (plateau) toplu-işlem (batch) boyutu" ve "donanımın tavan (ceiling) hızı" gibi iki somut sayı çıkarıp diğer tüm ayarları bu iki sayıdan türetir. Sonra uçtan-uca (E2E) pipeline'ı yine Nsight ile profilleyip her eklentinin (plugin) NVTX (Nsight'ın zaman-damgalama etiketleme sistemi) zamanlamasını raporluyor. Model ve pipeline'dan bağımsız çalışıyor — tespit, sınıflandırma, bölütleme, VLM ya da gömme (embedding) pipeline'larının hepsinde işliyor; sadece çıkarım elemanının `nvinfer` ya da `nvinferserver` olması yeterli.

Örnek: Claude'a "bu pipeline'ı verimli hale getir, FPS'i ölç" dersin — skill terminalde `nsys profile` ile pipeline'ı çalıştırıp `nsys stats` ile sonuçları çıkarır, gerçek ölçümlere göre batch/precision ayarlarını önerir.

deepstream-sop

Ne yapar: Fabrika/montaj hattında operatörlerin işlem adımlarını doğru sırada yapıp yapmadığını denetleyen "SOP (Standart Operasyon Prosedürü) Çıkarım Mikroservisi"ni kurma/genişletme/hata ayıklama/gecikme ölçme skill'i. İki aşamalı çalışıyor: önce GEBD (Generic Event Boundary Detection — videoda bir eylemin bittiği/yenisinin başladığı anı bulan bir bilgisayarlı-görüş tekniği, varsayılan model: DDM) ile video parçalara (chunk) bölünüyor, sonra her parça bir VLM (görüntü + dil modeli, varsayılan: Cosmos Reason) ile sınıflandırılıp prosedüre uygunluk kontrol ediliyor. Hem GEBD hem VLM modelleri değiştirilebilir (model-agnostik).

Örnek: Claude'a "bu montaj hattı kamerasında operatör adımları sırayla mı yapıyor kontrol eden servisi kur" dersin — skill GEBD ile video segmentlere böler, her segmenti VLM ile "doğru adım mı, eksik/sıra dışı mı" diye sınıflandıran servisi ayağa kaldırır.

Digital Health / Klinik ASR — Konuşma Tanıma Hattı

digital-health-clinical-asr-setup

Ne yapar: Klinik (tıbbi) konuşma tanıma (ASR — konuşmayı yazıya çeviren teknoloji) modellerini iyileştiren 4 aşamalı "flywheel" (döngüsel iyileştirme çarkı) sürecinin 1. aşaması — bir çalışma turunu (cycle) başlatan kurulum adımı. NVCF (NVIDIA Cloud Functions) ile ilgili veri-paylaşım açıklamasını gösteriyor, `NVIDIA_API_KEY` ortam değişkeninin doğru tanımlı olduğunu kontrol ediyor,

bağımlılıkları kuruyor ve tek bir klinik cümlelerin Magpie TTS (metinden-sese) → Parakeet/Nemotron ASR (sesten-metne) üzerinden başarıyla gidip geldiğini doğrulayan bir "duman testi" (smoke test) çalıştırıyor. Amaç: klinik terimlerdeki KER (Keyword Error Rate — anahtar kelime hata oranı, örn. ilaç/anatomi isimlerinin doğru tanınma oranı) metriğini aşama aşama düşürmek.

Örnek: Claude'a "klinik ASR döngüsünü başlat" dersin — skill API anahtarını kontrol eder, gerekli paketleri kurar ve bir tıbbi cümleyi TTS ile seslendirip ASR ile geri yazıya çevirerek uçtan uca bağlantının çalıştığını kanıtlar.

digital-health-clinical-asr-build

Ne yapar: Flywheel'in 2. aşaması — Stage 1'i geçen kullanıcı burada klinik terimleri derliyor (curate), bu terimlerin IPA (Uluslararası Fonetik Alfabe — kelimelerin gerçek telaffuzunu gösteren standart notasyon) etiketlemesini iki katmanlı bir yöntemle yapıyor (özel geçersiz kılma → Merriam-Webster sözlüğü → magpie_g2p otomatik telaffuz tahmini) ve sonunda NeMo (NVIDIA'nın konuşma/dil modeli araç seti) formatında bir manifest.jsonl (ses dosyalarına referans veren bir liste dosyası) sentezliyor. Süreç konuşmalı ve kapılı (gated) — modele/uzmanlığa özgü 1-2 açıklayıcı soru sorup QA-modu bir "prova" (audition) onayından geçtikten sonra tam üretime geçiyor.

Örnek: Claude'a "kardiyoloji terimleri için bir ASR test seti hazırla" dersin — skill önce hangi alt-uzmanlık/aksan olduğunu sorar, terimlerin telaffuzunu üç kademeli yöntemle etiketler, örnek seslendirmeleri onayına sunar ve sonunda eğitim/test için hazır bir manifest + ses dosyaları üretir.

digital-health-clinical-asr-eval

Ne yapar: Flywheel'in 3. aşaması — Stage 2'den gelen (ya da başka kaynaktan elde edilen) manifest.jsonl'ü seçilen bir ASR NIM (NVIDIA Inference Microservice — hazır paketlenmiş model servisi) ile yazıya çeviriyor, dört metrik üzerinden puanlıyor ve beş bölümlü bir KER lider tablosu (leaderboard) üretiyor — IPA kaynağına göre (özel geçersiz kılma/sözlük/otomatik-tahmin) kırılım dahil. Sonra bir karar ağacı ile kullanıcıyı ya fine-tune aşamasına, ya build'e geri dönmeye, ya da eval'i sıkılaştırmaya yönlendiriyor. Bu skill ses üretmiyor — manifest boşsa kullanıcıyı Stage 2'ye geri gönderiyor.

Örnek: Claude'a "bu manifest'i skorla, hangi model daha iyi çıkıyor göster" dersin — skill dört metriği hesaplayıp beş bölümlü tabloyu çıkarır ve "KER şu eşiğin üstünde, fine-tune'a geç" gibi somut bir öneriyle biter.

digital-health-clinical-asr-finetune

Ne yapar: Flywheel'in 4. aşaması — öncelikli KER değeri 0.3'ün üzerindeyse devreye girer; Parakeet TDT v2 (bir ASR modeli) üzerinde standart NeMo SFT (Supervised Fine-Tuning — etiketli veriyle ek eğitim) çalıştırır, sonra bir sonraki döngü (cycle N+1) için çevrimdışı yeniden-değerlendirme yapar. Genel amaçlı kelime-vurgulama (word boosting) için değil — spesifik olarak stok NeMo SFT tarifi, temel-model tablosu ve döngü kararı için var.

Örnek: Claude'a "KER hâlâ yüksek, modeli klinik terimlere göre ince ayar yap" dersin — skill uygun temel modeli önerir, standart NeMo SFT tarifini (hiperparametreler, container komutu) uygular ve yeni turun KER'ini eskisiyle karşılaştırır.

VSS (Video Search & Summarization) Ailesi

vss-ask-video

Ne yapar: VSS ajanının video_understanding aracını çağırarak kayıtlı bir video klibi hakkında YENİ bir görsel soru sorar — yani bir VLM'in (görüntü-dil modeli) videonun karelerine gerçekten "bakmasını" gerektiren durumlarda kullanılıyor. Var olan bir özet, arama sonucu ya da veritabanı cevabıyla zaten yanıtlanabilecek sorularda bu skill kullanılmıyor; sadece pikselleri yeniden görmek gerektiğinde devreye giriyor.

Örnek: Claude'a "bu klipte kaç kişi vardı, ne renk kıyafet giyorlardı" dersin — mevcut özet bu detayı içermiyorsa skill VLM'i tetikleyip videoyu yeniden izleterek cevap üretir.

vss-deploy-dense-captioning

Ne yapar: RT-VLM (gerçek-zamanlı görüntü-dil modeli) "dense captioning" (yoğun altyazılama — videonun her anını/segmentini sürekli metinle betimleme) mikroservisini bağımsız olarak kurup, tüm REST uç noktalarını (dosya yükleme, altyazı üretme, akış ekleme/silme, chat-completions, Kafka konuları) çalıştırır. Tam VSS profili kurulumu ya da video-arama alımı (ingestion) için değil, sadece bu tek servis için.

Örnek: Claude'a "RT-VLM dense captioning servisini kur ve bir kamera akışı ekle" dersin — skill Docker Compose ile servisi ayağa kaldırır, NGC'den modeli çeker ve akış ekleme/altyazı üretme uç noktalarını test eder.

vss-deploy-detection-tracking-2d

Ne yapar: RTVI-CV adlı 2 boyutlu nesne tespiti/takip mikroservisini (tek kamera, kutu-tabanlı tespit + kareler arası takip) kurma, çalıştırma, hata ayıklama, kapatma ve REST API'sini çağırma skill'i. "rtvi-cv deploy et", "depoda 2D başlat", "akış ekle", "sağlık kontrolü yap" gibi isteklerle tetikleniyor. VLM, gömme (embedding) ya da analitik istekleri için değil.

Örnek: Claude'a "depo kamerasında 2D tespit/takip servisini başlat" dersin — skill mikroservisi kurar, bir kamera akışı ekler ve servis sağlığını (health check) doğrular.

vss-deploy-detection-tracking-3d

Ne yapar: RTVI-CV-3D mikroservisini "MV3DT" (Multi-View 3D Tracking — çoklu kamera görünümünden 3B takip) modunda kurar: her kamera için ayrı bir DeepStream algı (perception) hattı + BEV Fusion (Bird's Eye View Fusion — birden çok kalibre kameradan gelen görüntüleri tek bir kuşbakışı 3B haritada birleştirme). Örnek veri seti, kullanıcının kendi videoları veya canlı RTSP ile çalışabiliyor; kalibrasyon eksikse otomatik olarak vss-generate-video-calibration skill'ine yönlendiriyor. Tam depo (warehouse) blueprint'i için vss-deploy-profile, tek-kamera 2B tespit için vss-deploy-detection-tracking-2d kullanılıyor.

Örnek: Claude'a "birden fazla kamerayla çoklu-kamera 3B takip aç" dersin — skill önce kameraların kalibre olup olmadığını kontrol eder, gerekirse kalibrasyon akışını tetikler, sonra her kamera için DeepStream algı hattı + BEV Fusion'ı birlikte kurar.

vss-deploy-profile

Ne yapar: VSS'in önceden tanımlı büyük "profil" paketlerinden birini (base, search, lvs [canlı video özetleme], warehouse [depo], edge/alerts) seçme, yapılandırma, kurma, doğrulama, hata ayıklama ve kapatma skill'i — yani tekil mikroservisler yerine TÜM YIĞINI (stack) bir arada yöneten üst-düzey orkestrasyon skill'i. Kullanıcının isteğini ("deploy vss", "depo blueprint'i kur", "canlı özetleme kur" gibi) doğru profile eşleyip o profilin boyutlandırma/servis/ortam-değişkeni/hata-ayıklama referansına yönlendiriyor.

Örnek: Claude'a "VSS depo blueprint'ini kur" dersin — skill isteği "warehouse" profiline eşler, o profile özel servisleri, ortam değişkenlerini ve boyutlandırma gereksinimlerini uygulayarak tüm yığını ayağa kaldırır.

vss-deploy-video-embedding

Ne yapar: VSS 3.2 GA "RT-Embed" Video Gömme (embedding — bir görüntü/videoyu/metni, benzerlik araması yapılabilecek sayısal bir vektöre çeviren işlem) mikroservisini kurup çalıştırır. Cosmos-Embed1-448p modeliyle metin ya da video gömmeleri üretiyor; yüklenen dosya, HTTP/S3/dosya/veri URL'si ya da canlı RTSP akışını görebiliyor. Redis (hızlı bellek-içi veri deposu), Kafka (mesaj kuyruğu/akış sistemi) ve OTel (OpenTelemetry — izleme/gözlemlenebilirlik standardı) entegrasyonlarını, ayrıca GPU/model-indirme/akış-yeniden-bağlanma hatalarının teşhisini kapsıyor.

Örnek: Claude'a "bu video klibinin gömme vektörünü çıkar, arama indeksine ekle" dersin — skill dosyayı RT-Embed servisine yükler, Cosmos-Embed1 modeliyle vektörü üretir ve servis sağlığını/GPU kullanımını doğrular.

vss-generate-video-calibration

Ne yapar: AMC'yi (AutoMagicCalib) VSS bağlamında çalıştıran sarmalayıcı (wrapper) skill — yerel MP4 dosyaları, canlı RTSP akışları ya da paketle gelen örnek veri seti üzerinde kalibrasyonu uçtan uca yürütüyor, gerekirse vss-auto-calibration mikroservisini kendisi kuruyor. Bu, çoklu-kamera 3B takip (vss-deploy-detection-tracking-3d) öncesinde kameraların birbirine göre konumunu hesaplayan adım.

Örnek: Claude'a "depo kameralarını kalibre et, sonra 3B takibi aç" dersin — skill önce girdi türünü (dosya/RTSP/örnek veri) belirler, AMC servisini gerekirse kurar ve kalibrasyonu tamamlar.

vss-generate-video-report

Ne yapar: İki farklı arka-uçtan (backend) birini seçerek bir VSS analiz raporu üretir — asla doğrudan VSS ajanının /generate uç noktasından değil. Mod A: tek bir kayıtlı klip için VIOS'tan (video giriş/çıkış deposu) klip URL'si alıp VLM chat/completions ile betimliyor. Mod B: bir zaman aralığı/olay listesi için vss-query-analytics'ten olay (incident) listesi çekip anlatı (narrative) tarzı bir rapor yazıyor. İstek belirsizse (sensör var ama zaman aralığı yoksa) varsayılan olarak Mod A kullanılıyor.

Örnek: Claude'a "şu kameranın dünkü 14:00-15:00 arası olaylarını raporla" dersin — skill zaman aralığı + olay ifadesi içerdiği için Mod B'yi seçer, analitik sorgusundan olay listesini çekip anlatı rapor yazar.

vss-manage-alerts

Ne yapar: VSS'in uyarı (alert) hattını yönetir — gerçek-zamanlı izleme/mod tespiti, Alert-Bridge abonelikleri, Slack bildirimleri, geçmiş olay sorguları, yeni kamera ekleme (onboarding) ve doğrulayıcı-prompt (verifier-prompt — bir olayın gerçek bir uyarı mı yoksa yanlış-pozitif mi olduğunu kontrol eden AI talimatı) özelleştirmesi dahil.

Örnek: Claude'a "bu kamerada hareket algılanınca Slack'e bildirim gönder" dersin — skill Alert-Bridge aboneliğini kurar, hangi olay türlerinin bildirim tetikleyeceğini yapılandırır.

vss-manage-video-io-storage

Ne yapar: VIOS (Video Input/Output Storage) ve NvStreamer API'lerini çağırarak kameraları/sensörleri, RTSP akışlarını, dosya yüklemelerini, anlık görüntüleri (snapshot), klip çıkarımını, zaman çizelgelerini (timeline) ve kayıt durumunu yönetir. Kamera ekleme, akış listeleme, klip indirme, dosya yükleme gibi istekler UI yerine doğrudan VIOS REST API'si üzerinden curl ile yapılıyor.

Örnek: Claude'a "yeni bir RTSP kamerası ekle, adı depo-giris-1 olsun" dersin — skill VIOS API'sine doğrudan istek atarak kamerayı sisteme kaydeder, akışın durumunu doğrular.

vss-query-analytics

Ne yapar: VA-MCP sunucusu (port 9901) üzerinden salt-okunur (read-only) analitik sorular — metrikler, olaylar (incidents), uyarılar, sensör verisi — yanıtlar. Canlı VLM sorguları ya da olay-aralığı anlatı raporları için değil (onlar sırasıyla vss-ask-video ve vss-generate-video-report skill'lerinde).

Örnek: Claude'a "bu hafta kaç güvenlik ihlali olayı kaydedildi" dersin — skill VA-MCP sunucusundan olay metriklerini sorgulayıp sayısal cevabı döndürür.

vss-search-archive

Ne yapar: Arşivlenmiş video üzerinde VSS'in en üst-düzey "füzyon araması"nı (fusion search — muhtemelen metin, gömme-vektör ve meta-veri sinyallerini birleştiren birleşik bir arama) çalıştırır; ayrıca aramaya dahil edilecek yeni video dosyalarını/RTSP akışlarını içe aktarır (ingest) ve arama-kaynaklarını silebilir. Ad-hoc görsel soru-cevap (vss-ask-video), canlı altyazılama (vss-deploy-dense-captioning) ya da özetleme/rapor (vss-summarize-video) için kullanılmıyor.

Örnek: Claude'a "arşivde sarı forklift geçen tüm klipleri bul" dersin — skill füzyon arama sorgusunu çalıştırıp eşleşen klip listesini döndürür.

vss-setup-behavior-analytics

Ne yapar: "Davranış analitiği" (behavior-analytics) servisini, tam depo (warehouse) kurulumunun tamamını kurmadan, tek başına ayağa kaldırır — giriş noktası (entrypoint), yapılandırma kaynağı ve isteğe bağlı kalibrasyon seçimiyle. Servisin tam olarak neyi analiz ettiği JSON verisinde detaylandırılmamış; muhtemelen depo/kamera görüntüsündeki hareket örüntülerini (bekleme süresi, güzergah gibi) çıkarıyor.

Örnek: Claude'a "davranış analitiği servisini tek başına kur, tam depo yığını istemiyorum" dersin — skill Docker Compose'daki ilgili tek servisi seçilen giriş noktası ve config kaynağıyla başlatır.

vss-setup-video-analytics-api

Ne yapar: "video-analytics-api" REST servisini bağımsız olarak kurar — yapılandırma kaynağı, veri-log klasör bağlama (bind), Elasticsearch (metin/veri arama-indeksleme motoru) ve isteğe bağlı Kafka bağlantısıyla. Bu servis muhtemelen vss-query-analytics skill'inin okuduğu API katmanı — yani analitik verinin depolandığı/sorgulandığı arka uç.

Örnek: Claude'a "video-analytics-api servisini Elasticsearch'e bağlı şekilde kur" dersin — skill Docker Compose ile servisi başlatıp /livez uç noktasından sağlık durumunu doğrular.

vss-summarize-video

Ne yapar: Kayıtlı bir video klibini LVS (muhtemelen "Live/Long Video Summarization" mikroservisi) üzerinden özetler; bu servise ulaşılamazsa VLM'e (görüntü-dil modeli) düşer (fallback). Süreç HITL (Human-In-The-Loop — bir insanın onayının/gözden geçirmesinin gerektiği) kapılı; sonuçta zaman-damgalı olaylar içeren tek, düzgün bir anlatı özeti üretiliyor. Rapor üretimi (vss-generate-video-report) ya da canlı RTSP altyazılama (vss-deploy-dense-captioning) için kullanılmıyor.

Örnek: Claude'a "bu videoyu özetle" dersin — skill LVS mikroservisine (ya da o erişilemezse VLM'e) videoyu gönderip zaman-damgalı olayları içeren tek paragraflık bir özet üretir.

Hızlandırılmış Hesaplama Ailesi (cuOpt, cuDF, cuPyNumeric, CUDA-Q, Earth2Studio, PhysicsNeMo, TileGym, cuFolio)

Bu ailedeki kütüphanelerin ortak fikri şu: normalde CPU'da (bilgisayarın genel amaçlı işlemcisi) saatler, hatta günler süren devasa hesaplamaları, GPU'nun (ekran kartındaki binlerce paralel çekirdek) gücüyle dakikalara, bazen saniyelere indirmek. Her biri farklı bir alana odaklanır: **cuOpt** lojistik ve optimizasyon problemlerini (hangi kamyon hangi rotayı izlesin, hangi ürün karışımı en kârlı) çözer; **cuDF** ve **cuPyNumeric**, pandas ve NumPy gibi klasik veri işleme araçlarının GPU'da çalışan hâlleridir; **CUDA-Q** kuantum bilgisayar programlama ve simülasyon platformudur; **Earth2Studio** hava durumu/iklim tahmini yapan yapay-zekâ modellerini çalıştırır; **PhysicsNeMo** fizik simülasyonlarını (akışkanlar, ısı transferi, malzeme bilimi) yapay-zekâ ile hızlandırır; **TileGym** GPU çekirdeği (kernel — donanımda doğrudan çalışan küçük hesaplama programı) yazmak için düşük seviyeli bir araç kutusudur; **cuFolio** ise cuOpt'u kullanarak finansal portföy optimizasyonu yapan özel bir uygulamadır. Günlük hayattan bir benzetme: CPU hesaplama tek kasiyerli bir markete benzer — herkes sırayla işlem görür; GPU hesaplama ise binlerce kasiyerin aynı anda çalıştığı dev bir hipermarket gibidir, basit ama çok sayıda işlemi aynı anda halledebilir. Önemli bir nokta: bu skill'lerin kendisi hesap yapmaz, kod da çalıştırmaz — Claude gibi bir yapay-zekâ kod asistanına "bu işi NVIDIA'nın önerdiği doğru şekilde nasıl yaparım" diye öğretir. Yani her biri, NVIDIA'nın resmi dokümantasyonunun asistan tarafından okunup uygulanabilir hâle getirilmiş bir özeti — asistan bu talimatı okuyunca doğru kütüphaneyi, doğru API'yi, doğru komutu seçiyor.

accelerated-computing-cudf

Ne yapar: cuDF, pandas'ın (Python'da tablo/veri işleme için kullanılan çok yaygın kütüphane) GPU hızlandırılmış hâlidir. Bu skill Claude'a cuDF ve dask-cuDF (birden fazla GPU'ya dağıtılmış cuDF) ile doğru kod yazmayı öğretir: tablo birleştirme (join), gruplama (groupby), CSV/Parquet dosya okuma-yazma gibi işlemler. cuDF'in "tatlı noktası" 100 binden fazla satırlık veri — daha küçük veride CPU zaten yeterince hızlıdır.

Örnek: Claude'a şöyle dersin: "500 bin satırlık film metadata CSV'sini pandas ile işliyorum ama groupby çok yavaş, GPU'ya taşı" — skill devreye girer ve ya cudf.pandas ile neredeyse hiç kod değiştirmeden hızlandırma yapar, ya da doğrudan cuDF API'siyle daha agresif bir çözüm yazar.

cudaq-guide

Ne yapar: CUDA-Q, kuantum bilgisayar programları (kuantum "kernel"leri) yazmak, bunları GPU üzerinde simüle etmek veya gerçek kuantum donanımına (QPU — Quantum Processing Unit) bağlanmak için NVIDIA'nın platformudur. Bu skill kurulumdan başlayıp ilk test programlarına kadar rehberlik eden bir tür başlangıç menüsü gibi çalışır.

Örnek: Claude'a şöyle dersin: "CUDA-Q kurulumunu yap ve basit bir kuantum devresi çalıştır" — skill devreye girer, gereksinimleri (Python 3.10+, CUDA Toolkit, isteğe bağlı NVIDIA GPU) kontrol eder ve ilk örnek kodu adım adım gösterir.

cufolio

Ne yapar: cuFOLIO, hisse senedi portföyü oluşturma, optimize etme ve geriye dönük test etme (backtest) için NVIDIA'nın cuOpt çözücüsünü kullanan özel bir uygulamadır. "Mean-CVaR" denen bir risk yöntemiyle (ortalama getiriyi maksimize ederken en kötü senaryodaki kaybı sınırlayan bir matematiksel yaklaşım) portföy ağırlıklarını hesaplar ve "etkin sınır" (her risk seviyesinde ulaşılabilecek en iyi getiriyi gösteren eğri) çizer.

Örnek: Claude'a şöyle dersin: "S&P 500 hisselerinden risk-ayarlı bir portföy oluştur, kötü senaryo kaybını sınırla" — skill devreye girer, fiyat verisinden getiri hesaplar, senaryolar üretir ve cuOpt ile optimum ağırlıkları bulur.

cuopt-developer

Ne yapar: Bu skill cuOpt'u KULLANMAK için değil, cuOpt'un kendi kaynak koduna (C++/CUDA, Python, sunucu tarafı) katkıda bulunmak, hata ayıklamak ya da değiştirmek isteyenler içindir — yani "cuOpt'u nasıl kullanırım" değil "cuOpt'un içini nasıl değiştiririm" sorusuna cevap verir. Güvenlik gereği sudo/root gibi sistem seviyesi işlemleri kesinlikle reddeder.

Örnek: Claude'a şöyle dersin: "cuOpt'un rota çözücüsünde bir bug buldum, kaynak koddan düzeltmek istiyorum" — skill devreye girer ve geliştirme ortamı kurulumundan (conda, derleme, test) kod katkı sürecine (PR, DCO onayı) kadar yol gösterir.

cuopt-install

Ne yapar: cuOpt'u Python, C veya REST sunucusu olarak kurmak (pip/conda/Docker ile) ve kurulumun gerçekten çalıştığını doğrulamak için kullanılır. Kaynak koddan derleme bu skill'in kapsamında değildir, sadece hazır paket kurulumu.

Örnek: Claude'a şöyle dersin: "cuOpt'u makinemde kullanıma hazır hale getir" — skill devreye girer, önce GPU/CUDA sürüm uyumunu (Volta veya sonrası, CUDA 12/13) sorar, sonra doğru pip/conda/Docker komutunu verir.

cuopt-multi-objective-exploration

Ne yapar: cuOpt tek seferde sadece TEK bir hedefi (örneğin sadece maliyet) optimize eder; ama gerçek dünyada "maliyet düşük OLSUN ama teslimat hızı da yüksek olsun" gibi birbiriyle çelişen birden çok hedef olur. Bu skill, aynı problemi farklı ağırlıklarla defalarca çözerek "Pareto sınırı" denen bir uzlaşma eğrisi çıkarır — yani "birini iyileştirmek istersen diğerinden ne kadar fedakârlık etmen gerekir" tablosunu gösterir.

Örnek: Claude'a şöyle dersin: "Rota planlamamda hem mesafeyi hem araç sayısını minimize etmek istiyorum ama ikisi çelişiyor" — skill devreye girer, cuOpt'u farklı ağırlıklarla tekrar tekrar çalıştırıp sana ödünleşim eğrisini çizer.

cuopt-numerical-optimization-api

Ne yapar: cuOpt'un LP (doğrusal programlama), MILP (tam sayı kısıtlı doğrusal programlama) ve QP (ikinci dereceden/karesel programlama, hâlâ beta aşamasında) problemlerini Python, C veya komut satırından nasıl çözeceğini gösteren API rehberidir. Kullanıcının hangi arayüzü kullandığına göre doğru referansa yönlendirir; ayrıca AMPL, GAMS, PuLP, Pyomo gibi mevcut modelleme araçlarına da cuOpt'u çözücü motoru olarak neredeyse hiç kod değişikliği gerekmeden bağlayabilir.

Örnek: Claude'a şöyle dersin: "PuLP ile yazdığım bir MILP modelini GPU'da çözdürmek istiyorum" — skill devreye girer ve modelin çözücüsünü cuOpt'a çevirecek minimal kod değişikliğini gösterir.

cuopt-numerical-optimization-formulation

Ne yapar: Bu, kod yazmadan ÖNCEKİ aşamadır — bir problemi LP/MILP/QP diline nasıl çevireceğini (karar değişkenleri, kısıtlar, amaç fonksiyonu) öğretir. Yani "bu gerçek dünya problemi hangi matematiksel kalıba uyuyor" sorusuna cevap verir; API kodu içermez.

Örnek: Claude'a şöyle dersin: "Elimde 50 ürün var, hammadde kısıtlı, hangi karışımı üretsem kârı maksimize ederim" — skill devreye girer, problemi önce karar değişkenleri/kısıtlar/amaç fonksiyonu olarak MILP formuna döker, ardından gerçek kodlama için API skill'ine yönlendirir.

cuopt-routing-api-python

Ne yapar: Araç rotalama problemlerini (VRP: birden fazla araçla teslimat rotası, TSP: gezgin satıcı problemi, PDP: alma-bırakma problemi) sadece Python API'siyle çözmeyi öğretir — rotalama özelliğinin ayrı bir C API'si yoktur. Depo sayısı, araç kapasitesi, zaman penceresi gibi kısıtları önce sorar, sonra kodu yazar.

Örnek: Claude'a şöyle dersin: "10 kamyonla 200 müşteriye teslimat rotası planla, her kamyonun kapasitesi farklı" — skill devreye girer, konum/kapasite/zaman penceresi bilgilerini sorup cuOpt'un

routing.DataModel ve routing.Solve API'siyle çalışan Python kodunu üretir.

cuopt-server-api-python

Ne yapar: cuOpt'u yerel kurulum yerine bir REST sunucusu (ağ üzerinden HTTP istekleriyle çağrılabilen bir servis) olarak çalıştırmayı ve buna Python veya curl ile nasıl istek atılacağını gösterir. Rotalama, LP ve MILP problemlerini destekler; QP bu yoldan desteklenmez.

Örnek: Claude'a şöyle dersin: "cuOpt'u Docker'da sunucu olarak ayağa kaldırıp başka bir uygulamadan HTTP ile çağırmak istiyorum" — skill devreye girer, docker run komutunu, sağlık kontrolünü (/cuopt/health) ve istek gönderip sonuç alma akışını (POST → reqld → sonucu sorgulama) gösterir.

cupynumeric-hdf5

Ne yapar: cuPyNumeric (NumPy'nin çoklu-GPU/dağıtık hızlandırılmış hâli) dizilerini HDF5 denen büyük bilimsel veri dosya formatına kaydetmeyi ve okumayı öğretir. Legate altyapısı sayesinde her GPU kendi parçasını paralel olarak okur/yazar — veriyi tek bir işlemciden geçirmeye gerek kalmaz; GPUDirect Storage ile diskten okuma da hızlandırılabilir.

Örnek: Claude'a şöyle dersin: "100 GB'lık bir cuPyNumeric dizisini .h5 dosyasına kaydetmem gerekiyor, her GPU kendi parçasını yazsın" — skill devreye girer ve legate.io.hdf5.to_file / from_file ile paralel okuma-yazma kodunu verir.

cupynumeric-install

Ne yapar: cuPyNumeric'i Python'da kullanılabilir hale getirmek için kurulum komutlarını ve GPU'nun gerçekten kullanıldığını doğrulama adımlarını verir. Kurulumu kendisi ASLA çalıştırmaz — sadece doğru komutu yazar, kullanıcı kendi elleriyle çalıştırır. Kaynak koddan derleme bu skill'in kapsamı dışındadır.

Örnek: Claude'a şöyle dersin: "cuPyNumeric'i conda ile kurmak istiyorum, hangi komutu çalıştırayım" — skill devreye girer, GPU/CUDA/Python sürüm uyumunu kontrol eder ve doğru conda/pip komutunu önerir; çalıştırma kararı sende kalır.

cupynumeric-migration-readiness

Ne yapar: Mevcut NumPy koduna GPU'ya TAŞIMADAN ÖNCE bakıp "bu kod GPU'da gerçekten hızlanır mı, yoksa önce yeniden yazılması mı gerekir" sorusuna cevap verir. Kodu sadece okur — çalıştırmaz, değiştirmez — ve her NumPy kalıbını "dağıtık ölçeklenmeye uygun" ya da "engel" (desteklenmeyen API, gereksiz CPU-GPU arası veri transferi gibi) diye sınıflandırıp gerekçeli bir rapor üretir.

Örnek: Claude'a şöyle dersin: "Bu 2000 satırlık NumPy simülasyon kodunu cuPyNumeric'e taşımadan önce riskleri görmek istiyorum" — skill devreye girer, kodu satır satır tarar ve "şu fonksiyon iyi ölçeklenir, şu satır GPU'dan gereksiz veri geri çekiyor, önce onu düzelt" tarzında bir bulgu listesi verir.

cupynumeric-parallel-data-load

Ne yapar: Tek dosyaya sığmayan, parçalara bölünmüş (sharded) büyük veri setlerini (özel format, parçalı Parquet/Arrow, ham ikili dosya, parçalı HDF5) dağıtık bir cuPyNumeric dizisine yüklemenin yolunu gösterir. Hazır bir yükleyici yoksa (örn. cupynumeric.load tek dosya için yeterli değilse), her GPU'nun kendi parçasını okuduğu bir Legate görevi (task) yazdırır.

Örnek: Claude'a şöyle dersin: "100 parçaya bölünmüş özel formatlı dosyalarım var, hepsini tek bir dağıtık diziye yüklemem lazım" — skill devreye girer ve her parçayı ayrı bir GPU görevine atayan Legate @task kalıbını yazar.

earth2studio-create-datasource

Ne yapar: Earth2Studio (NVIDIA'nın hava durumu/iklim yapay-zekâ platformu) için yeni bir "veri kaynağı" bağlayıcısı — S3, GCS, Azure, HTTP veya HuggingFace gibi uzak depolardan veri çeken bir

modül — yazmayı adım adım yönetir. Analiz, kodlama, test, doğrulama ve kod katkısı (PR) sürecinin tamamını kapsar.

Örnek: Claude'a şöyle dersin: "Earth2Studio'ya yeni bir uydu verisi kaynağı eklemek istiyorum, verim Azure'da duruyor" — skill devreye girer, DataSource sınıfını adım adım yazdırır, test eder ve örnek grafiklerle sonucu doğrular.

earth2studio-create-diagnostic

Ne yapar: Earth2Studio için "diagnostic" model sarmalayıcısı yazmayı öğretir — bu, mevcut hava verisinden tek adımda türetilmiş yeni bir değişken hesaplayan bir model türüdür; basit bir formülden karmaşık üretici/difüzyon tabanlı modele kadar üç farklı sınıfı kapsar.

Örnek: Claude'a şöyle dersin: "Sıcaklık ve nem verisinden hissedilen sıcaklığı hesaplayan bir diagnostic model eklemek istiyorum" — skill devreye girer, earth2studio/models/dx/ altında doğru sınıf yapısını ve mock (sahte veriyle) testini oluşturur.

earth2studio-create-prognostic

Ne yapar: "Prognostic" model, zaman içinde ileriye doğru tahmin üreten (bugünün havasından yarının havasını hesaplayan) model türüdür. Bu skill üçüncü parti bir yapay-zekâ hava tahmin modelini Earth2Studio'ya bağlamak için gereken özel sınıf yapısını (üçlü kalıtım) ve testleri sırayla oluşturur.

Örnek: Claude'a şöyle dersin: "Yeni yayınlanan bir hava tahmin modelini Earth2Studio'ya entegre etmek istiyorum, model 6 saatlik adımlarla ileri tahmin yapıyor" — skill devreye girer, earth2studio/models/px/ altında model sarmalayıcısını ve testlerini yazdırır.

earth2studio-data-fetch

Ne yapar: Belirli değişkenler (sıcaklık, rüzgar vb.) ve zamanlar için Earth2Studio veri kaynaklarından hava/iklim verisi indiren bir betik yazmayı öğretir; hangi kaynağın istenen değişkeni desteklediğini "lexicon" (kaynaklar arası uyumluluk sözlüğü) üzerinden kontrol ederek doğru eşleşmeyi bulur.

Örnek: Claude'a şöyle dersin: "2020-2023 arası global 2 metre sıcaklık verisini indirmem lazım" — skill devreye girer, uygun veri kaynağını (GCS/S3/CDS gibi) bulur ve xarray formatında veri döndüren çalışan bir Python betiği yazar.

earth2studio-deterministic-forecast

Ne yapar: Tek-üyeli (ensemble/topluluk değil, tek bir kesin sonuç veren) hava tahmini üretmek için model + veri kaynağı + çıktı kaydetme (IO) + çıkarım adımlarını birleştiren uçtan uca bir tahmin betiği kurmayı öğretir.

Örnek: Claude'a şöyle dersin: "Şu modelle 5 günlük deterministik bir Avrupa sıcaklık tahmini üretmek istiyorum" — skill devreye girer, uygun modeli ve veri kaynağını earth2studio.run.deterministic çağrısıyla birleştiren betiği yazar.

earth2studio-discover

Ne yapar: "Hangi model/veri kaynağı benim işime yarar" sorusuna cevap veren bir keşif/rehberlik aracıdır — kendisi kod yazmaz; GPU/VRAM (ekran kartı belleği) gereksinimine, tahmin ufkuna (kisa vadeli/orta vadeli/mevsimsel) göre canlı NVIDIA dokümantasyonundan doğru bileşenleri bulup önerir.

Örnek: Claude'a şöyle dersin: "Elimde tek bir GPU var, orta vadeli (5-10 gün) global tahmin yapabilecek hangi modeller var" — skill devreye girer, güncel model listesini VRAM/bölge/sınıf etiketleriyle karşılaştırıp sana en uygun olanları önerir.

earth2studio-install

Ne yapar: Earth2Studio'yu ve ihtiyaç duyulan opsiyonel model paketlerini (extras) kurmak için doğru komutu ve ortam değişkeni ayarlarını verir. Kurulumu kendisi ÇALIŞTIRMAZ; sadece komutu sunar ve kullanıcının bunu kendisinin çalıştırmasını bekler.

Örnek: Claude'a şöyle dersin: "Earth2Studio'yu belirli bir model ailesiyle birlikte kurmak istiyorum" — skill devreye girer, hangi opsiyonel paketin gerektiğini açıklar ve tam uv/pip komutunu verir, çalıştırmayı sana bırakır.

physicsnemo-discover

Ne yapar: PhysicsNeMo (fizik simülasyonlarını — akışkanlar dinamiği, ısı transferi, malzeme bilimi gibi — yapay-zekâ ile hızlandıran NVIDIA kütüphanesi) içinde hangi model/veri-hattı (datapipe)/örneğin işine yaradığını bulmaya yardım eder. Kendisi kod yazmaz; sadece repo'yu canlı tarayıp doğru dosya ve klasörlere yönlendirir, çünkü kütüphane sık değiştiği için ezberlenmiş bir liste güvenilmez olurdu.

Örnek: Claude'a şöyle dersin: "Bir akışkan simülasyonu için sinir ağı tabanlı bir vekil (surrogate) model arıyorum, PhysicsNeMo'da hangi örnek buna uyar" — skill devreye girer, model ailesi x veri-hattı x eğitim stratejisi eksenlerinde arama yapıp en yakın örnek dosyaları gösterir.

tilegym-adding-cutile-kernel

Ne yapar: TileGym (GPU çekirdekleri için performans test/geliştirme ortamı) içine yeni bir cuTile tabanlı GPU işlemi (operatör) eklemenin adım adım sırasını zorunlu bir kontrol listesiyle yönetir: dispatch (yönlendirme) kaydı, cuTile arka uç implementasyonu, dışa aktarım, test ve benchmark — hiçbir adım atlanamaz.

Örnek: Claude'a şöyle dersin: "TileGym'e yeni bir layer_norm operatörü eklemek istiyorum" — skill devreye girer ve sırasıyla ops.py'de kayıt, cuTile implementasyonu, __init__.py dışa aktarımı ve test/benchmark dosyalarını oluşturur.

tilegym-converting-cutile-to-julia

Ne yapar: Python'da yazılmış cuTile GPU çekirdeklerini (@ct.kernel) Julia dilindeki cuTile.jl karşılığına çevirir. Python'un 0'dan başlayan indeksleme sistemini Julia'nın 1'den başlayan sistemine, satır-öncelikli bellek düzenini sütun-öncelikli düzene çevirmek gibi dile özgü teknik farkları yönetir.

Örnek: Claude'a şöyle dersin: "Python'da yazdığım matris çarpım cuTile çekirdeğini Julia'ya taşımam gerekiyor" — skill devreye girer, indeksleme/bellek düzeni/tip sistemi farklarını gözeterek eşdeğer Julia kodunu yazar.

tilegym-converting-cutile-to-triton

Ne yapar: cuTile çekirdeklerini Triton'a (başka bir popüler, GPU çekirdeği yazmak için kullanılan Python tabanlı dil) çevirir; standart dönüştürmenin yanı sıra hata ayıklama (bellek adresi hatası, şekil uyumsuzluğu, sayısal fark) ve performans-kritik durumlar (dikkat/attention mekanizması gibi) için ayrı stratejiler içerir.

Örnek: Claude'a şöyle dersin: "Bu cuTile attention çekirdeğini Triton'a çevirmem lazım ve performansı da korumam gerekiyor" — skill devreye girer, önce özel optimizasyon stratejisini uygular, sonra analiz → dönüştür → doğrula → test → benchmark adımlarını sırayla işletir.

tilegym-cutile-autotuning

Ne yapar: Bir cuTile GPU çekirdeğine "autotuning" (farklı parametre kombinasyonlarını otomatik deneyip en hızlısını seçme) yeteneği ekler — karo (tile — GPU'nun işlediği küçük veri bloğu) boyutu, doluluk oranı (occupancy) gibi parametreleri farklı GPU mimarilerine (sm80'den sm120'ye) göre ayarlar ve "bir kere ayarla, sonucu önbelleğe al, sonra doğrudan çalıştır" kalıbını uygular.

Örnek: Claude'a şöyle dersin: "Bu çekirdek her GPU mimarisinde farklı performans veriyor, otomatik en iyi ayarı bulsun" — skill devreye girer, parametre arama uzayını tasarlar, autotuning

kodunu ekler ve sabit bir konfigürasyona karşı A/B karşılaştırmasıyla doğrular.

tilegym-cutile-python

Ne yapar: cuTile ile GPU çekirdeği yazma, hata ayıklama ve optimize etme konusunda genel amaçlı bir uzman rehberdir. cuTile, NVIDIA GPU'ları için "karo" (tile) tabanlı bir programlama modelidir ve tensor çekirdekleri gibi ileri donanım özelliklerinden otomatik olarak yararlanır.

Örnek: Claude'a şöyle dersin: "Sıfırdan bir toplama/çarpma cuTile çekirdeği yazmak istiyorum, float16 kullanacağım" — skill devreye girer, cuTile'ın API'sini ve doğru/optimize kod kalıplarını kullanarak çekirdeği yazar, dener ve doğrular.

tilegym-improve-cutile-kernel-perf

Ne yapar: Var olan bir cuTile çekirdeğinin performansını sistematik olarak iyileştirir — önce profil çıkarır, darboğazı ("aritmetik yoğunluk" ölçüsüyle bellek-sınırlı mı işlem-sınırlı mı olduğunu) sınıflandırır, sonra karo boyutu/doluluk/autotune ayarları gibi noktaları döngüsel biçimde dener, ölçer ve kaydeder.

Örnek: Claude'a şöyle dersin: "Bu matmul çekirdeği yavaş çalışıyor, adım adım hızlandırmak istiyorum" — skill devreye girer, yeni bir git dalı açar, çekirdeği bellek-sınırlı/işlem-sınırlı diye sınıflandırır ve deney-ölç-iyileştir döngüsünü başlatır.

tilegym-monkey-patch-kernels-to-transformers

Ne yapar: TileGym'de yazılan hızlı GPU çekirdeklerini, Hugging Face'in transformers kütüphanesindeki hazır dil modellerine (LLM) "monkey-patch" denen bir teknikle entegre eder — bu teknik, kütüphanenin kaynak kodunu değiştirmeden, çalışma anında ilgili sınıf/metotları TileGym implementasyonlarıyla değiştirmek anlamına gelir. Amaç, modelin uçtan uca doğruluğunu ve hızını gerçek bir model üzerinde doğrulamaktır.

Örnek: Claude'a şöyle dersin: "TileGym'de yazdığım hızlı attention çekirdeğini gerçek bir Hugging Face modeliyle test etmek istiyorum, kütüphane kaynağını değiştirmeden" — skill devreye girer, modelin ilgili sınıflarını çalışma anında TileGym çekirdekleriyle değiştiren monkey-patch kodunu kurar ve doğruluk/performans karşılaştırması yapar.

Fiziksel Yapay Zekâ / Omniverse Ailesi

Omniverse, NVIDIA'nın bir fabrikanın, bir robotun ya da bir şehrin **dijital ikizini** (gerçek dünyadaki bir şeyin bilgisayar içinde birebir 3D kopyasını) kurmaya yarayan platformu. Bu platformun ortak "dil"i **USD** (Universal Scene Description) adlı dosya formatı — Pixar tarafından geliştirilmiş, farklı 3D programların (Maya, Blender, Omniverse'in kendisi) aynı sahneyi okuyup üzerinde çalışabilmesini sağlıyor; belge dünyasında PDF'in oynadığı role benzer bir rol oynuyor 3D dünyasında. **Fiziksel Yapay Zekâ (Physical AI)** ise gerçek dünyayı "anlayan" ve "simüle eden" modeller ailesi: bir video izleyip o sahneyi 3D olarak yeniden inşa edebilen (**NuRec** — videodan 3D sahne yeniden kurma teknolojisi), üretim hattındaki hataları tanıyabilen veya insan görünümünü analiz edebilen modeller. Günlük hayattan bir benzetmeyle: bir fabrikanın ya da bir caddenin video-oyunu kalitesinde sanal bir kopyasını kurup, gerçek robotu veya otonom aracı sokağa çıkarmadan önce o sanal kopyada binlerce kez "prova ettirmek" gibi düşünebilirsin — gerçek dünyada test etmek pahalı, yavaş ve bazen tehlikeli; sanal ikizde ise ucuz, hızlı ve güvenli. Aşağıdaki 8 skill, bu iş akışının farklı parçalarını kapsıyor: gerçek bir nesneyi (CAD çizimini) simülasyona hazır 3D modele çevirmekten, o sahneyi gerçek zamanlı görüntülemeye, sahne performansını iyileştirmeye, yapay hata görselleri üretmeye, bulut altyapısı kurmaya, videodan 3D sahne çıkarmaya, insan görüntüsü veri setini çoğaltmaya ve video veri setini çoğaltmaya kadar uzanıyor. Skill'in kendisi bu işleri YAPMIYOR — Claude Code gibi bir yapay-zekâ kod asistanına "bu işi hangi sırayla, hangi komutlarla, hangi ön koşullarla doğru yaparsın" bilgisini veren resmi NVIDIA talimat paketi bu. Yani skill bir tarif kitabı; asistan o tarife bakarak doğru kodu/komutu yazıyor.

omniverse-cad-to-simready

Ne yapar: Bir CAD (bilgisayar destekli tasarım — mühendislerin ürün/parça çizmek için kullandığı yazılım) dosyasını ya da herhangi bir kaynak 3D modeli, simülasyona hazır ("SimReady") bir USD varlığına dönüştüren uçtan uca süreci yönetir. Dönüştürme, malzeme ve fizik özelliklerinin atanması, SimReady standartlarına uygunluk denetimi, doğrulama ve isteğe bağlı paketleme adımlarını sırasıyla birbirine bağlar. Tek dev bir komutla değil, var olan alt-adım tariflerini doğru sırada çalıştırarak ilerler; genelde önce bir "ön kontrol" (preflight) adımıyla ortamın hazır olduğunu doğrular.

Örnek: Claude'a şöyle dersin: "Bu CAD dosyasını simülasyona hazır bir USD paketine çevir, fizik özelliklerini ata ve doğrula." — skill devreye girer, önce ön kontrolü çalıştırır, sonra dönüştürme, malzeme/fizik atama, uygunluk denetimi ve paketleme adımlarını sırayla yürütür.

omniverse-realtime-viewer

Ne yapar: Omniverse tabanlı, gerçek zamanlı (kullanıcı etkileşimine anında tepki veren) bir 3D görüntüleyici (viewer) uygulaması inşa etme isteklerini doğru teknik tarife yönlendiren üst-düzey bir "trafik polisi" skill'i. İstenen görüntüleyici tipini sınıflandırır, kamera, klavye/fare girdisi, nesne seçimi, ekran görünümü (viewport), veri akışı (streaming) protokolü, sahne yükleme ve ortam davranışıyla ilgili doğru referans belgeleri okur; önce görüntüleme (render) altyapısını kurar, sonra etkileşim ve arayüz davranışını üstüne ekler, son olarak sonucu test edip kanıtlar.

Örnek: Claude'a şöyle dersin: "Bir USD sahnesini web tarayıcısında gerçek zamanlı gösterecek bir viewer uygulaması yaz, kullanıcı fareyle kamerayı döndürebilsin." — skill devreye girer, doğru referans dokümanlarını seçip önce render/veri-akışı altyapısını, sonra kamera ve fare etkileşimini doğru sırayla kodlatır.

omniverse-usd-performance-tuning

Ne yapar: Bir USD sahnesi yavaş açılıyorsa, çok bellek (RAM/VRAM) tüketiyorsa, FPS (saniyede gösterilen kare sayısı — akıcılığın ölçüsü) düşükse ya da ekran kartı çöküyorsa devreye giren performans teşhis ve iyileştirme skill'i. Zorunlu bir ortam/çalışma-zamanı kontrolünden başlar, sonra sahneyi adım adım ölçüp (önce/sonra karşılaştırmalı) darboğazı bulan ve düzelten bir zincir kurar; varsayılan olarak 3 turluk kapsamlı bir iyileştirme döngüsü önerir, kullanıcı isterse daha

hızlı/tek geçişlik bir yol da seçilebilir.

Örnek: Claude'a şöyle dersin: "Bu Omniverse sahnesi açılırken çok yavaş yükleniyor ve bellek taşıyor, optimize et." — skill devreye girer, önce mevcut performansı ölçer (baseline), hangi varlık/malzemenin soruna yol açtığını bulur, düzeltir ve öncesi/sonrası karşılaştırmalı bir rapor çıkarır.

physical-ai-defect-image-generation

Ne yapar: Gerçek hatalı ürün fotoğrafı toplamanın pahalı ve yavaş olduğu üretim hattı kalite kontrolü (AOI — Otomatik Optik İnceleme) için YAPAY (üretilmiş) hatalı ürün görselleri oluşturma sürecini yönetir. NVIDIA'nın Cosmos AnomalyGen adlı görüntü üretim modelini (devre kartı/PCBA, metal yüzey, cam için ayrı ayrı eğitilmiş sürümleri var) kullanarak çizik, leke, kırık gibi kusurları simüle eden görseller üretir. İki akışı vardır: "Day 0" — hiç örnek yokken 3D modelden (USD) başlayarak sıfırdan bir başlangıç veri seti kurmak; "Day 1" — artık gerçek görüntüler üzerinde çıkarım (inference) yapıp etiketlemek.

Örnek: Claude'a şöyle dersin: "Devre kartı hattımız için yapay hatalı görüntü veri seti oluşturmam lazım, hiç örneğim yok." — skill devreye girer, Day 0 akışını seçer: 3D modelden gerçekçi kusurlu görüntüler üretir ve ilk eğitim veri setini kurar.

physical-ai-infrastructure-setup-and-resilient-scaling

Ne yapar: Fiziksel-AI sentetik veri üretimi (SDG — Synthetic Data Generation, yani gerçek veri yerine yapay/simüle veri üretme) için gereken bilgisayar altyapısını kurar: yerel Kubernetes kümesi (MicroK8s) ya da Azure bulut kümesi (AKS), model çalıştırma (inference) uç noktaları, OSMO adlı iş dağıtım/orkestrasyon sistemi ve bunların ölçeklendirilmesi/dayanıklı hale getirilmesiyle ilgilenir. Bir sorun çıktığında rastgele geçici yama atmak yerine "en alt sorumlu katmandan" başlayarak (önce ayar dosyası, sonra script, sonra skill rehberi) kök nedeni bulup kalıcı, repo'ya işlenmiş bir düzeltmeyle onarır — yani iz bırakmayan, tekrar bozulacak geçici çözümlerden kaçınır.

Örnek: Claude'a şöyle dersin: "Azure'da fiziksel-AI veri üretimi için bir Kubernetes altyapısı kurmam lazım, model sunucularını da bağla." — skill devreye girer, hangi bileşenlerin (küme, inference, OSMO) gerekli olduğunu belirler ve doğrulanabilir, tekrarlanabilir bir kurulum zinciri işletir.

physical-ai-neural-reconstruction

Ne yapar: Bu bir "ince yönlendirici" (thin router) — kendi başına iş yapmıyor, NVIDIA'nın NuRec/NRE (Neural Reconstruction — sinirsel yeniden inşa) teknolojisiyle ilgili istekleri doğru uzman kaynağa yönlendiriyor. NuRec, bir VIDEODAN 3D bir sahneyi yeniden kurma teknolojisi (3D Gaussian Splatting/3DGS gibi yöntemlerle) — yani gerçek bir mekânın videosunu çekip ondan gezilebilir, sanal bir 3D kopya üretiyorsun. Bu skill; USD render, NCore veri dönüşümü, sensör simülasyonu (gRPC üzerinden), nesne toplama (asset-harvester) ve veri seti indirme gibi konularda hangi "kardeş" skill'in devreye girmesi gerektiğine karar verir; SimReady paketleme veya genel USD performans işleriyle karıştırılmaması gerektiğini de belirtir.

Örnek: Claude'a şöyle dersin: "Bir sokağın videosundan 3D bir sahne çıkarıp otonom araç simülasyonunda kullanmak istiyorum." — skill devreye girer, bu isteğin NuRec ailesinde hangi adıma (veri dönüşümü, eğitim, render, sensör simülasyonu) ait olduğunu belirleyip doğru kaynağa yönlendirir.

physical-ai-people-attribute-search

Ne yapar: "İnsan özneliği arama" (PAS — Person Attribute Search, yani "kırmızı ceketli, uzun boylu kişi" gibi özelliklere göre görüntü arama) için görüntü çoğaltma ve otomatik etiketleme iş akışını yönetir. Var olan insan-kırpma (fotoğraftan kesilmiş kişi) görsellerini alıp farklı kıyafet/görünüm varyasyonları (görsel tarafta) ve bunlara karşılık gelen öznelik açıklamaları (metin tarafta) üreterek veri setini büyütür; bu tür genişletilmiş veri setleri kişi yeniden-tanıma (re-identification — aynı kişiyi farklı kamera görüntülerinde tanıma) modellerinin eğitiminde

kullanılır. Akış seçimi, ön koşul kontrolü, iş gönderimi, izleme ve sonuç indirmeyi kapsar.

Örnek: Claude'a şöyle dersin: "Elimdeki insan fotoğrafı veri setini farklı kıyafet varyasyonlarıyla çoğaltıp etiketle." — skill devreye girer, gerekli anahtar/token'ların hazır olup olmadığını kontrol eder, OSMO üzerinde çoğaltma ve etiketleme işini gönderir, sonucu indirir.

physical-ai-video-data-augmentation

Ne yapar: Video veri çoğaltma ve otomatik ("sözde"/pseudo) etiketleme iş akışını yönetir — az sayıda gerçek video örneğinden daha büyük ve çeşitli bir eğitim veri seti türetir. OSMO (iş dağıtım sistemi) üzerinde akış seçimi, ön koşul kontrolü, önbellek hazırlığı, hangi model-çalıştırma yolunun kullanılacağına karar verme, iş gönderimi, izleme ve sonuç indirmeyi kapsayan uçtan uca bir orkestrasyon sağlar. Bileşen skill'leri sadece danışma amaçlıdır; asıl karar ve yürütme bu skill'de toplanır.

Örnek: Claude'a şöyle dersin: "Bu video klasöründeki ham görüntüleri çoğaltıp otomatik etiketle, eğitim için hazırla." — skill devreye girer, gerekli kimlik bilgilerini (ör. Hugging Face token) kontrol eder, OSMO'ya işi gönderir ve etiketlenmiş çıktıyı teslim eder.

Diğer Aileler (RAG, AI-Q, Dynamo, Megatron-Core ve Tekil Araçlar)

Bu bölümdeki 27 skill dört ana aileye ve bir avuç bağımsız (tekil) araca ayrılıyor.

RAG (Retrieval-Augmented Generation — "getirerek üretme"), bir yapay zekâ modelinin cevap üretmeden önce kendi belge/döküman havuzunuzda arama yapıp ilgili parçaları bulup cevaba katması demek; yani "belgelerine soru sorma sistemi" — kendi PDF'lerinize, raporlarınıza soru sorduğunuzda devreye giren mimari. **AI-Q**, NVIDIA'nın "derin araştırma ajanı" — tek bir arama yerine çok adımlı, çok kaynaklı araştırma yapıp sentezlenmiş bir cevap üreten bir sistem (bir nevi otomatik araştırma asistanı).

Dynamo, eğitilmiş bir modeli gerçek kullanıcı trafiğine açacak şekilde sunmaya/ölçeklemeye yarayan altyapı — yani "modeli nasıl hızlı ve güvenilir şekilde binlerce isteğe cevap verecek hale getiririz" sorusunun çözümü. ****Megatron-Core (mcore)****, NVIDIA'nın çok büyük dil modellerini (milyarlarca parametre) birden fazla GPU/makineye dağıtarak eğitmeye yarayan motor — büyük model eğitiminin "ağır sanayi" kısmı. Bu dört ailenin dışında kalan skill'ler ise tek başına duran, belirli bir işe odaklanmış araçlar: sentetik veri/görüntü üretimi, tıbbi görüntü segmentasyonu, pekiştirmeli öğrenme eğitimi, skill kataloğu yönlendirme ve skill dokümantasyonu gibi. Unutulmaması gereken önemli nokta: bu skill'lerin hiçbiri kendi başına "iş yapmıyor" — her biri Claude Code gibi bir yapay-zekâ kod asistanına yüklenen bir **talimat paketi**. Yani skill, asistana "bu işi NVIDIA'nın önerdiği doğru şekilde nasıl yaparım, hangi komutu hangi sırada çalıştırırım, hangi tuzaklara dikkat ederim" bilgisini veriyor. Asıl işi (komutu çalıştırmak, dosyayı düzenlemek) yine asistan yapıyor — skill sadece resmi kılavuzu elinin altına koyuyor.

aiq-deploy

Ne yapar: AI-Q Blueprint (NVIDIA'nın derin araştırma sistemi) sunucusunu yerel makinede veya kendi sunucunuzda kurar, ayağa kaldırır ve sağlıklı çalıştığını doğrular. Docker/Compose, yerel Python süreci, tarayıcı arayüzü (Node.js) veya Kubernetes/Helm gibi birden fazla kurulum yolunu destekler. Kurulum/başlatma/durdurma/sorun giderme işini üstlenir — asıl araştırma sorgusunu çalıştırmaz, bu iş aiq-research'e devredilir.

Örnek: Claude'a "AI-Q araştırma sistemini bilgisayarımda ayağa kaldır" dersin — skill Docker Compose ile sunucuyu kurar, sağlık kontrolü yapar ve hazır olduğunda hangi adreste çalıştığını söyler.

aiq-research

Ne yapar: Zaten çalışır durumda olan bir AI-Q Blueprint sunucusuna bağlanıp derin araştırma sorguları gönderir. "Şu konuda derin araştırma yap", "AI-Q ile araştır" gibi istekler için kullanılır; kurulum, başlatma, durdurma gibi altyapı işleriyle ilgilenmez — o aiq-deploy'un görevi.

Örnek: Sunucu zaten çalışıyorsa Claude'a "AI-Q ile 'batarya teknolojisindeki son gelişmeler' konusunu araştır" dersin — skill sorguyu sunucuya gönderir, çok kaynaklı derin araştırma sonucunu senin için sentezleyip getirir.

data-designer

Ne yapar: NVIDIA'nın Data Designer kütüphanesini kullanarak sentetik (yapay, gerçek veriye benzeyen ama gerçek kişi/olay içermeyen) veri seti üretir. Kullanıcıya sorular sorarak adım adım ilerleyen "Interactive" modu veya "sen karar ver, ben makul varsayımlarla ilerleyeyim" diyen "Autopilot" modu vardır. Üretilen tüm sütunları varsayılan olarak korur; bir sütunu sadece açıkça istenirse veya sadece ara hesaplama için var olan yardımcı bir sütunsa siler.

Örnek: Claude'a "1000 satırlık sahte müşteri şikayeti veri seti oluştur, ürün adı ve şikayet kategorisi olsun" dersin — skill gerekirse birkaç netleştirici soru sorar, sonra tabloyu üretir.

dynamo-interconnect-check

Ne yapar: Dynamo (model sunum/ölçekleme sistemi) dağıtımında GPU'lar arasındaki veri taşıma hattının — NIXL/UCX/NCCL denen, donanımlar arası çok hızlı veri aktarım katmanlarının, RDMA/NVLink gibi fiziksel bağlantılar üzerinden — gerçekten doğru çalışıp çalışmadığını salt-okunur biçimde kontrol eder. Bir dağıtım "uç nokta cevap veriyor" testini geçebilir ama arka planda sessizce yavaş/bozuk bir yola düşmüş olabilir; bu skill tam da bunu yakalamak için var. Kümede hiçbir şeyi değiştirmez, sadece teşhis eder.

Örnek: "Disaggregated serving" (sunucu görevlerinin ayrı makinelere bölündüğü dağıtım modu) kurulduktan sonra Claude'a "bağlantı gerçekten RDMA üzerinden mi gidiyor kontrol et" dersin — skill worker makinelere bağlanıp donanım/bağlantı durumunu kontrol eder ve sorun varsa raporlar.

dynamo-recipe-runner

Ne yapar: Var olan hazır NVIDIA Dynamo Kubernetes "tarifleri"ni (recipe — önceden hazırlanmış dağıtım şablonları) seçer, doğrular, gereken en küçük düzenlemeleri yapar ve dağıtır. Hangi model, hangi çıkarım motoru (vllm/sglang/trtllm gibi), hangi GPU tipi/sayısı ve hangi dağıtım moduna göre en uygun tarifi bulur. Dağıtım bitince OpenAI-uyumlu bir test isteği göndererek gerçekten çalıştığını kanıtlar.

Örnek: Claude'a "Llama modelini vLLM ile 2 GPU üzerinde Kubernetes'e kur" dersin — skill uygun tarifi bulur, gerekli dosyaları düzenler, dağıtır ve bir test isteğiyle çalıştığını doğrular.

dynamo-router-starter

Ne yapar: Dynamo'nun istekleri worker'lara dağıtma (routing) modlarını başlatır veya ayarlarını değiştirir: round-robin (sırayla dağıtım), KV-aware (önbelleği bilerek yönlendirme), least-loaded (en boş olana gönderme), device-aware (donanıma göre ağırlıklandırma) gibi. Kurulumdan sonra uç noktanın gerçekten çalıştığını basit bir testle kanıtlar.

Örnek: Claude'a "isteklerimi her zaman en az yüklü worker'a gönder" dersin — skill least-loaded yönlendirme modunu başlatır ve çalıştığını test eder.

dynamo-troubleshoot

Ne yapar: Bozuk veya sağlıklı bir Dynamo dağıtımını teşhis eder. Pod'lar, model-önbellek işleri, depolama birimleri (PVC), worker'lar, önyüz/router sağlığı gibi katmanları sırayla, gizli bilgilere (secrets) dokunmadan, salt-okunur kanıt toplayarak inceler. Sorunu net bir kategoriye (küme sorunu / ad-alanı sorunu / pod sorunu / uygulama katmanı sorunu gibi) oturtup somut bir sonraki adım önerir.

Örnek: Dağıtım çöktüğünde Claude'a "Dynamo neden çalışmıyor, bul" dersin — skill salt-okunur bir kanıt paketi toplar, hatayı sınıflandırır ve "şu worker pod'u bellek yetersizliğinden düştü, çözüm şu" der.

launch-nemo-rl

Ne yapar: NeMo-RL (NVIDIA'nın pekiştirmeli öğrenmeyle — deneme-yanılma ve ödül sinyaliyle model eğitime yöntemiyle — çalışan çerçevesi) tariflerini bir Kubernetes kümesinde `nr1-k8s` komut satırı aracıyla başlatma, izleme, durdurma ve hata ayıklama rehberidir. İki mod sunar: "ephemeral" (tek seferlik — iş bitince küme otomatik söner) ve "long-lived" (kalıcı — geliştirme döngüsü boyunca küme açık kalır). Küme paylaşımlı ve yanlış bir komutun maliyeti yüksek olduğundan, her adımdan önce kümenin ve kod deposunun gerçek durumunu kontrol etmeyi şart koşar.

Örnek: Claude'a "şu NeMo-RL eğitim tarifini kümede tek seferlik çalıştır" dersin — skill önce mevcut durumu kontrol eder, sonra `nr1-k8s run` komutuyla işi başlatır ve bitince kümenin otomatik kapandığını doğrular.

mcore-create-issue

Ne yapar: Megatron-LM deposunda (Megatron-Core'un GitHub kod deposu) başarısız olmuş bir GitHub Actions (otomatik test/derleme) çalıştırmasını inceler, kök nedeni çıkarır ve düzgün biçimlendirilmiş bir hata kaydı (GitHub issue) açar.

Örnek: CI (sürekli entegrasyon testleri) kırmızı yandığında Claude'a "şu GitHub Actions linkini incele ve bir issue aç" dersin — skill logları çeker, hatanın kök nedenini bulur ve yapılandırılmış bir issue oluşturur.

mcore-linting-and-formatting

Ne yapar: Megatron-LM projesinde kod biçimlendirme ve stil denetimi (linting — otomatik kod kalite/stil tarayıcısı) için gereken araçları (ruff, black, isort, pylint, mypy) ve komutları anlatır. PR (pull request — kod birleştirme isteği) açmadan önce çalıştırılması gereken `autoformat.sh` betiğini ve kod stil kurallarını (tip belirteçleri, docstring biçimi, isimlendirme kuralları, satır uzunluğu sınırı) içerir.

Örnek: Claude'a "bu Megatron-LM koduna PR açmadan önce format kontrolü yap" dersin — skill `autoformat.sh`'i doğru bayraklarla çalıştırır, importları `isort` ile düzenler ve stil kurallarına uyup uymadığını kontrol eder.

mcore-run-on-slurm

Ne yapar: Megatron-LM ile dağıtık (birden fazla makineye yayılmış) eğitim işlerini bir SLURM kümesinde (büyük hesaplama kümelerinde iş sırasını yöneten sistem) nasıl başlatacağını anlatır. Minimal bir sbatch betik iskeleti, `torch.distributed.run`

için ortam değişkeni ayarları, donanım/paralellik moduna göre değişen CUDA_DEVICE_MAX_CONNECTIONS kuralları, konteyner kullanım kuralları, izleme ve düğüm bazlı hata teşhisini kapsar.

Örnek: Claude'a "bu modeli 4 makine, makine başına 8 GPU ile SLURM'da eğit" dersin — skill doğru sbatch betiğini, ortam değişkenlerini (MASTER_ADDR, WORLD_SIZE vb.) ve doğru başlatma komutunu hazırlar.

mc core-split-pr

Ne yapar: Büyük bir pull request'i (kod değişikliği isteği), gereken CODEOWNERS (dosya sahipliği/inceleiyici grupları) sayısını azaltacak şekilde daha küçük, bağımsız PR'lara böler. Testleri ait oldukları koddan koparmaz; PR'lar arasında bağımlılık varsa (örneğin bir isim değişikliği) bunu açıkça belirtip geriye-uyumlu köprüler ekler. Bölme planını uygulamaya geçirmeden önce kullanıcı onayı bekler.

Örnek: Claude'a "bu büyük PR'ı çok fazla ekip inceliyor, bunu bölelim" dersin — skill dosyaları hangi ekiplerin incelediğine göre analiz eder, "şu 3 PR'a bölünebilir" planını sunar, onay alınca taslak (draft) PR'ları oluşturur.

mc core-testing

Ne yapar: Megatron-LM'nin test sistemini anlatır: birim testlerinin (unit_tests — tek makine/çoklu GPU ile çalışan hızlı testler) ve uçtan-uca fonksiyonel testlerin (functional_tests) klasör düzenini, tarif (recipe) YAML formatını, "golden values" denen (beklenen doğru referans sonuçlar) kavramını ve bir testi silmeden nasıl geçici olarak devre dışı bırakılacağını (özel etiketleme ile) kapsar.

Örnek: Claude'a "şu testi geçici olarak kapat ama silme" dersin — skill fonksiyonel bir test için YAML'daki scope alanına -broken eki eklemeyi, birim testi için ise @pytest.mark.flaky etiketini kullanmayı gösterir.

nv-generate-ct-rflow

Ne yapar: NV-Generate-CTMR modelinin "rflow-ct" modunu kullanarak sentetik (yapay, gerçek hastadan gelmeyen) BT (bilgisayarlı tomografi) hacimleri ve segmentasyon maskeleri üretir. Üretim eğitim verisi olarak incelemenden kullanılmaması gereken bir araçtır; resmi NVIDIA sarmalayıcı (wrapper) betiğini olduğu gibi çalıştırır, kendi çıkarım kodunu yazmaz.

Örnek: Claude'a "test amaçlı birkaç sentetik akciğer BT taraması üret" dersin — skill ilgili betiği doğru ayarlarla çalıştırır ve çıktı hacimlerini/etiket haritalarını kaydeder.

nv-generate-mr

Ne yapar: Aynı NV-Generate-CTMR ailesinin "rflow-mr" modunu kullanarak sentetik vücut MR (manyetik rezonans) görüntüleri üretir; eşlik eden maske üretmez. Resmi sarmalayıcıyı olduğu gibi kullanır; üretim veya klinik veri onayı için değildir.

Örnek: Claude'a "test amaçlı birkaç sentetik karın MR hacmi üret" dersin — skill ilgili betiği çalıştırır ve sentetik hacimleri kaydeder.

nv-generate-mr-brain

Ne yapar: Aynı ailenin "rflow-mr-brain" modunu kullanarak özellikle beyin MR görüntüleri üretir. Üretim verisi için değildir, sadece deneme/test amaçlıdır.

Örnek: Claude'a "modelimi test edeceğim, birkaç sentetik beyin MR'ı örneği üret" dersin — skill ilgili betiği çalıştırıp sentetik hacimleri üretir.

nv-generate-mr-brain-finetune

Ne yapar: NV-Generate-CTMR'nin beyin-MR difüzyon modelini (rflow-mr-brain), kullanıcının kendi NIFTI (tıbbi görüntü dosya formatı) eğitim verisiyle ince ayar (finetune — hazır bir modeli yeni veriyle biraz daha eğitip ona özelleştirme) yapar. Klinik yorumlama veya üretim verisi onayı için kullanılmaz; sadece upstream (kaynak NVIDIA reposundaki) eğitim/çıkartım betiklerini sarmalar, kendi eğitim kodunu yazmaz.

Örnek: Claude'a "kendi beyin MR veri setimle bu difüzyon modelini ince ayar yap" dersin — skill eğitim verisi listesini alır, gerekli yapılandırma dosyalarını hazırlar ve eğitim betiğini çalıştırıp ince ayarlı model çıktısını üretir.

nv-generate-vae-finetune

Ne yapar: NV-Generate-CTMR'nin MAISI VAE'sini (autoencoder — bir görüntüyü sıkışık bir temsile indirip geri açan sinir ağı bileşeni) kullanıcının kendi BT/MR NIFTI verisiyle ince ayar yapar. Klinik onay için değildir; şu an için resmi bir hazır komut satırı aracı olmadığından, gereken yapılandırma/veri listesi dosyalarını arka planda hazırlayıp üst düzey (kaynak depodaki) yardımcı fonksiyonları çağırır.

Örnek: Claude'a "VAE modelini kendi BT veri setimle ince ayar yap" dersin — skill gerekli yapılandırma dosyalarını üretir ve eğitilmiş model/ayırıcı (discriminator) kontrol noktalarını (checkpoint) kaydeder.

nv-reason-cxr

Ne yapar: NV-Reason-CXR modeliyle göğüs röntgeni (chest X-ray) görüntüleri üzerinde "akıl yürütme" (reasoning) testleri çalıştırır — komutun doğru biçimde çalıştığını kontrol için veya gerçek bir görüntüyle canlı deneme (smoke test) amaçlı. Tanı koymak veya klinik rapor üretmek için değildir; modelin ürettiği düşünme adımlarını ve nihai cevabı özetlemeden, olduğu gibi raporlar.

Örnek: Claude'a "bu göğüs röntgeni görüntüsünü modele ver, ne diyor bak (test amaçlı)" dersin — skill betiği çalıştırır ve modelin tam çıktısını (düşünme süreci + cevap) gösterir.

nv-segment-ct

Ne yapar: NV-Segment-CT (VISTA3D modeli) ile BT NIFTI hacimlerinde organ/doku segmentasyonu yapar — yani görüntüdeki farklı yapıları (karaciğer, böbrek gibi) otomatik olarak sınırlarıyla ayırır — ve hangi etiketin hangi bölgeye karşılık geldiğini kanıt olarak kaydeder. Klinik yorumlama için değildir.

Örnek: Claude'a "bu BT taramasında karaciğer ve böbrekleri segmentle" dersin — skill ilgili betiği çalıştırır ve etiket haritasını (label map) üretir.

nv-segment-ct-finetune

Ne yapar: NV-Segment-CT (VISTA3D) modelini kullanıcının kendi etiketli BT Nifti verisiyle ya çok kısa bir "duman testi" (smoke — hızlı doğrulama denemesi) ile ya da tam veri setiyle ince ayar yapar. Klinik doğrulama için değildir; kaynak MONAI paketinin orijinal giriş noktasını sarmalar, elle yazılmış eğitim kodu kullanmaz.

Örnek: Claude'a "VISTA3D'yi kendi böbrek-etiketli BT verimle hızlı test amaçlı ince ayar yap" dersin — skill --smoke bayrağıyla kısa bir deneme çalıştırır ve sonucu doğrular.

nv-segment-ctmr

Ne yapar: NV-Segment-CTMR ile hem BT hem MR Nifti hacimlerinde segmentasyon yapar ve etiket haritası kanıtını kaydeder — nv-segment-ct'nin hem BT hem MR'ı destekleyen daha genel bir sürümü gibi düşünülebilir. Klinik yorumlama için değildir.

Örnek: Claude'a "bu MR görüntüsünde beyin bölgelerini segmentle" dersin — skill ilgili betiği görüntü türünü (--modality) belirterek çalıştırır.

nvidia-skill-finder

Ne yapar: Kullanıcı açıkça bir "skill" istemese bile, isteği NVIDIA ürünleri/donanımı/yazılımıyla (Jetson, CUDA, NIM, NeMo, Omniverse, RAPIDS, Dynamo, Holoscan, TensorRT, DeepStream, TAO, NGC gibi) ilgiliyse, henüz kurulu olmayan hangi NVIDIA skill'inin işe yarayabileceğini bulan bir katalog/yönlendirme aracıdır. Kendisi asıl işi yapmaz, doğru skill'e ve kurulumuna yönlendirir.

Örnek: Claude'a "Jetson cihazımı nasıl flashlarım" dersin ama ilgili skill henüz kurulu değildir — bu skill devreye girip "bunun için şu NVIDIA skill'i var, kurulumu şöyle" der.

rag-blueprint

Ne yapar: NVIDIA RAG Blueprint'in (belgelerinize soru sorma sistemi) kurulumu, yapılandırması, sorun giderme ve özellik yönetimini kapsayan ana kontrol merkezidir — Agentic RAG (kendi kararlarını verebilen RAG), VLM (görsel-dil modeli ile görsel anlama), guardrails (güvenlik/sınır filtreleri), sorgu yeniden yazma, arama, veri yükleme (ingestion), gözlemlenebilirlik, özetleme, akıl yürütme gibi tüm özellikleri içerir. Docker, Helm (Kubernetes paket yönetimi) ve doğrudan kütüphane olarak üç dağıtım yolunu da destekler; her isteği doğru rehber dosyasına yönlendirir.

Örnek: Claude'a "RAG sistemimde belge özetleme özelliğini aç" veya "RAG neden çöktü, düzelt" dersin — skill isteği doğru playbook'a (rehber belgeye) yönlendirir ve dağıtım yapılandırma dosyaları üzerinden değişikliği uygular.

rag-eval

Ne yapar: NVIDIA RAG Blueprint'in cevap kalitesini ölçer — corpus/ (belge kümesi) ve train.json (soru-cevap test seti) dosya düzeninde RAGAS (RAG kalitesini puanlayan bir metrik sistemi) değerlendirmesi çalıştırır. Arama (retrieval) ve üretim (generation) ayarlarını kalite açısından karşılaştırmak, sonuç JSON'larını yorumlamak ve hataları (HTTP/akış hataları, boş bağlam, koleksiyon uyumsuzluğu, hakem model API sorunları) teşhis etmek için kullanılır. Hız/verim ölçümü bu skill'in işi

değildir — o rag-perf'e ait.

Örnek: Claude'a "RAG sisteminin cevap kalitesini ölç, iki farklı arama ayarını karşılaştır" dersin — skill değerlendirme betiğini çalıştırır, RAGAS puanlarını getirir ve hangi ayarın daha iyi sonuç verdiğini gösterir.

rag-perf

Ne yapar: Zaten çalışır durumdaki bir NVIDIA RAG Blueprint sunucusunun hızını/performansını ölçer — tek bir YAML yapılandırma dosyasıyla sunucu tarafında aşama aşama süre profili çıkarır (hangi adım ne kadar sürüyor, tıkanıklık nerede) ve isteğe bağlı olarak yük testi (ilk-token süresi, uçtan-uca gecikme, saniyedeki istek/token sayısı, hata oranı) çalıştırıp birleşik bir rapor üretir. Cevap kalitesini ölçmez (o rag-eval'ın işi), servis kurmaz/onarmaz (o rag-blueprint'in işi).

Örnek: Claude'a "RAG sunucumun saniyede kaç isteği kaldırdığını ve gecikmesini ölç" dersin — skill tek komutla profil çıkarma ve yük testini çalıştırır, birleşik gecikme/verim raporunu getirir.

skill-card-generator

Ne yapar: Var olan bir NVIDIA skill'i için, o skill'in ne yaptığını, kaynaklarını ve güvenlik/inceleme bilgilerini özetleyen resmi bir "skill kartı" (yönetişim belgesi) üretir veya günceller. Yeni bir skill yazmaz, skill'i açıklamaz/tartışmaz — sadece hukuki/güvenlik incelemesi için gereken standart dokümantasyon kartını hazırlar; insan onayının yerini almaz.

Örnek: Yeni bir NVIDIA skill'i yazıldıktan sonra Claude'a "şu skill için yönetim kartı oluştur" dersin — skill kaynak dosyaları tarar, yapılandırılmış bir bağlam kurar ve standart markdown kartını (insan incelemesi öncesi taslak olarak) üretir.